



UNIVERSITÀ DI PISA

Appunti Progettazione Web

A. Vecchio

Christian Cantavenera | Ingegneria Informatica Unipi

A.A. 2025/26

Indice

0	Introduzione al corso	1
0.1	Informazioni sul Corso e Materiale	1
0.1.1	Contatti del Docente	1
0.1.2	Argomenti e Propedeuticità	1
0.1.3	Materiale e Risorse per l'Esame	1
0.2	Capitolo 1: Come Funziona il Web	1
0.2.1	Internet vs. World Wide Web	1
0.2.2	L'evoluzione della Comunicazione	1
0.2.3	La Nascita del Web e del TCP/IP	2
0.3	L'Architettura di Rete (Modello a 4 Livelli)	2
0.3.1	Indirizzi IP (IPv4 vs IPv6)	2
0.3.2	Il protocollo NAT (Network Address Translation)	2
0.4	L'Infrastruttura Fisica e i Server	3
0.4.1	Server Farms e Data Center	3
0.4.2	Come viaggiano i dati?	3
0.5	DNS, URL e Protocollo HTTP	3
0.5.1	DNS (Domain Name System)	3
0.5.2	Anatomia dell'URL (Uniform Resource Locator)	3
0.5.3	Il Protocollo HTTP	3
1	HTML - Overview(02)	5
1.1	Fondamenti di HTML	5
1.2	Sintassi HTML	5
1.2.1	Elementi e Attributi	5
1.2.2	Gerarchia e Annidamento	6
1.3	Markup Semantico	7
1.4	Struttura del Documento HTML	7
1.4.1	Tag Fondamentali	7
1.4.2	La sezione Head	7
1.5	Elementi di Testo e Contenuto	8
1.5.1	Intestazioni (Headings)	8
1.5.2	Paragrafi e Interruzioni	8
1.6	Link e Navigazione	9
1.6.1	L'attributo href	9
1.6.2	Elemento link nell'intestazione	9
1.7	Gestione dei Percorsi (URL)	9
1.7.1	URL Assoluti	9
1.7.2	URL Relativi	10
1.8	Elementi Testuali Inline (Phrasing Content)	11
1.9	Liste e Caratteri Speciali	11
1.9.1	Liste Non Ordinate (Unordered Lists)	12

1.9.2	Liste Ordinate (Ordered Lists)	12
1.9.3	Liste di Definizione (Definition Lists)	12
1.9.4	Entità Carattere (Character Entities)	13
1.10	Immagini e Mappe Immagine	13
1.10.1	L'elemento img	13
1.10.2	Mappe Immagine (Image Maps)	14
1.11	Architettura e Semantica in HTML5	15
1.11.1	Macro-aree del Layout Strutturale	15
1.11.2	Sezionamento dei Contenuti	15
1.11.3	Elementi Complementari e di Dettaglio	16
1.12	Elementi di Testo Inline (Phrasing Content)	16
1.12.1	Il tag generico span	16
1.12.2	Elementi Semantici Inline	17
1.13	Contenuto Multimediale Embedded	17
1.13.1	L'elemento iframe	17
1.13.2	Riproduzione Video	18
1.13.3	Riproduzione Audio	19
2	CSS - Intro(03)	21
2.1	Introduzione al CSS e Sintassi Base	21
2.1.1	La Sintassi	21
2.2	Posizionamento degli Stili (Location of Styles)	22
2.2.1	Stili Inline	22
2.2.2	Stili Interni (Embedded)	22
2.2.3	Stili Esterni (External)	22
2.3	I Selettori CSS Fondamentali	22
2.3.1	Selettori di Elemento	22
2.3.2	Selettori di Classe	23
2.3.3	Selettori di ID	23
2.4	Selettori Avanzati e Contestuali	23
2.4.1	Selettori Relazionali (Combinators)	23
2.4.2	Pseudo-classi (Stati dell'elemento)	24
2.5	The Cascade (La Cascata) e l'Ereditarietà	24
2.5.1	L'Ereditarietà (Inheritance)	24
2.5.2	La Specificità (Specificity)	24
2.6	Il Box Model (Modello a Scatola)	25
2.6.1	Le 4 componenti della scatola	25
2.6.2	La rivoluzione del box-sizing	26
2.7	Proprietà del Box Model: Sfondi, Bordi e Spaziature	26
2.7.1	Gestione dello Sfondo (Background)	26
2.7.2	Bordi e la Notazione Shorthand (TRBL)	26
2.8	Il Comportamento dei Margini (Collapsing Margins)	27
2.9	Gestione dell'Overflow	28
2.10	Tipografia e Gestione dei Font	28
2.10.1	Font-Family e lo Stack	28
2.10.2	Introduzione a @font-face	29
2.11	L'Incubo delle Unità di Misura: px, em, % e rem	29
2.11.1	Il Problema del Compounding (Effetto a Cascata)	29

2.11.2	La Soluzione Definitiva: L'unità rem	29
3	HTML - Tabelle e Forms(04)	31
3.1	Struttura Base delle Tabelle HTML	31
3.1.1	Unione di Celle (Spanning)	31
3.2	Struttura Semantica Avanzata e Accessibilità	32
3.2.1	Raggruppamento delle Righe	32
3.2.2	Titolo e Gruppi di Colonne	32
3.3	Stilizzazione delle Tabelle (CSS)	33
3.3.1	Il problema dei bordi doppi (border-collapse)	33
3.3.2	Strisce Zebrate (Zebra Striping)	33
3.4	Introduzione ai Form HTML	33
3.4.1	Il contenitore globale: l'elemento form	34
3.4.2	Metodi di Invio: GET vs POST	34
3.5	I Controlli di Input Standard	35
3.5.1	Campi Testuali e Segreti	35
3.5.2	Scelte Multiple e Singole	35
3.5.3	Pulsanti di Azione	35
3.6	Evoluzione: I Tipi di Input in HTML5	35
3.6.1	Input Dati Specifici	36
3.6.2	Controlli Interattivi e Selettori	36
3.7	Elementi Strutturali e Accessibilità	36
3.7.1	Menù a Tendina: select e option	36
3.7.2	Aree di Testo Multilinea: textarea	36
3.7.3	L'importanza vitale del tag label	37
3.7.4	Raggruppamento Logico: fieldset e legend	37
3.8	Controlli Specializzati e Upload File	37
3.8.1	Campi Nascosti (Hidden)	37
3.8.2	Caricamento File (File Upload)	37
3.9	Pulsanti Avanzati (Button Controls)	38
3.9.1	L'elemento button	38
3.9.2	Input di tipo Image	38
3.10	Autocompletamento e Validazione Personalizzata	38
3.10.1	L'elemento datalist	38
3.10.2	Validazione tramite Espressioni Regolari (Pattern)	39
3.11	Elementi di Feedback e Output (HTML5)	39
4	CSS Avanzato - Layout(05)	41
4.1	Il Flusso Normale (Normal Flow)	41
4.1.1	Elementi a Blocco (Block-level) e Inline	41
4.2	Posizionamento degli Elementi (Positioning)	42
4.2.1	Posizionamento Relativo (Relative)	42
4.2.2	Posizionamento Assoluto (Absolute)	42
4.2.3	Posizionamento Fisso (Fixed)	42
4.2.4	L'Indice Z (Z-Index)	43
4.3	Elementi Fluttuanti (Floats)	43
4.3.1	La Proprietà Clear	43
4.3.2	Il Problema del Contenitore (Containing Floats)	43
4.4	Nascondere gli Elementi: Display vs Visibility	44

4.5	Layout Moderni: Introduzione a Flexbox	44
4.5.1	Proprietà del Flex Container (Il Genitore)	44
4.5.2	Allineamento e Distribuzione dello Spazio	45
4.5.3	Proprietà dei Flex Items (I Figli)	45
4.6	CSS Grid Layout (Griglie Bidimensionali)	45
4.6.1	Definire la Struttura (Colonne e Righe)	45
4.6.2	Posizionamento Esplicito delle Celle	46
4.7	Approcci al Layout e Responsive Design	46
4.7.1	I Pilastri del Responsive Design	46
4.8	Filtri, Transizioni e Animazioni	47
4.8.1	Filtri (Filters)	47
4.8.2	Transizioni (Transitions)	48
5	Fondamenti del Web Dev(06/7)	49
5.1	Introduzione a JavaScript	49
5.1.1	Svantaggi dell'Approccio Client-Side	49
5.1.2	Inserimento del Codice in HTML	50
5.2	Variabili e Tipi di Dato	50
5.2.1	Tipi Primitivi	50
5.2.2	Memoria: Primitivi vs Riferimento	51
5.2.3	Dichiarazione delle Variabili: let vs const	51
5.2.4	Interazione Base e Output	52
5.3	Strutture di Controllo e Flusso di Esecuzione	52
5.3.1	Condizionali e Operatore Ternario	52
5.3.2	Iterazioni: Cicli e for...of	53
5.3.3	Gestione degli Errori (try...catch)	53
5.4	Strutture Dati: Array, Oggetti e JSON	53
5.4.1	Array e Destrutturazione	53
5.4.2	Oggetti (Object Literal)	54
5.4.3	JSON (JavaScript Object Notation)	54
5.5	Le Funzioni in JavaScript	55
5.5.1	Parametri: Default e Rest	55
5.5.2	L'Hoisting: Il sollevamento delle variabili	56
5.5.3	Funzioni di Callback	56
5.6	Oggetti, Costruttori e contesto ('this')	56
5.6.1	I Function Constructors	56
5.6.2	Arrow Syntax (Funzioni a Freccia)	57
5.7	Scope e Closures (Concetti Avanzati)	57
5.7.1	Le Closures (Chiusure)	58
5.8	Il Document Object Model (DOM)	59
5.8.1	Proprietà Fondamentali dei Nodi	59
5.9	Accesso agli Elementi del DOM	60
5.9.1	Metodi di Selezione Tradizionali	60
5.9.2	Selettori Moderni (Query Selectors)	60
5.10	Manipolazione del Contenuto e degli Attributi	61
5.10.1	Modifica del Testo e del Markup	61
5.10.2	Gestione degli Attributi	61

5.11	Creazione Dinamica dei Nodi	61
5.11.1	Metodi di Manipolazione Strutturale	61
5.12	DOM Timing: Quando eseguire lo script	63
5.13	Gestione degli Eventi (Event Handling)	63
5.13.1	addEventListener	63
5.13.2	L'Oggetto Event (e)	63
5.14	Propagazione degli Eventi: Capturing e Bubbling	64
5.14.1	Event Delegation (Delegazione degli Eventi)	64
5.15	Accesso ai Dati Custom (Dataset)	65
5.16	Interazione coi Form e Validazione	65
5.16.1	Eventi Chiave dei Form	65
5.16.2	Validazione Client-Side	66
5.16.3	Regular Expressions (Espressioni Regolari)	66
6	JavaScript - Addendum(08)	67
6.1	Variabili e Scope	67
6.1.1	Differenze tra var, let e const	67
6.2	L'Oggetto Globale e la keyword this	68
6.2.1	L'Oggetto Globale	68
6.2.2	Il contesto this	68
6.3	Operatori di Confronto e Type Coercion	68
6.3.1	Strict Equality (=== e !==)	68
6.3.2	Abstract Equality (== e !=)	69
6.4	Funzioni e Dinamismo degli Argomenti	69
6.4.1	Funzioni Predefinite (Globali)	69
6.5	Interazione Base: Finestre di Dialogo	70
6.6	Gestione degli Errori (Eccezioni)	70
6.7	Oggetti: Fondamenti	70
6.7.1	Creazione tramite Object Initializer	71
6.7.2	Accesso alle Proprietà: Dot vs Associative Array Notation	71
6.7.3	Iterazione sulle Proprietà: il costrutto for...in	71
6.8	Costruttori e Prototipi	72
6.8.1	Le Funzioni Costruttore	72
6.8.2	La Catena dei Prototipi (Prototype Chain)	72
6.8.3	Manipolazione sicura dei Prototipi	73
6.9	Oggetti Predefiniti (Built-in Objects): Array	73
6.9.1	Metodi fondamentali degli Array	74
6.10	Oggetti Predefiniti (Built-in Objects): Seconda Parte	74
6.10.1	Boolean e Regole di Coercizione	74
6.10.2	Date	75
6.10.3	Math	75
6.10.4	Number e String	75
6.11	Elementi di JavaScript Moderno (ES6+)	76
6.11.1	Il ciclo for...of	76
6.11.2	Destructuring Assignment (Assegnamento Destrutturato)	76
6.11.3	Template String Literals e Default Parameters	76
6.11.4	Classi e Costruttori	77
6.12	Strict Mode	77

6.13	Arrow Functions (Funzioni a Freccia)	78
6.14	Spread Operator (...)	78
6.14.1	Spread sugli Array e Stringhe	78
6.14.2	Spread sugli Oggetti	79
7	Intro al lato server - PHP	81
7.1	Introduzione allo Sviluppo Server-Side	81
7.1.1	Architettura Client vs Server	81
7.1.2	Tecnologie Server-Side Comuni	81
7.2	Il Web Server e l'Ambiente di Esecuzione	82
7.2.1	Responsabilità del Web Server	82
7.2.2	Architettura LAMP e Apache	82
7.2.3	Internals di PHP	83
7.3	Fondamenti e Sintassi di PHP	83
7.3.1	Tag e Integrazione con HTML	83
7.3.2	Variabili, Costanti e Commenti	84
7.3.3	Output e Manipolazione delle Stringhe	84
7.4	Strutture di Controllo e Inclusione File	85
7.4.1	Condizionali e Cicli	85
7.4.2	L'Inclusione dei File (Include e Require)	85
7.5	Funzioni e Ambito delle Variabili	85
7.5.1	Definizione e Sintassi Base	86
7.5.2	Parametri e Valori di Default	86
7.5.3	Passaggio per Valore vs Passaggio per Riferimento	86
7.6	Lo Scope (Ambito) delle Variabili	87
7.6.1	Function Scope	87
7.6.2	La Keyword <code>global</code>	87
8	PHP - Arrays e Superglobals	89
8.1	Array in PHP	89
8.1.1	Chiavi e Valori	89
8.1.2	Array Multidimensionali	89
8.1.3	Iterazione degli Array	90
8.1.4	Operazioni Comuni e Ordinamento	90
8.2	Array Superglobali (<code>\$_GET</code> , <code>\$_POST</code> , <code>\$_SERVER</code>)	90
8.2.1	<code>\$_GET</code> e <code>\$_POST</code>	90
8.2.2	L'array <code>\$_SERVER</code>	91
8.3	Gestione degli Upload: L'Array <code>\$_FILES</code>	91
8.3.1	Requisiti del Form HTML	91
8.3.2	Struttura di <code>\$_FILES</code>	92
8.3.3	Spostare il File Definitivamente	92
8.4	Interazione con il File System (I/O)	92
8.4.1	Stream Access (Accesso Sequenziale)	92
8.4.2	All-In-Memory Access	93
9	PHP - Classi e Oggetti	95
9.1	Programmazione Orientata agli Oggetti (OOP) e UML	95
9.2	Definizione di Classi e Oggetti in PHP	95
9.2.1	Visibilità e Incapsulamento	96

9.2.2	Il Riferimento all'Oggetto Corrente: \$this	96
9.2.3	Costruttori	96
9.3	Funzionalità Avanzate OOP in PHP	97
9.3.1	Membri Statici (Static)	97
9.3.2	Ereditarietà (Inheritance)	97
9.3.3	Classi Astratte e Interfacce	98
9.3.4	Polimorfismo e Type Checking (instanceof)	98
10	Databases(12)	99
10.1	Interazione tra PHP e Database	99
10.1.1	Il Ruolo dei Database nello Sviluppo Web	99
10.1.2	Transazioni e Integrità dei Dati	99
10.1.3	API PHP per la Gestione dei Database	100
10.2	Il Ciclo di Vita dell'Interazione con il Database	100
10.2.1	Connessione e Gestione degli Errori	101
10.2.2	Esecuzione delle Query	101
10.3	Sicurezza: Prevenzione e Prepared Statements	102
10.3.1	La vulnerabilità SQL Injection	102
10.3.2	I Prepared Statements (Istruzioni Prepare)	102
10.4	Elaborazione dei Risultati (Result Set)	103
10.4.1	Fetching con PDO	103
10.4.2	Fetching con MySQLi (Prepared Statements)	104
10.5	Chiusura della Connessione e Rilascio Risorse	104
10.6	Gestione di File e Immagini con il Database	104
10.6.1	1. Salvataggio del Percorso (Consigliato)	104
10.6.2	2. Salvataggio come Dati BLOB (Binary Large Object)	105
11	Validazione(13)	107
11.1	Validazione dei Dati	107
11.1.1	Controllo dell'Input in PHP: isset() vs empty()	107
11.1.2	Criteri di Validazione Lato Server	107
11.1.3	Validazione Lato Client con HTML5	108
11.1.4	Esempi Pratici di Validazione PHP	108
11.1.5	Espressioni Regolari (Regex) e preg_match()	109
11.1.6	Best Practice: Architettura della Validazione a "Early Exit"	109
12	PHP - Gestione degli errori(14)	111
12.1	Gestione degli Errori e delle Eccezioni in PHP	111
12.1.1	Tipologie di Problemi e Classificazione	111
12.1.2	Configurazione del Reporting (Sviluppo vs Produzione)	111
12.1.3	Eccezioni e il Costrutto Try-Catch	111
12.1.4	La Rivoluzione di PHP 7: L'Interfaccia Throwable	112
13	JavaScript - Operazioni Asincrone(15)	113
13.1	Asincronia in JavaScript	113
13.1.1	Callbacks ed Eventi	113
13.2	AJAX e XMLHttpRequest	113
13.2.1	L'approccio AJAX	113
13.2.2	L'oggetto XMLHttpRequest	115

13.3	Promises e Fetch API	116
13.3.1	Le Promises	116
13.3.2	La Fetch API	117
13.4	Async/Await	117
13.4.1	La keyword async	118
13.4.2	La keyword await	118
13.5	Il Modello di Esecuzione in JavaScript	118
13.5.1	L'Event Loop e la Gestione dei Task	118
13.5.2	Sincronicità e Blocco dell'Interfaccia	119
14	Canvas(17)	121
14.1	HTML5 Canvas: Fondamenti	121
14.1.1	Inizializzazione e Supporto	121
14.2	Disegno di Percorsi e Trasformazioni	121
14.2.1	Gestione dei Path	121
14.2.2	Trasformazioni del Contesto spaziale	122
14.2.3	Isolamento dello Stato: Save e Restore	123
14.3	Stili e Rappresentazione Visiva	123
14.3.1	Personalizzazione di Tratti e Riempimenti	123
14.3.2	Disegno di Curve (Quadratic Curves)	123
14.4	Immagini, Pattern e Gradienti	124
14.4.1	Inserimento di Immagini	124
14.4.2	Pattern di Sfondo	124
14.4.3	Transizioni: I Gradienti	125
14.5	Trasformazioni Matematiche Avanzate	125
14.5.1	Scaling e Comportamento dell'Origine	125
14.5.2	La Matrice di Trasformazione (transform)	125
14.6	Testo e Ombreggiature	126
14.6.1	Renderizzare il Testo	126
14.6.2	Ombre (Shadows)	126
14.7	Animazioni ed Eventi Interattivi	126
15	Gestione stato(18)	129
15.1	Gestione dello Stato nelle Applicazioni Web	129
15.1.1	Il Problema dello Stato	129
15.1.2	Metodi di Trasferimento delle Informazioni	129
15.1.3	I Cookie (Approccio Client-Side)	130
15.2	Serializzazione degli Oggetti	131
15.2.1	Concetto e Funzionamento	131
15.2.2	L'interfaccia Serializable	131
15.3	Session State (Stato di Sessione)	132
15.3.1	Funzionamento Interno	132
15.3.2	Sessioni in Ambienti Distribuiti (Web Farm)	133
15.4	Web Storage HTML5	133
16	Accessibilità(19)	135
16.1	Introduzione all'Accessibilità Web	135
16.1.1	I Principi delle WCAG 2.2 (POUR)	135

16.2	WAI-ARIA (Accessible Rich Internet Applications)	136
16.2.1	Ruoli (Roles)	136
16.2.2	Proprietà e Stati	137
16.3	Gestione del Focus e Navigazione	138
16.3.1	L'attributo tabindex	138
16.4	ARIA Live Regions	138
16.5	Contesto, Lingua e Strumenti di Analisi	138
16.5.1	Lingua del Documento e Ordine Logico	138
16.5.2	Strumenti per la Verifica dell'Accessibilità (Auditing)	139
17	JavaScript - Funzioni Avanzate(20)	141
17.1	Funzioni Avanzate degli Array	141
17.1.1	Iterazione con forEach	141
17.1.2	Ricerca: find e filter	141
17.1.3	Trasformazione con map	142
17.1.4	Aggregazione con reduce	142
17.1.5	Ordinamento Personalizzato con sort	142
17.2	Moduli JavaScript (ES6)	143
17.2.1	Export: Condividere il Codice	143
17.2.2	Import: Integrare il Codice	143
17.3	Web API (Integrate ed Esterne)	144
17.3.1	Web Speech API (Integrata)	144
17.3.2	Geolocation API (Integrata)	144
17.3.3	API Esterne: Google Maps e Plotly.js	144
18	Sicurezza Web(21)	147
18.1	Sicurezza dell'Informazione	147
18.1.1	La Triade CIA	147
18.1.2	Concetti Chiave: Vulnerabilità, Minaccia, Rischio e Attacco	147
18.2	Secure By Design	148
18.2.1	Fasi del Ciclo di Sviluppo Sicuro	148
18.3	Autenticazione e Autorizzazione	148
18.3.1	Fattori di Autenticazione	148
18.3.2	Autorizzazione e Principio del Privilegio Minimo	149
18.3.3	Autenticazione di Terze Parti e OAuth	149
18.4	Crittografia	149
18.4.1	Cifrari Storici e Crittografia Simmetrica	149
18.4.2	Cifrari a Blocchi Moderni	150
18.4.3	Il Problema dello Scambio delle Chiavi e Diffie-Hellman	150
18.4.4	Crittografia a Chiave Pubblica (Asimmetrica) e RSA	150
18.5	Firme Digitali e HTTPS	151
18.5.1	Firme Digitali e Funzioni di Hash	151
18.5.2	HTTPS e Certificate Authorities	151
18.6	Security Best Practices e Attacchi Comuni	152
18.6.1	1. Archiviazione Sicura dei Dati (Password)	152
18.6.2	2. SQL Injection (SQLi)	152
18.6.3	3. Cross-Site Scripting (XSS)	152
18.6.4	4. Insecure Direct Object Reference	153
18.6.5	5. Denial of Service (DoS e DDoS)	153

0 Introduzione al corso

0.1 Informazioni sul Corso e Materiale

0.1.1 Contatti del Docente

- **Docente:** Alessio Vecchio
- **Email:** alessio.vecchio@unipi.it
- **Ufficio:** Dipartimento di Ingegneria dell'Informazione (Largo L. Lazzarino 1, Edificio A, ultimo piano).
- **Ricevimento:** Lunedì 8:30-10:30 (previo appuntamento via email).

0.1.2 Argomenti e Propedeuticità

Il corso richiede **Basi di Dati** come propedeuticità e copre il seguente stack tecnologico:

- HTML 5.0 e CSS (Cascading Style Sheets).
- Programmazione lato Client: JavaScript.
- Programmazione lato Server: PHP.
- Protocollo HTTP e laboratori pratici.

0.1.3 Materiale e Risorse per l'Esame

- **Libro di testo:** “*Fundamentals of Web Development*” (R. Connolly, R. Hoar).
- **Repository Ufficiale (Esami Passati):** <https://github.com/alessiovecchio/PWEB> (Cruciale per la preparazione della prova pratica).
- Le regole dettagliate per l'esame e il progetto finale sono consultabili nei documenti presenti sul Team del corso.

0.2 Capitolo 1: Come Funziona il Web

0.2.1 Internet vs. World Wide Web

Spesso confusi, sono due concetti distinti:

- **Internet:** L'infrastruttura globale di reti interconnesse.
- **Web (WWW):** Solo *uno* dei tanti servizi che viaggiano su Internet (insieme a Email, FTP, gaming online).

0.2.2 L'evoluzione della Comunicazione

1. **Circuit Switching (Commutazione di Circuito):** Tipica delle vecchie reti telefoniche. Crea una connessione fisica dedicata tra mittente e ricevente. *Difetti:* Spreca larghezza di banda (anche i silenzi occupano la linea) ed è vulnerabile (se il circuito cade, la comunicazione si interrompe).

2. **Packet Switching (Commutazione di Pacchetto):** Utilizzata da ARPANET (la prima Internet degli anni '60) e dalle reti moderne. Il messaggio viene frammentato in piccoli "pacchetti" numerati. Ogni pacchetto viaggia in modo indipendente e può seguire percorsi diversi. > È molto più robusta (se un nodo cade, i pacchetti cambiano strada) ed efficiente (la linea è condivisa tra più comunicazioni).

0.2.3 La Nascita del Web e del TCP/IP

- Negli anni '80 i protocolli **TCP/IP** unificano le varie reti globali.
- Nel 1990, **Tim Berners-Lee** (al CERN) inventa il . Sviluppa le fondamenta: URL, HTTP, HTML, il primo Web Server e il primo Browser. Successivamente fonda il **W3C** per gestire gli standard aperti.

Evoluzione dei Siti Web:

- **Statici (Anni '90):** Pagine HTML fisse, uguali per tutti gli utenti. Modificabili solo manualmente dal webmaster.
- **Dinamici:** Pagine generate in tempo reale (es. tramite PHP) interrogando un database. Il contenuto cambia in base all'utente o al contesto.
- **Web 2.0 (Anni 2000+):** L'utente diventa creatore di contenuti. La logica si sposta in parte sul browser (tramite JavaScript), creando interfacce altamente interattive.

0.3 L'Architettura di Rete (Modello a 4 Livelli)

Internet funziona grazie a uno stack di protocolli, semplificabile in 4 livelli:

1. **Link Layer (Livello Collegamento):** Trasmissione fisica dei bit (cavi, wireless, indirizzi MAC).
2. **Internet Layer (IP):** Instrada i pacchetti tra le reti usando gli indirizzi IP.
3. **Transport Layer (TCP/UDP):** Garantisce la consegna. Il **TCP** spaccchetta i dati, aggiunge un numero di sequenza, richiede un **ACK** (conferma di ricezione) per ogni pacchetto e ritrasmette quelli persi o corrotti, riordinandoli all'arrivo.
4. **Application Layer (Livello Applicazione):** Protocolli specifici per i software (HTTP per il web, FTP per i file, SMTP/POP per le mail).

0.3.1 Indirizzi IP (IPv4 vs IPv6)

Ogni dispositivo su Internet ha un indirizzo univoco.

- **IPv4:** 32 bit (es. 192 . 168 . 1 . 1). Permette circa 4.2 miliardi di indirizzi (ormai esauriti).
- **IPv6:** 128 bit. Introdotto per risolvere l'esaurimento degli IPv4, garantisce 2^{128} indirizzi. Scritto in formato esadecimale.

0.3.2 Il protocollo NAT (Network Address Translation)

Poiché gli indirizzi IPv4 pubblici costano e sono rari, le reti locali (es. la rete di casa) usano indirizzi privati (es. 10 . 0 . 0 . X).

Il **NAT** è un meccanismo implementato nei router: sostituisce l'indirizzo IP privato di un dispositivo locale con l'unico indirizzo IP pubblico assegnato al router dal provider (ISP), assegnando porte diverse per distinguere le richieste dei vari dispositivi interni.

0.4 L'Infrastruttura Fisica e i Server

Il web si basa sul modello **Client-Server**:

- **Server:** Un computer sempre acceso in ascolto di richieste.
- **Client:** Il dispositivo dell'utente (browser) che invia richieste e riceve risposte (Request-Response Loop).

0.4.1 Server Farms e Data Center

I siti ad alto traffico non girano su un singolo computer, ma su **Server Farms** (gruppi di server replicati) gestite da un **Load Balancer**, che smista il traffico per evitare sovraccarichi e garantire ridondanza se una macchina si guasta. Queste infrastrutture fisiche si trovano nei **Data Center**.

0.4.2 Come viaggiano i dati?

Dal router di casa (che fa da ponte), il segnale arriva all'**ISP** (Internet Service Provider). Per connettere reti mondiali, i provider si scambiano dati attraverso nodi enormi chiamati **IXP (Internet Exchange Points)**, abbattendo la latenza e usando cavi sottomarini in fibra ottica per attraversare gli oceani.

0.5 DNS, URL e Protocollo HTTP

0.5.1 DNS (Domain Name System)

Il DNS è la rubrica telefonica del web: traduce nomi di dominio leggibili dall'uomo (es. www.funwebdev.com) in indirizzi IP numerici comprensibili alle macchine (es. 66 . 147 . 244 . 79). Se il browser non ha l'IP in cache, interroga il resolver del sistema operativo, poi il server DNS dell'ISP, risalendo la gerarchia dai Root Server fino ai server TLD (.com, .it).

0.5.2 Anatomia dell'URL (Uniform Resource Locator)

Un URL è composto da parti specifiche:

<http://www.esempio.com/percorso/file.php?utente=mario#sezione1>

1. **Protocollo:** <http://>
2. **Dominio:** www.esempio.com
3. **Path (Percorso):** [/percorso/file.php](http://www.esempio.com/percorso/file.php)
4. **Query String:** [?utente=mario](http://www.esempio.com/percorso/file.php?utente=mario) (passa dati al server tramite coppie chiave=valore).
5. **Fragment (Ancora):** [#sezione1](http://www.esempio.com/percorso/file.php?utente=mario#sezione1) (scorre a un punto specifico della pagina).

0.5.3 Il Protocollo HTTP

L'HTTP opera sulla porta 80 (di default). Definisce i metodi di richiesta:

- **GET:** Richiede una risorsa. I dati inviati al server sono visibili in chiaro nell'URL (Query String). Ottimo per i preferiti e la navigazione, ma limitato in lunghezza e insicuro per dati sensibili.
- **POST:** Invia dati al server per essere processati. I dati viaggiano nascosti nel corpo (body) della richiesta HTTP. Indispensabile per password e file.

Codici di Risposta HTTP principali:

- 200: OK (Successo)
- 301 / 307: Redirect (La risorsa è stata spostata)
- 404: Not Found (Errore lato client, file inesistente)
- 500: Internal Server Error (Errore fatale del codice o del server)

1 HTML - Overview(02)

1.1 Fondamenti di HTML

L'HTML è definito come un linguaggio di marcatura.

Il **Markup** (marcatura) è un meccanismo per fornire informazioni aggiuntive sul contenuto di un documento. Consiste in annotazioni strutturali che rimangono distinte dal testo stesso, originariamente nate in ambito editoriale per comunicare istruzioni di revisione e formattazione.

Il **W3C** (World Wide Web Consortium) è l'organizzazione principale che definisce gli standard per il Web, garantendo la compatibilità attraverso specifiche e raccomandazioni.

Nel corso degli anni, l'HTML ha attraversato diverse fasi evolutive:

- **XHTML**: Introdotto per forzare regole di sintassi rigorose derivate dall'XML (tag in minuscolo, attributi sempre racchiusi tra virgolette, chiusura obbligatoria di ogni elemento) al fine di rendere il rendering più stabile e prevedibile.
- **XHTML 2.0 vs WHATWG**: Il W3C propose XHTML 2.0 abbandonando la retrocompatibilità. In risposta, un gruppo di sviluppatori creò il WHATWG, definendo uno standard alternativo che il W3C finì per adottare, abbandonando XHTML 2.0.
- **HTML5**: È l'esito di questa adozione. Specifica senza ambiguità come i browser debbano gestire il markup non valido, fornisce un framework aperto tramite JavaScript ed è retrocompatibile.

HTML5 è oggi considerato un *living language*: si tratta di un linguaggio che si evolve e si sviluppa continuamente nel tempo, le cui funzionalità vengono implementate gradualmente dai vari browser.

1.2 Sintassi HTML

I documenti HTML sono composti da contenuto testuale puro e da **elementi HTML**.

1.2.1 Elementi e Attributi

Un elemento HTML racchiude il contenuto e ne definisce il significato o il comportamento.

La struttura convenzionale di un elemento comprende:

- **Tag di apertura**: Il nome dell'elemento racchiuso tra parentesi angolari (es. `<a>`).
- **Attributi**: Inseriti nel tag di apertura sotto forma di coppie chiave/valore, forniscono direttive aggiuntive all'elemento.
- **Contenuto**: Il testo o altri elementi HTML nidificati.
- **Tag di chiusura**: Identico a quello di apertura, ma con il nome preceduto da uno slash (es. `</a>`).

```

1 <!-- Struttura completa di un elemento -->
2 <a href="https://www.esempio.it" target="_blank">Visita il sito</a>
3
4 <!-- Elemento vuoto: nessun tag di chiusura obbligatorio in HTML5 -->
5 
6 <br>
7 <hr>

```

Esistono anche **elementi vuoti** (empty elements) che non contengono alcun testo testuale o figlio (es. `<img>`). Essi agiscono come istruzioni dirette per il browser. In HTML5, lo slash di chiusura finale per questi elementi è opzionale.

1.2.2 Gerarchia e Annidamento

Gli elementi HTML possono (e devono) contenere altri elementi, creando una rigorosa struttura ad albero.

Relazioni strutturali del DOM (Document Object Model):

- **Parent** (Genitore): L'elemento contenitore diretto di un altro elemento.
- **Child** (Figlio): L'elemento contenuto in modo diretto all'interno del genitore.
- **Descendants** (Discendenti): Tutti gli elementi racchiusi all'interno di un genitore, indipendentemente dalla profondità dell'annidamento.
- **Ancestors** (Antenati): Tutti gli elementi che avvolgono un determinato nodo, risalendo fino alla radice del documento.
- **Siblings** (Fratelli): Elementi che condividono esattamente lo stesso genitore diretto.

```

1 <ul>                                <!-- parent di tutti gli <li> -->
2 <li>Primo elemento</li>            <!-- child di <ul>, sibling degli altri <li> -->
3 <li>                                <!-- child di <li>, descendant di <ul> -->
4 <a href="#">Link</a>
5 </li>
6 <li>Terzo elemento</li>
7 </ul>

```

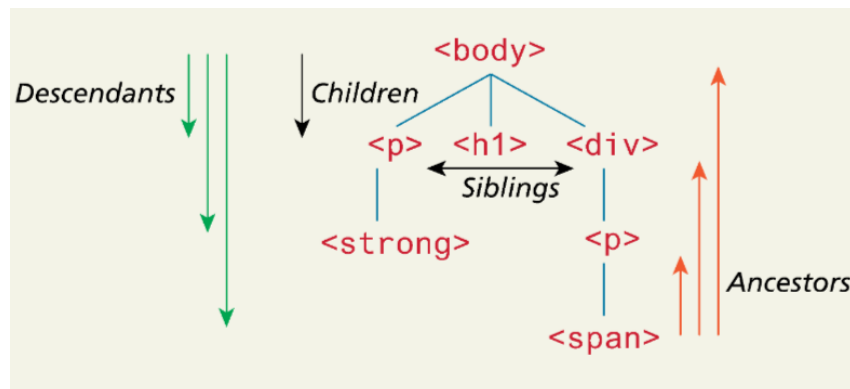


Figura 1.1: Schema ad albero delle relazioni strutturali tra nodi (Parent, Child, Ancestor, Sibling) per chiarire visivamente la gerarchia di annidamento.

Per garantire la validità sintattica, l'annidamento deve rispettare una logica speculare: il tag di chiusura di un elemento figlio deve **sempre** precedere il tag di chiusura dell'elemento genitore in cui è contenuto.

```

1 <!-- CORRETTO: chiusura speculare -->
2 <p><strong>Testo importante</strong></p>
3
4 <!-- ERRATO: annidamento non speculare -->
5 <p><strong>Testo importante</p></strong>

```

1.3 Markup Semantico

Un principio cardine della progettazione web moderna impone la netta separazione tra contenuto e presentazione visiva. L'HTML deve concentrarsi unicamente sulla **struttura** del documento.

Il markup semantico descrive esclusivamente il significato strutturale dei dati. Le informazioni su come il contenuto debba apparire visivamente nel browser devono essere relegate interamente ai fogli di stile (CSS).

```
1 <!-- SBAGLIATO: usare tag per ottenere effetti visivi -->
2 <b>Titolo della pagina</b>
3 <br><br>
4
5 <!-- CORRETTO: usare tag per il loro significato strutturale -->
6 <h1>Titolo della pagina</h1>
7 <p>Primo paragrafo del contenuto.</p>
```

Sfruttare un markup puramente semantico garantisce vantaggi strategici critici:

- **Mantenibilità:** Separare la logica visiva semplifica radicalmente gli aggiornamenti del codice.
- **Performance:** Le pagine prive di codice stilistico "inline" risultano più leggere e veloci da scaricare.
- **Accessibilità:** Strumenti assistivi (come gli screen reader per utenti ipovedenti) richiedono semantica rigida per interpretare e comunicare l'interfaccia correttamente.
- **Ottimizzazione SEO:** I motori di ricerca sfruttano i tag semantici per estrarre e indicizzare i contenuti rilevanti del sito.

1.4 Struttura del Documento HTML

In HTML5, l'impalcatura minima di una pagina web richiede pochi elementi macro-strutturali, universalmente utilizzati per suddividere logica e contenuto.

1.4.1 Tag Fondamentali

- **<!DOCTYPE html>:** Document Type Definition. È la prima istruzione del file e indica al client che il file deve essere elaborato come documento HTML. In HTML5 non dichiara una versione specifica, ma attiva la modalità standard del browser.
- **<html>:** Il *root element* (elemento radice) che racchiude l'intero ecosistema della pagina web.
- **<head>:** L'intestazione della pagina. Contiene esclusivamente elementi descrittivi e di supporto. **Il suo contenuto non viene mai renderizzato nell'area visibile del browser.**
- **<body>:** Il corpo del documento. Contiene tutto l'HTML che si tradurrà nel layout visibile all'utente.

```
1 <!DOCTYPE html>
2 <html lang="it">
3   <head>
4     <meta charset="utf-8">
5     <title>La mia pagina</title>
6     <link rel="stylesheet" href="style.css">
7   </head>
8   <body>
9     <h1>Benvenuto</h1>
10    <p>Contenuto della pagina.</p>
11  </body>
12 </html>
```

1.4.2 La sezione Head

L'elemento **<head>** funge da centro di controllo invisibile per la pagina, ospitando tag cruciali:

- **<meta>**: Imposta i metadati della pagina. L'attributo `charset="utf-8"` è fondamentale per dichiarare la corretta codifica dei caratteri.
- **<title>**: Fornisce una descrizione semantica globale del documento, visibile solo nella scheda o nella barra del titolo del browser.
- **<link>**: Associa risorse esterne al file, tipicamente utilizzato per incorporare i CSS (es. `<link rel="stylesheet" href="style.css">`).
- **<script>**: Importa ed esegue file JavaScript esterni o logiche di script incorporate.

```

1 <head>
2   <meta charset="utf-8">
3   <meta name="description" content="Descrizione della pagina per i motori di ricerca">
4   <meta name="viewport" content="width=device-width, initial-scale=1.0">
5   <title>Titolo visibile nella scheda del browser</title>
6   <link rel="stylesheet" href="style.css">
7   <script src="script.js" defer></script>
8 </head>

```

1.5 Elementi di Testo e Contenuto

L'HTML fornisce una serie di tag per strutturare il testo in modo gerarchico e logico.

1.5.1 Intestazioni (Headings)

I tag da `<h1>` a `<h6>` definiscono l'importanza dei titoli.

Non si devono utilizzare questi tag per ridimensionare il testo (compito che spetta ai CSS), ma per definire la struttura del contenuto. Un documento dovrebbe avere idealmente un solo `<h1>` che ne rappresenti l'argomento principale.

```

1 <h1>Titolo principale del documento (uno solo per pagina)</h1>
2 <h2>Sezione principale</h2>
3   <h3>Sottosezione</h3>
4   <h3>Altra sottosezione</h3>
5 <h2>Seconda sezione principale</h2>
6   <h3>Sottosezione della seconda sezione</h3>

```

1.5.2 Paragrafi e Interruzioni

- **<p>**: Definisce un paragrafo di testo. I browser aggiungono automaticamente uno spazio (margine) prima e dopo l'elemento.
- **
**: Inserisce un'interruzione di riga forzata senza creare un nuovo paragrafo. Va usato con parsimonia, solo dove l'interruzione fa parte del contenuto (es. poesie o indirizzi).
- **<hr>**: Rappresenta una rottura tematica tra paragrafi. Visivamente appare come una linea orizzontale, ma semanticamente indica un cambio di argomento.

```

1 <p>Questo è il primo paragrafo. I browser aggiungono spazio attorno automaticamente.</p>
2 <p>Secondo paragrafo. Nessun bisogno di <br> per separare i paragrafi.</p>
3
4 <!-- Uso corretto di <br>: indirizzo postale -->
5 <p>
6   Via Roma 12<br>
7   95100 Catania<br>
8   Italia
9 </p>
10
11 <!-- Rottura tematica tra due argomenti distinti -->
12 <p>Fine del primo argomento.</p>
13 <hr>
14 <p>Inizio di un nuovo argomento.</p>

```

1.6 Link e Navigazione

L'elemento `<a>` (anchor) è il pilastro del Web, permettendo la creazione di collegamenti ipertestuali.

1.6.1 L'attributo href

L'attributo fondamentale è `href` (hypertext reference), che specifica la destinazione del link.

Un link può puntare a:

1. **Risorse esterne:** URL completi su altri domini.
2. **Risorse interne:** Altre pagine o file all'interno dello stesso sito.
3. **Ancore (Fragment Identifiers):** Punti specifici all'interno della pagina stessa o di un'altra pagina, identificati dall'attributo `id`. Sintassi: `<a href="#sezione1">`.

```
1 <!-- 1. Risorsa esterna -->
2 <a href="https://www.wikipedia.org" target="_blank">Apri Wikipedia in nuova scheda</a>
3
4 <!-- 2. Risorsa interna -->
5 <a href="contatti.html">Vai alla pagina Contatti</a>
6 <a href="prodotti/scarpe.html">Scarpe</a>
7
8 <!-- 3. Ancora nella stessa pagina -->
9 <a href="#sezione-contatti">Salta ai contatti</a>
10 <!-- ...più in basso nel documento... -->
11 <h2 id="sezione-contatti">Contatti</h2>
12
13 <!-- Ancora in un'altra pagina -->
14 <a href="about.html#team">Il nostro team</a>
```

1.6.2 Elemento link nell'intestazione

Il tag `<link>` (da non confondere con `<a>`) viene inserito nel `<head>` e definisce relazioni tra il documento corrente e risorse esterne.

Gli attributi comuni per `<link>` includono `rel` (es. `stylesheet`), `type` (es. `text/css`) e `href`. Non è un elemento cliccabile dall'utente, ma un'istruzione per il browser.

```
1 <head>
2 <!-- Collegamento a un foglio di stile -->
3 <link rel="stylesheet" href="style.css">
4
5 <!-- Icona visualizzata nella scheda del browser (favicon) -->
6 <link rel="icon" type="image/png" href="favicon.png">
7
8 <!-- Font esterno da Google Fonts -->
9 <link rel="preconnect" href="https://fonts.googleapis.com">
10 </head>
```

1.7 Gestione dei Percorsi (URL)

Comprendere la differenza tra percorsi assoluti e relativi è critico per la portabilità del progetto web e per il superamento dello scritto.

1.7.1 URL Assoluti

Un URL assoluto fornisce l'indirizzo completo della risorsa, inclusi protocollo e dominio.

Esempio: <https://www.esempio.it/immagini/logo.png>

Vengono utilizzati principalmente per collegarsi a siti esterni. Se il dominio esterno cambia o la risorsa viene rimossa, il link si interrompe.

```
1 <!-- URL assoluto: include protocollo, dominio e percorso completo -->
2 <a href="https://www.esempio.it/blog/articolo.html">Leggi l'articolo</a>
3 
```

1.7.2 URL Relativi

Un URL relativo descrive la posizione di una risorsa rispetto alla posizione del file HTML corrente. È la scelta preferibile per i collegamenti interni perché permette di spostare l'intero progetto tra diverse cartelle o server senza rompere i link.

Sintassi dei percorsi (derivata da Unix):

- `nomefile.html`: Il file si trova nella **stessa cartella** del documento corrente.
- `cartella/file.html`: Il file si trova in una **sottocartella** chiamata “cartella”.
- `../file.html`: Risale di **un livello** nella gerarchia delle cartelle (cartella genitore).
- `/`: Rappresenta la **root** (radice) del sito web (attenzione: il comportamento può variare tra ambiente locale e server web).

```
1 <!-- Struttura di esempio:
2 sito/├─
3   index.html├─
4   about.html├─
5   css/├─
6       style.css├─
7   blog/├─
8       articolo.html├─
9           immagini/├─
10               foto.jpg
11 -->
12
13 <!-- Da index.html -->
14 <a href="about.html">Chi siamo</a>           <!-- stessa cartella -->
15 <a href="blog/articolo.html">Blog</a>         <!-- sottocartella -->
16 <link rel="stylesheet" href="css/style.css">   <!-- sottocartella -->
17
18 <!-- Da blog/articolo.html -->
19 <a href="../index.html">Torna alla home</a>    <!-- cartella genitore -->
20 <a href="../about.html">Chi siamo</a>         <!-- cartella genitore -->
21       <!-- sottocartella locale -->
```

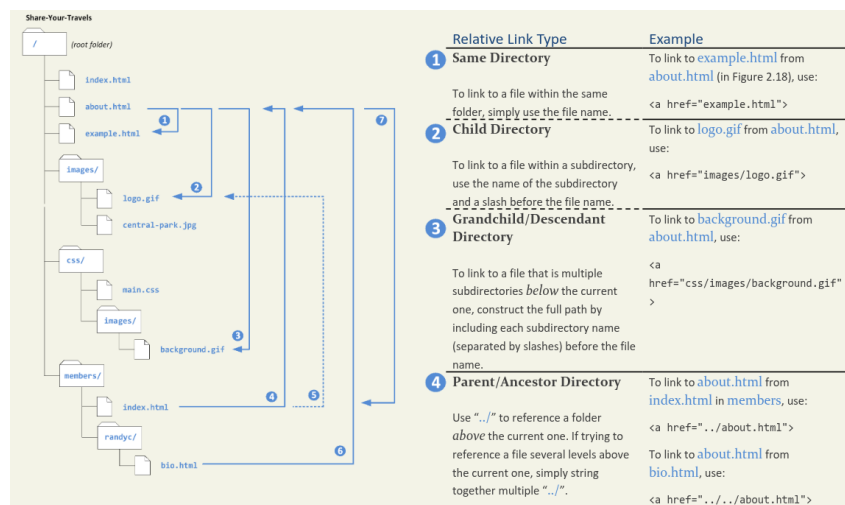


Figura 1.2: Schema di una struttura a cartelle con frecce che mostrano come navigare tra file in cartelle diverse usando `../` e percorsi discendenti.

1.8 Elementi Testuali Inline (Phrasing Content)

Questi elementi vengono utilizzati all'interno dei blocchi di testo per aggiungere enfasi o significato a specifiche porzioni di parole.

- `<em>`: Indica enfasi (solitamente visualizzato in corsivo). Rappresenta uno stress verbale su una parola.
- `<strong>`: Indica forte importanza o urgenza (solitamente visualizzato in grassetto).
- `<code>`: Utilizzato per frammenti di codice informatico.
- `<sub>` e `<sup>`: Pedice e apice, utili per formule chimiche o matematiche (es. H_2O).

In HTML5, la distinzione tra `<b>` (grassetto estetico) e `<strong>` (importanza semantica) è fondamentale. Sebbene il risultato visivo possa essere identico, uno screen reader darà un'enfasi diversa a `<strong>`.

```

1 <p>Ricorda: <strong>non salvare mai la password in chiaro</strong>.</p>
2
3 <p>Devi premere <em>esattamente</em> questo tasto per continuare.</p>
4
5 <p>Per stampare usa il comando <code>console.log("ciao")</code>.</p>
6
7 <p>L'acqua ha formula chimica H<sub>2</sub></sub>O.</p>
8 <p>Il quadrato di x si scrive x<sup>2</sup></sup>.</p>
9
10 <!-- Differenza semantica b vs strong -->
11 <p><b>Grassetto puramente estetico</b>, nessun significato aggiunto.</p>
12 <p><strong>Attenzione: campo obbligatorio.</strong> (importanza semantica)</p>
```

1.9 Liste e Caratteri Speciali

L'HTML fornisce tre macro-strutture fondamentali per raggruppare elementi testuali sotto forma di elenco, ognuna con precisi scopi semantici.

1.9.1 Liste Non Ordinate (Unordered Lists)

Utilizzate per collezioni di elementi in cui l'ordine gerarchico o sequenziale non è rilevante. Di default, i browser ne effettuano il rendering visivo tramite punti elenco (bullet points).

- `<ul>`: Definisce il contenitore principale della lista non ordinata.
- `<li>`: Definisce il singolo elemento della lista (List Item). Questo tag può contenere a sua volta testo puro o altri elementi HTML nidificati (come link, immagini o altre liste).

```
1 <ul>
2   <li>Pomodori</li>
3   <li>Mozzarella</li>
4   <li>
5     Condimenti
6     <!-- Lista annidata -->
7     <ul>
8       <li>Olio d'oliva</li>
9       <li>Sale</li>
10      <li>Origano</li>
11    </ul>
12  </li>
13 </ul>
```

1.9.2 Liste Ordinate (Ordered Lists)

Da impiegare rigorosamente quando la sequenza logica dei passaggi è fondamentale per la comprensione del contenuto (es. ricette, classifiche o procedure passo-passo).

- `<ol>`: Il contenitore principale. Di default utilizza una numerazione decimale sequenziale.
- Attributi specifici di `<ol>`:
 - `reversed`: Attributo booleano che inverte l'ordine della numerazione (il conteggio procede all'indietro).
 - `start`: Specifica il valore ordinale da cui far partire la numerazione (es. `start="10"` farà partire la lista da 10).
 - `type`: Definisce il tipo di marcatore visivo (1 per numeri decimali, `a/A` per lettere alfabetiche minuscole/maiuscole, `i/I` per numeri romani).

```
1 <!-- Lista ordinata standard -->
2 <ol>
3   <li>Preriscalda il forno a 180°C</li>
4   <li>Mescola gli ingredienti</li>
5   <li>Inforna per 30 minuti</li>
6 </ol>
7
8 <!-- Classifica con lettere, a partire dal terzo posto -->
9 <ol type="A" start="3">
10  <li>Terzo classificato</li>
11  <li>Quarto classificato</li>
12 </ol>
13
14 <!-- Conto alla rovescia -->
15 <ol reversed>
16  <li>Terzo posto</li>
17  <li>Secondo posto</li>
18  <li>Primo posto</li>
19 </ol>
20
21 <!-- Numerazione romana -->
22 <ol type="I">
23  <li>Introduzione</li>
24  <li>Sviluppo</li>
25  <li>Conclusione</li>
26 </ol>
```

1.9.3 Liste di Definizione (Definition Lists)

Utilizzate per raggruppare concettualmente coppie di “termine/descrizione” o “nome/valore”, risultando ideali per glossari, metadati o sezioni FAQ (Frequently Asked Questions).

- `<dl>`: Il contenitore globale della lista di definizione.
- `<dt>`: Il termine da definire (Definition Term).
- `<dd>`: La descrizione o definizione del termine (Definition Description).

I valori all'interno di un gruppo sono considerati alternative o espansioni; per questo motivo, è semanticamente corretto e frequente far seguire a un singolo `<dt>` più elementi `<dd>` se si necessita di definizioni multiple o paragrafi separati per descrivere lo stesso termine.

```
1 <dl>
2   <dt>HTML</dt>
3   <dd>HyperText Markup Language. Linguaggio di marcatura per il Web.</dd>
4   <dd>Standard mantenuto dal W3C e dal WHATWG.</dd>
5
6   <dt>CSS</dt>
7   <dd>Cascading Style Sheets. Linguaggio per la presentazione visiva dei documenti HTML.</dd>
8
9   <dt>HTTP</dt>
10  <dt>HTTPS</dt> <!-- due termini con la stessa definizione -->
11  <dd>Protocolli di comunicazione per il trasferimento di dati sul Web.</dd>
12 </dl>
```

1.9.4 Entità Carattere (Character Entities)

Molti caratteri sono riservati dalla sintassi HTML (come i delimitatori di tag `<` o `>`) oppure non sono direttamente digitabili tramite tastiera standard (come il simbolo del copyright). Per assicurarne la corretta visualizzazione, si utilizzano sequenze di escape chiamate entità:

- `<`; per il segno minore `<` (less than).
- `>`; per il segno maggiore `>` (greater than).
- `©`; per il simbolo di copyright ©.
- `™`; per il simbolo di trademark ™.
- ` `; per lo spazio non interrompibile (non-breaking space), utile per evitare che due parole vengano separate andando a capo.

```
1 <p>Il tag paragrafo si scrive &lt;p&gt; in HTML.</p>
2
3 <p>Per mostrare il simbolo minore: 5 &lt; 10</p>
4
5 <footer>
6   <p>&copy; 2026 La Mia Azienda&trade;. Tutti i diritti riservati.</p>
7 </footer>
8
9 <!-- Spazio non interrompibile: "10 km" non andrà mai a capo tra numero e unità -->
10 <p>La distanza è di 10&nbsp;km.</p>
```

1.10 Immagini e Mappe Immagine

L'inserimento di immagini in un documento web richiede una chiara distinzione concettuale: le immagini puramente decorative (sfondi, pattern, layout) devono essere gestite via CSS, mentre le immagini che veicolano contenuto informativo necessitano del markup HTML.

1.10.1 L'elemento `img`

Il tag `<img>` è un elemento vuoto (empty element) che segnala al browser di caricare e renderizzare una risorsa grafica esterna.

- `src`: Attributo strettamente obbligatorio che specifica il percorso (URL assoluto o relativo) per recuperare il file dell'immagine.

- **alt**: Attributo obbligatorio che fornisce un testo alternativo (Alternative Text). È di importanza critica: viene letto dagli screen reader per l'accessibilità degli ipovedenti e viene visualizzato dal browser se l'immagine fallisce il caricamento.
- **title**: Fornisce testo aggiuntivo visualizzato solitamente come tooltip quando l'utente posiziona il cursore sull'immagine.
- **width** e **height**: Definiscono le dimensioni dell'immagine in pixel.

```

1 <!-- Immagine base con attributi obbligatori -->
2 
3
4 <!-- Con dimensioni e tooltip -->
5 
12
13 <!-- Alt vuoto per immagini puramente decorative (lo screen reader la ignora) -->
14 

```

1.10.2 Mappe Immagine (Image Maps)

Le mappe immagine consentono di prendere una singola immagine e definire al suo interno specifiche “aree sensibili” geometriche, assegnando a ciascuna di esse un hyperlink differente.

Architettura di una Image Map:

1. L'immagine di base viene inserita con `<img>` a cui si aggiunge l'attributo `usemap="#nome_mappa"`.
2. Si crea un contenitore invisibile tramite il tag `<map>` con l'attributo `name="nome_mappa"` (il collegamento avviene tramite questo nome).
3. All'interno di `<map>`, si annidano uno o più elementi `<area>`, i quali non hanno tag di chiusura e definiscono le regioni cliccabili.

```

1 
2
3 <map name="mappa-regioni">
4   <!-- Area rettangolare: left-x, top-y, right-x, bottom-y -->
5   <area
6     shape="rect"
7     coords="100,50,200,150"
8     href="Lombardia.html"
9     alt="Lombardia"
10  >
11  <!-- Area circolare: center-x, center-y, raggio -->
12  <area
13    shape="circle"
14    coords="180,320,40"
15    href="sicilia.html"
16    alt="Sicilia"
17  >
18  <!-- Area poligonale: coppie x,y dei vertici -->
19  <area
20    shape="poly"
21    coords="90,200,130,180,150,220,100,240"
22    href="toscana.html"
23    alt="Toscana"
24  >
25 </map>

```

Gli attributi fondamentali di `<area>` includono:

- **shape**: Specifica la geometria della zona (**rect** per rettangoli, **circle** per cerchi, **poly** per poligoni complessi).
- **coords**: Un set di coordinate (dipendenti dalla **shape** scelta) che delineano fisicamente la regione. Per un rettangolo, ad esempio, si usano **left-x**, **top-y**, **right-x**, **bottom-y**.
- **href**: L'URL di destinazione che si attiverà cliccando quell'area.

1.11 Architettura e Semantica in HTML5

Prima dell'avvento di HTML5, gli sviluppatori tendevano ad abusare del tag generico `<div>` per costruire l'intero layout (pratica tristemente nota come "div-tastic"), affidandosi ad attributi `id` o `class` per simulare un significato logico. HTML5 risolve questa confusione introducendo container strutturali dotati di intrinseco valore semantico.

1.11.1 Macro-aree del Layout Strutturale

- `<header>`: Rappresenta l'intestazione di una pagina o di una singola sezione. Contiene elementi introduttivi come loghi, titoli principali e strumenti di navigazione.
- `<footer>`: Rappresenta il piè di pagina (sia del documento intero che di un elemento di sezionamento), contenendo comunemente note di copyright, link legali o informazioni di contatto.
- `<main>`: Incapsula il contenuto primario e univoco del documento. **Regola aurea:** ci può essere al massimo un solo `<main>` per documento e non deve mai racchiudere blocchi ripetitivi come menù di navigazione o sidebar.
- `<nav>`: Progettato per racchiudere un blocco di importanza primaria dedicato ai link di navigazione (es. il menù principale o globale). Non tutte le serie di link necessitano del tag `<nav>`.

```
1 <body>
2   <header>
3     
4     <h1>Nome del Sito</h1>
5     <nav>
6       <a href="index.html">Home</a>
7       <a href="about.html">Chi siamo</a>
8       <a href="contatti.html">Contatti</a>
9     </nav>
10  </header>
11
12  <main>
13    <h2>Benvenuto nel sito</h2>
14    <p>Contenuto principale della pagina.</p>
15  </main>
16
17  <footer>
18    <p>&copy; 2026 Nome del Sito. Tutti i diritti riservati.</p>
19    <a href="privacy.html">Privacy Policy</a>
20  </footer>
21 </body>
```

1.11.2 Sezionamento dei Contenuti

- `<article>`: Definisce un blocco di contenuto autonomo, autosufficiente e teoricamente riutilizzabile o sindacabile al di fuori del contesto in cui si trova (es. un articolo di blog, una news in una testata online, un commento degli utenti).
- `<section>`: Indica un raggruppamento logico o tematico generico di contenuto, tipicamente provvisto di un proprio titolo di intestazione.

Il W3C specifica chiaramente che `<section>` non va usato come mero contenitore estetico. **Regola discriminante:** Se il blocco ha senso di figurare in una mappa logica o in un indice dei contenuti (outline) del documento, allora usa `<section>`. Se serve solo ad agganciare logiche CSS o script, utilizza un normale `<div>`.

```
1 <main>
2   <!-- section: raggruppa contenuti tematici con un proprio titolo -->
3   <section>
4     <h2>Ultime Notizie</h2>
5
6     <!-- article: ogni notizia è autonoma e riutilizzabile -->
7     <article>
8       <h3>Titolo della notizia</h3>
9       <p>Testo della notizia...</p>
10    </article>
11
```

```

12     <article>
13       <h3>Seconda notizia</h3>
14       <p>Testo della seconda notizia...</p>
15     </article>
16   </section>
17
18   <section>
19     <h2>Recensioni degli utenti</h2>
20     <article>
21       <h3>Ottimo prodotto!</h3>
22       <p>Recensione di un utente soddisfatto...</p>
23     </article>
24   </section>
25 </main>

```

1.11.3 Elementi Complementari e di Dettaglio

- `<aside>`: Identifica un contenuto che è collegato solo tangenzialmente al blocco principale che lo circonda (es. box laterali di approfondimento, banner pubblicitari o glossari contestuali).
- `<figure>` e `<figcaption>`: Il tag `<figure>` funge da wrapper per contenuti essenziali alla comprensione dell'argomento trattato (immagini, snippet di codice, grafici) ma il cui esatto posizionamento nel flusso del testo è ininfluente. `<figcaption>` definisce la didascalia che descrive il contenuto di `<figure>`.
- `<details>` e `<summary>`: Costituiscono insieme un vero e proprio widget interattivo nativo. Il testo contenuto in `<summary>` viene renderizzato come un'etichetta cliccabile che, una volta attivata, espande e mostra l'intero contenuto nascosto racchiuso nel blocco `<details>`.

```

1 <article>
2   <h2>Guida alla fotografia notturna</h2>
3   <p>La fotografia notturna richiede attrezzatura specifica e pazienza...</p>
4
5   <!-- figure con didascalia -->
6   <figure>
7     
8     <figcaption>Scatto a lunga esposizione, 30s, f/2.8, ISO 1600.</figcaption>
9   </figure>
10
11   <p>Per ottenere i migliori risultati è fondamentale...</p>
12
13   <!-- aside: approfondimento tangenziale -->
14   <aside>
15     <h3>Lo sapevi?</h3>
16     <p>La Via Lattea è visibile ad occhio nudo solo lontano dall'inquinamento luminoso.</p>
17   </aside>
18
19   <!-- details/summary: FAQ o approfondimento opzionale -->
20   <details>
21     <summary>Quali impostazioni ISO usare di notte?</summary>
22     <p>Per la fotografia notturna si consiglia un ISO compreso tra 800 e 3200, a seconda del sensore della fotocamera.</p>
23   </details>
24 </article>

```

1.12 Elementi di Testo Inline (Phrasing Content)

Oltre agli elementi strutturali a blocco e ai tag di base già visti, l'HTML fornisce tag "inline" specifici (noti in HTML5 come *phrasing content*). Questi elementi non interrompono il flusso del documento e servono a marcare o isolare porzioni di testo all'interno di un blocco più grande.

1.12.1 Il tag generico span

Così come il `<div>` è il contenitore generico per gli elementi a blocco, lo `<span>` è il contenitore generico per gli elementi inline.

Lo `<span>` è privo di qualsiasi valore semantico intrinseco. Deve essere utilizzato esclusivamente quando non esiste un elemento HTML semantico più appropriato, e serve quasi sempre come “gancio” (hook) per applicare regole CSS a una o più parole o per manipolazioni tramite JavaScript.

```

1 <!-- span usato come hook CSS per colorare una parola -->
2 <p>Il prezzo è <span class="prezzo-evidenziato">€ 29,99</span> IVA inclusa.</p>
3
4 <!-- span usato come hook JavaScript -->
5 <p>Hai <span id="contatore-messaggi">0</span> nuovi messaggi.</p>
6
7 <!-- SBAGLIATO: usare span quando esiste un tag semantico appropriato -->
8 <p>Ricorda: <span class="importante">non dimenticare la password</span>.</p>
9
10 <!-- CORRETTO: usare strong per l'importanza semantica -->
11 <p>Ricorda: <strong>non dimenticare la password</strong>.</p>

```

1.12.2 Elementi Semantici Inline

HTML5 ha perfezionato diversi tag inline per descrivere in modo inequivocabile la natura specifica di alcuni frammenti di testo:

- `<mark>`: Rappresenta testo evidenziato per scopi di riferimento, indicando che una parte del documento è particolarmente rilevante nel contesto attuale (simula l'uso di un evidenziatore giallo su carta).
- `<time>`: Utilizzato per rappresentare date e orari in un formato leggibile sia dall'utente che dalle macchine. L'uso dell'attributo `datetime` permette di specificare il valore assoluto standardizzato (es. `<time datetime="2026-04-03">Oggi</time>`), fondamentale per agende, indicizzazione SEO ed eventi.
- `<abbr>`: Indica un'abbreviazione o un acronimo. L'utilizzo dell'attributo `title` fornisce l'espansione testuale completa del termine, che i browser spesso rivelano come tooltip al passaggio del cursore.
- `<q>`: Definisce una breve citazione in linea (a differenza di `<blockquote>` che è un elemento a blocco). I browser moderni inseriscono automaticamente le virgolette tipografiche attorno al testo racchiuso in questo tag.

```

1 <!-- mark: evidenziazione contestuale (es. risultati di una ricerca) -->
2 <p>Risultati per "gatto": il <mark>gatto</mark> è un mammifero domestico.</p>
3
4 <!-- time: data leggibile dalle macchine -->
5 <p>L'evento si terrà il <time datetime="2026-06-15">15 giugno 2026</time>.</p>
6 <p>I biglietti aprono alle <time datetime="10:00">10 di mattina</time>.</p>
7
8 <!-- abbr: acronimi con espansione -->
9 <p>Il <abbr title="World Wide Web Consortium">W3C</abbr> definisce gli standard web.</p>
10 <p>Usa il protocollo <abbr title="HyperText Transfer Protocol Secure">HTTPS</abbr> per connessioni sicure.</p>
11
12 <!-- q: citazione inline -->
13 <p>Come disse Einstein: <q>La fantasia è più importante della conoscenza.</q></p>

```

1.13 Contenuto Multimediale Embedded

Una delle innovazioni più incisive di HTML5 è stata l'introduzione del supporto nativo per i contenuti multimediali, eliminando totalmente la dipendenza da plugin di terze parti (come Flash o Silverlight).

1.13.1 L'elemento iframe

L'Inline Frame (`<iframe>`) permette di incorporare un intero documento HTML indipendente all'interno della pagina corrente. È lo standard di fatto per integrare widget esterni, come mappe interattive o feed social.

Dal punto di vista della sicurezza, l'uso degli `<iframe>` richiede estrema cautela. L'attributo `sandbox` è uno strumento cruciale che impone rigide restrizioni al contenuto incorporato (impedendo, ad esempio, l'esecuzione di script maligni o l'invio di form dal sito terzo), proteggendo così l'architettura della pagina ospitante.

```

1 <!-- Incorporare una mappa Google Maps -->
2 <iframe
3   src="https://www.google.com/maps/embed?pb=!1m18..."
4   width="600"
5   height="450"
6   allowfullscreen
7   loading="lazy"
8   title="Mappa della sede"
9 ></iframe>
10
11 <!-- iframe con sandbox per sicurezza -->
12 <iframe
13   src="widget-esterno.html"
14   width="400"
15   height="300"
16   sandbox="allow-scripts allow-same-origin"
17   title="Widget esterno"
18 ></iframe>

```

1.13.2 Riproduzione Video

Il tag `<video>` fornisce un'interfaccia pulita per lo streaming video nativo.

Architettura robusta di un elemento video:

1. Il tag `<video>` funge da contenitore globale e ospita gli attributi di comportamento.
2. All'interno, più elementi vuoti `<source>` specificano i percorsi e i relativi tipi MIME (es. `video/mp4`, `video/webm`). Il client valuta la lista dall'alto verso il basso e carica il primo formato compatibile.

```

1 <video
2   controls
3   poster="anteprima.jpg"
4   width="800"
5   height="450"
6 >
7 <!-- Il browser usa il primo formato che supporta -->
8 <source src="video/presentazione.webm" type="video/webm">
9 <source src="video/presentazione.mp4" type="video/mp4">
10 <!-- Fallback per browser che non supportano <video> -->
11 Il tuo browser non supporta la riproduzione video HTML5.
12 <a href="video/presentazione.mp4">Scarica il video</a>
13 </video>
14
15 <!-- Video in autoplay silenzioso (es. sfondo animato) -->
16 <video autoplay muted loop>
17   <source src="sfondo.webm" type="video/webm">
18   <source src="sfondo.mp4" type="video/mp4">
19 </video>

```

Attributi fondamentali per pilotare il tag `<video>`:

- **controls**: Delega al browser la generazione dell'interfaccia utente predefinita (play, pausa, barra spaziale, volume).
- **autoplay**: Avvia la riproduzione non appena i dati base sono caricati (fortemente limitato dai browser moderni per policy di usabilità, solitamente funziona solo se accompagnato dall'attributo `muted`).
- **loop**: Riavvia la riproduzione in modo continuo.
- **muted**: Silenzia nativamente la traccia audio.
- **poster**: Definisce l'URL di un'immagine statica (thumbnail) da renderizzare mentre il file video viene scaricato o finché l'utente non avvia la riproduzione.

1.13.3 Riproduzione Audio

Il tag `<audio>` condivide quasi interamente la logica e gli attributi dell'elemento video, ma è specializzato per le tracce sonore. Sfrutta gli attributi di controllo (`controls`, `autoplay`, `loop`, `muted`) e utilizza gli elementi annidati `<source>` per fornire file in formati differenziati (come `audio/mpeg` per gli MP3 o `audio/ogg`).

```
1 <audio controls>
2   <source src="audio/podcast-episodio1.ogg" type="audio/ogg">
3   <source src="audio/podcast-episodio1.mp3" type="audio/mpeg">
4   <!-- Fallback testuale -->
5   Il tuo browser non supporta la riproduzione audio HTML5.
6   <a href="audio/podcast-episodio1.mp3">Scarica l'audio</a>
7 </audio>
8
9 <!-- Audio in loop (es. sottofondo musicale) -->
10 <audio autoplay muted loop>
11   <source src="musica-sfondo.ogg" type="audio/ogg">
12   <source src="musica-sfondo.mp3" type="audio/mpeg">
13 </audio>
```

Gestione del Fallback: Sia per `<video>` che per `<audio>`, è essenziale inserire un testo di avviso puro appena prima del tag di chiusura. Se un browser obsoleto non è in grado di interpretare i tag multimediali HTML5, li ignorerà e mostrerà il testo “di ripiego” all’utente.

2 CSS - Intro(03)

2.1 Introduzione al CSS e Sintassi Base

Il **CSS** (Cascading Style Sheets) è lo standard W3C deputato a definire l'aspetto visivo dei documenti HTML. Il suo scopo principale è permettere una netta separazione tra la struttura semantica (l'HTML) e la presentazione visiva (colori, layout, font, spaziature).

2.1.1 La Sintassi

Una direttiva CSS, detta “regola”, è composta fondamentalmente da due parti: un **selettore** e un **blocco di dichiarazioni**.

Anatomia di una regola CSS:

- **Selettore:** Indica al browser a quali elementi HTML applicare lo stile.
- **Blocco di dichiarazioni:** Racchiuso tra parentesi graffe {}. Contiene una o più dichiarazioni separate dal punto e virgola ;.
- **Dichiarazione:** Formata da una coppia **proprietà: valore**.

```
1 /*Esempio Pratico: Sintassi base */
2 h1 {
3   color: blue;      /* Imposta il colore del testo */
4   font-size: 24px;  /* Imposta la grandezza del carattere*/
5 }
```

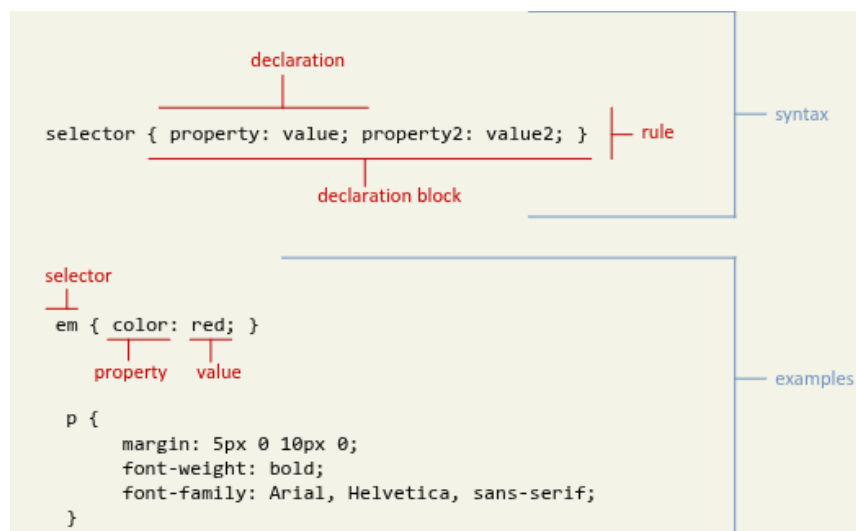


Figura 2.1: Immagine dell'anatomia di una regola CSS, solitamente un diagramma che scompone “h1 { color: blue; }” etichettando le varie parti (Selector, Property, Value, Declaration).

2.2 Posizionamento degli Stili (Location of Styles)

Esistono tre metodi per applicare il codice CSS a una pagina web. Scegliere il metodo corretto è fondamentale sia in sede d'esame che per sviluppare un progetto scalabile.

2.2.1 Stili Inline

Lo stile viene applicato direttamente all'interno di un tag HTML specifico tramite l'attributo `style`.

Da usare **solo** per test rapidi o casi in cui occorre forzare programmaticamente uno stile. Rende il codice impossibile da mantenere su larga scala poiché mescola irrimediabilmente struttura e design.

```
1 <p style="color: red; font-weight: bold;">Testo di avviso importante.</p>
```

2.2.2 Stili Interni (Embedded)

Le regole CSS vengono inserite all'interno del tag `<style>`, il quale deve risiedere rigorosamente nel blocco `<head>` del documento HTML.

Utile quando si sta sviluppando una pagina autonoma (single-page) o se le regole devono avere validità esclusiva per un solo e specifico documento.

```
1 <head>
2   <style>
3     body { background-color: #f4f4f4; }
4     p { line-height: 1.5; }
5   </style>
6 </head>
```

2.2.3 Stili Esterni (External)

Il metodo **ottimale e standard nel settore**. Tutto il codice CSS viene estratto in un file separato (con estensione `.css`). Tale file viene poi collegato alla pagina HTML (o a centinaia di pagine contemporaneamente) tramite il tag `<link>`.

```
1 <head>
2   <link rel="stylesheet" href="css/style.css">
3 </head>
```

2.3 I Selettori CSS Fondamentali

I selettori sono il meccanismo che permette di individuare con estrema precisione a quali nodi dell'albero HTML (DOM) vogliamo applicare le nostre direttive visive.

2.3.1 Selettori di Elemento

Selezionano tutti i tag di un determinato tipo presenti nella pagina. Sono ideali per definire le regole globali di base (es. resettare margini, impostare il font globale del sito).

```
1 /*Seleziona TUTTI i paragrafi della pagina*/
2 p {
3   margin-bottom: 20px;
4   font-family: Arial, sans-serif;
5 }
```

2.3.2 Selettori di Classe

Si definiscono anteposando un punto `.` al nome scelto. Le classi sono progettate specificamente per essere **riutilizzabili**: più elementi possono condividere la stessa classe, e un singolo elemento HTML può ospitare più classi contemporaneamente (separate da uno spazio vuoto).

```
1 /*CSS*/
2 .pulsante-primario {
3     background-color: blue;
4     color: white;
5 }
6 .arrotondato {
7     border-radius: 5px;
8 }
```

```
1 <button class="pulsante-primario arrotondato">Invia Dati</button>
2 <a href="#" class="pulsante-primario">Torna indietro</a>
```

2.3.3 Selettori di ID

Si definiscono anteposando il cancelletto `#` al nome.

Regola d'oro per l'esame: Un attributo `id` deve essere rigorosamente **univoco** all'interno dell'intero documento HTML. Non possono esistere due elementi con lo stesso ID. Si utilizzano per identificare in modo univoco una macrostruttura singola (come l'header o il footer) o per creare "ganci" mirati per le interazioni JavaScript.

```
1 /*CSS*/
2 # header-principale {
3     border-bottom: 2px solid black;
4     padding: 10px;
5 }
```

```
1 <header id="header-principale">
2     <h1>Il mio Sito</h1>
3 </header>
```

2.4 Selettori Avanzati e Contestuali

Oltre a tag, classi e ID, il CSS permette di selezionare gli elementi in base alla loro posizione nell'albero HTML (DOM) o al loro "stato" interattivo.

2.4.1 Selettori Relazionali (Combinators)

Questi selettori colpiscono gli elementi in base alla relazione che hanno con altri elementi.

- **Selettore Discendente (Spazio):** Seleziona un elemento che si trova *ovunque* all'interno di un altro.

```
1 /*Seleziona tutti gli <a> che sono dentro un <article>, a qualsiasi livello di profondità*/
2 article a { color: red; }
```

- **Selettore Figlio Diretto (>):** Seleziona un elemento che è figlio *diretto* (di primo livello) di un genitore.

```
1 /*Seleziona gli <li> che sono figli diretti di .menu, ma non quelli annidati in sottomenù*/
2 .menu > li { list-style-type: none; }
```

- **Selettore Fratello Adiacente (+):** Seleziona un elemento che segue *immediatamente* un altro, condividendo lo stesso genitore.

```
1 /*Applica margine solo al primo paragrafo che segue un h2*/  
2 h2 + p { margin-top: 0; }
```

2.4.2 Pseudo-classi (Stati dell'elemento)

Le pseudo-classi iniziano con i due punti `:` e permettono di stilizzare un elemento in base a uno specifico stato dell'interfaccia o alla sua posizione.

Pseudo-classi interattive fondamentali:

- `:hover`: Si attiva quando il cursore del mouse passa sopra l'elemento.
- `:focus`: Si attiva quando un elemento riceve il focus (es. cliccando in un campo di testo o navigando con il tasto Tab).
- `:active`: Si attiva nel momento esatto in cui l'utente fa clic sull'elemento.

```
1 /*Esempio Pratico: Effetto hover sui bottoni */  
2 .btn {  
3     background-color: blue;  
4 }  
5 .btn:hover {  
6     background-color: darkblue; /* Cambia colore al passaggio del mouse*/  
7 }
```

2.5 The Cascade (La Cascata) e l'Ereditarietà

Il termine "Cascading" nel CSS indica l'algoritmo che il browser usa per risolvere i conflitti quando più regole cercano di stilizzare lo stesso identico elemento.

2.5.1 L'Ereditarietà (Inheritance)

Alcune proprietà CSS (tipicamente quelle testuali come `color`, `font-family`, `font-size`) vengono passate automaticamente dal contenitore padre a tutti i figli. Altre proprietà (come `border`, `margin`, `padding` e `background`) **non** vengono ereditate per evitare layout caotici.

2.5.2 La Specificità (Specificity)

Quando due regole sono in conflitto, vince quella con il selettore più "specifico" o "pesante".

Come si calcola il peso (Regola per gli esercizi d'esame):

Il calcolo si basa su quattro slot: (`Stile Inline`, `ID`, `Classi/Pseudo-classi`, `Elementi/Pseudo-elementi`)

1. **Stile Inline** batte tutto (1, 0, 0, 0).
2. **ID (#)** è fortissimo (0, 1, 0, 0).
3. **Classi (.) e Pseudo-classi (:)** hanno un peso medio (0, 0, 1, 0).
4. **Elementi (tag)** sono i più deboli (0, 0, 0, 1).

```

1  /*ESERCIZIO PRATICO DI CONFLITTO*/
2
3  /*Regola A: Specificità (0, 0, 0, 1)*/
4  p { color: black; }
5
6  /*Regola B: Specificità (0, 0, 1, 0) -> VINCE LA CLASSE!*/
7  .evidenziato { color: red; }
8
9  /*Regola C: Specificità (0, 1, 0, 0) -> VINCE L'ID SU TUTTO!*/
10 # paragrafo-speciale { color: blue; }

```

La regola !important: Se si aggiunge **!important** alla fine di una dichiarazione, questa sovrascrive qualsiasi altra regola, ignorando il calcolo della specificità. **Va usato con estrema parsimonia**, solo per fix di emergenza, poiché rompe il naturale flusso a cascata.

2.6 Il Box Model (Modello a Scatola)

Il Box Model è il concetto più importante del Web Design. Il browser tratta **ogni singolo elemento HTML** come se fosse racchiuso in una scatola invisibile rettangolare.

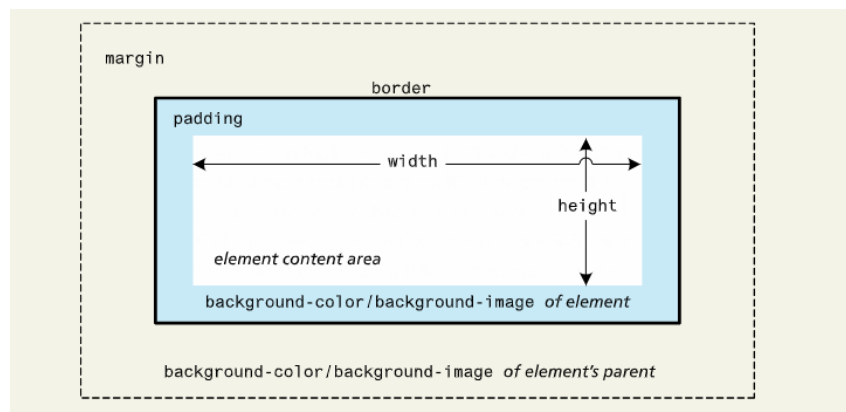


Figura 2.2: Diagramma visivo del CSS Box Model. Mostra un rettangolo centrale “Content”, circondato dall’area “Padding”, a sua volta circondata dal “Border”, circondato all’esterno dal “Margin”.

2.6.1 Le 4 componenti della scatola

1. **Content (Contenuto):** L’area effettiva in cui risiedono il testo, le immagini o i figli dell’elemento. Si manipola con **width** (larghezza) e **height** (altezza).
2. **Padding (Spaziatura Interna):** Lo spazio trasparente *tra* il contenuto e il bordo della scatola. Aggiungere padding allarga la scatola (nello standard model).
3. **Border (Bordo):** Una linea (visibile o invisibile) che delimita la fine della scatola.
4. **Margin (Margine Esterno):** Lo spazio trasparente *all’esterno* del bordo, che allontana la scatola dagli altri elementi adiacenti.

```

1  /*Esempio Pratico: Manipolazione del Box Model */
2  .scheda {
3      width: 300px;
4      padding: 20px; /* Distanza il testo dal bordo */
5      border: 2px solid black; /* Rende la scatola visibile */
6      margin-bottom: 30px; /* Crea spazio sotto la scheda */
7  }

```

2.6.2 La rivoluzione del box-sizing

Per impostazione predefinita (`box-sizing: content-box`), se imposti una larghezza di `300px` e poi aggiungi `20px` di padding per lato e `2px` di bordo, la larghezza reale diventerà `344px`. Questo complica terribilmente la creazione di layout a griglia.

La best-practice moderna è cambiare questo comportamento globale applicando `box-sizing: border-box`. In questo modo, la larghezza che imposti (es. `300px`) include **già** padding e bordo, stringendo automaticamente il contenuto interno verso il centro e mantenendo la scatola esattamente di `300px`.

```
1 /*Reset CSS fondamentale che quasi tutti usano oggi*/
2 - {
3     box-sizing: border-box;
4 }
```

2.7 Proprietà del Box Model: Sfondi, Bordi e Spaziature

Dopo aver compreso che ogni elemento HTML è una “scatola”, esploriamo come il CSS permette di manipolare le singole componenti di questo Box Model, partendo dallo sfondo fino ad arrivare ai margini esterni.

2.7.1 Gestione dello Sfondo (Background)

Lo sfondo di un elemento riempie l'intera area del contenuto e del padding, spingendosi fino al bordo (incluso).

Le proprietà principali per manipolare gli sfondi sono:

- `background-color`: Imposta un colore a tinta unita (es. `silver`, `#FF0000`, `rgba(0,0,0,0.5)`).
- `background-image`: Carica un'immagine esterna. La sintassi richiede la funzione `url()`.
- `background-repeat`: Gestisce la ripetizione dell'immagine se è più piccola del contenitore. Valori: `repeat` (default, piastrella ovunque), `repeat-x` (solo asse orizzontale), `repeat-y` (solo asse verticale), `no-repeat`.
- `background-position`: Sposta l'immagine all'interno del box (es. `center`, `top left`, oppure in pixel/percentuali come `30px 50px`).

```
1 /*Esempio Pratico: Sfondo combinato*/
2 .hero-section {
3     background-color: #f4f4f4;
4     background-image: url('../images/pattern.png');
5     background-repeat: repeat-x;
6     background-position: top center;
7 }
8
9 /*Esiste anche la proprietà shorthand "background" per scrivere tutto su una riga*/
10 .hero-section-shorthand {
11     background: #f4f4f4 url('../images/pattern.png') repeat-x top center;
12 }
```

2.7.2 Bordi e la Notazione Shorthand (TRBL)

Il bordo delimita visivamente la scatola. Per definirlo correttamente servono tre valori: spessore, stile (es. `solid`, `dashed`, `dotted`) e colore.

La Regola dell'Orologio (TRBL per gli esami):

Quando si assegnano valori ai quattro lati di un elemento (applicabile a bordi, margini e padding), il CSS segue rigorosamente un ordine orario: **T**op (Alto), **R**ight (Destra), **B**ottom (Basso), **L**eft (Sinistra). L'acronimo da memorizzare è **TRBL** (pronunciato "Trouble").



Figura 2.3: Diagramma dell'orologio che punta in senso orario illustrando l'acronimo TRBL (Top, Right, Bottom, Left).

```

1  /*Esempio Pratico: Notazione TRBL*/
2
3  /*4 valori: Top(10px) Right(20px) Bottom(30px) Left(40px)*/
4  .box-asimmetrico {
5      margin: 10px 20px 30px 40px;
6  }
7
8  /*2 valori: Top/Bottom(10px) e Right/Left(20px) -> Molto usato per centrare!*/
9  .box-centrato {
10     margin: 10px 20px;
11 }
12
13 /*Shorthand per il bordo su tutti i lati*/
14 .box-evidenziato {
15     border: 2px solid red;
16 }

```

2.8 Il Comportamento dei Margini (Collapsing Margins)

Questo è uno dei concetti più ingannevoli del CSS e **spesso oggetto di domande trabocchetto negli scritti**.

Cosa succede quando due elementi HTML si susseguono verticalmente e ad entrambi è stato assegnato un margine?

Regola del Collasso: In CSS, i margini **verticali** (Top e Bottom) adiacenti di due blocchi non si sommano matematicamente. Al contrario, essi "collassano": il browser prende solo il valore più grande tra i due e scarta il più piccolo. I margini orizzontali, invece, non collassano mai.

```

1  /*Esempio Pratico: Margini Collassati*/
2  .paragrafo-uno {
3      margin-bottom: 90px;
4  }
5
6  .paragrafo-due {
7      margin-top: 50px;
8  }
9  /*Se il "paragrafo-due" segue immediatamente il "paragrafo-uno",
10 lo spazio totale tra i due NON sarà 140px, ma esattamente 90px
11 (il valore maggiore vince, il 50px scompare).*/

```

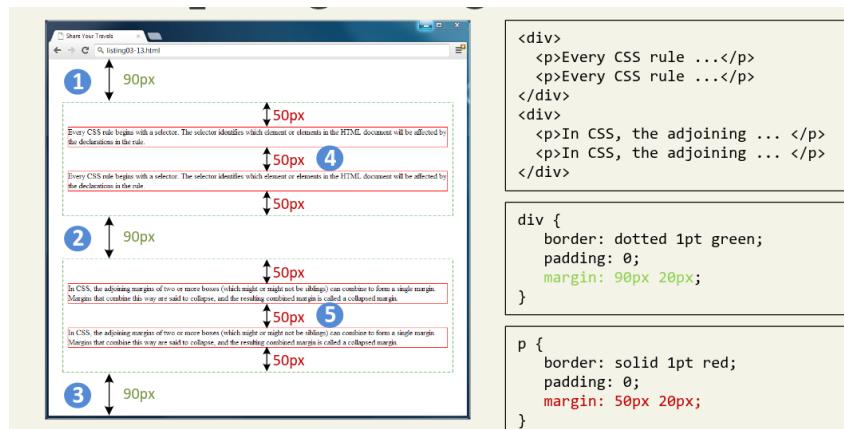


Figura 2.4: Schema visuale con frecce rosse che dimostrano fisicamente il collasso dei margini verticali tra box (90px e 50px che diventano uno spazio unico da 90px).

2.9 Gestione dell'Overflow

Cosa succede se specifichi larghezza (*width*) e altezza (*height*) fisse per un elemento, ma il testo al suo interno è troppo lungo per starci? Il comportamento è regolato dalla proprietà *overflow*.

- **overflow: visible:** (Default). Il testo sfora i confini del box e continua ad essere visibile sovrapponendosi agli elementi sottostanti (rompendo il layout).
- **overflow: hidden:** Il testo in eccesso viene semplicemente tagliato in modo netto al confine del box e scompare.
- **overflow: scroll:** Vengono forzate le barre di scorrimento (sia orizzontali che verticali) sul box, indipendentemente dal fatto che servano o meno.
- **overflow: auto:** Il browser valuta dinamicamente: inserisce le barre di scorrimento solo e unicamente se il contenuto supera effettivamente le dimensioni del box.

2.10 Tipografia e Gestione dei Font

Il CSS separa la stilizzazione tipografica in due categorie: proprietà che influenzano il testo stesso (allineamento, spaziatura) e proprietà che alterano il font.

2.10.1 Font-Family e lo Stack

Non puoi avere la certezza che l'utente che visita il sito abbia installato sul proprio computer il font esoterico che hai scelto. Per questo motivo, si definisce sempre una lista di font (Font Stack) separati da virgola. Il browser leggerà la lista da sinistra verso destra finché non trova un font installato nel sistema.

```

1 /*Esempio Pratico: Font Stack */
2 p {
3   /* 1. Prova Cambria. Se manca, 2. Prova Georgia. Se manca, 3. Usa un serif di base*/
4   font-family: Cambria, Georgia, serif;
5 }

```

I 5 Font Generici di salvataggio (Fallback):

L'ultimo elemento di uno stack deve **sempre** essere una famiglia generica universale:

1. **serif**: Caratteri con “grazie” (gli allungamenti alle estremità delle lettere, es. Times New Roman).
2. **sans-serif**: Caratteri “bastoni” senza grazie, più leggibili a schermo (es. Arial).
3. **monospace**: Tutti i caratteri hanno l'esatta stessa larghezza. Usato per i blocchi di codice.
4. **cursive**: Font che simulano la scrittura a mano.
5. **fantasy**: Font decorativi esagerati.

2.10.2 Introduzione a @font-face

Oggi il problema dei font non installati è superato dai **Web Fonts** (come *Google Fonts*). Tramite la regola **@font-face** (o importando un foglio di stile esterno dal provider), si obbliga il browser a scaricare temporaneamente il font dal server insieme alla pagina web.

```
1 /*Esempio Pratico: Ombreggiatura Testo (Avanzato) */
2 h1 {
3   /* offset-X, offset-Y, sfocatura (blur), colore*/
4   text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.5);
5 }
```

2.11 L'Incubo delle Unità di Misura: px, em, % e rem

La gestione delle grandezze dei font (e dei box) è fondamentale per il design responsivo (adattabile a PC e Smartphone). I pixel (**px**) pongono problemi sui dispositivi moderni ad alta densità (Retina display), spingendo verso unità relative.

2.11.1 Il Problema del Compounding (Effetto a Cascata)

Se si usano le percentuali % o l'unità **em** (relativa alla tipografia), la dimensione viene calcolata **in relazione alla dimensione del font del genitore diretto**.

Se il font base del browser è 16px:

- **font-size: 1.5em**; significa $1.5 \times 16 = 24px$.

Il problema (compounding) nasce quando si annidano gli elementi:

```
1 <style>
2   div { font-size: 75%; } /* Applica il 75% della dimensione del padre */
3 </style>
4
5 <div>
6   Testo a 12px
7   <div>Testo a 9px</div>
8 </div>
```

2.11.2 La Soluzione Definitiva: L'unità rem

Il CSS3 ha introdotto un salvavita: l'unità **rem** (Root em). Il **rem** non calcola la sua dimensione in base all'elemento padre, ma si riferisce in modo assoluto, rigido e unico al root element del documento (ovvero il tag **<html>**, che di base è sempre 16px).

Utilizzando `font-size: 1.5rem`, il testo sarà **sempre** di 24px (se il root è 16), indipendentemente da quanto profondamente l'elemento sia annidato dentro altri div con font rimpiccioliti. Elimina totalmente il rischio di compounding.

3 HTML - Tabelle e Forms(04)

3.1 Struttura Base delle Tabelle HTML

Le tabelle in HTML servono per rappresentare dati strutturati in una griglia bidimensionale di righe e colonne.

Errore da Esame: Le tabelle devono essere utilizzate **esclusivamente** per esporre dati tabellari (come calendari, bilanci finanziari, orari, inventari). Negli anni '90 venivano usate per costruire il layout dell'intera pagina web, pratica oggi severamente vietata e penalizzata dai motori di ricerca e dagli screen reader. Il layout si gestisce solo con il CSS.

L'architettura di una tabella si basa su quattro tag fondamentali:

- `<table>`: L'elemento contenitore globale che definisce la tabella.
- `<tr>` (Table Row): Definisce una singola riga orizzontale. Deve essere inserito all'interno di `<table>` (o dei suoi raggruppamenti semantici).
- `<th>` (Table Header): Definisce una cella di intestazione. Di default, i browser ne formattano il testo in grassetto e centrato.
- `<td>` (Table Data): Definisce una cella di dati standard all'interno di una riga.

Le colonne in HTML non vengono definite esplicitamente con un tag "colonna". Sono invece un risultato implicito: il numero di celle (`<td>` o `<th>`) inserite all'interno di una riga (`<tr>`) determina il numero di colonne di quella riga.

```
1 <table>
2   <tr>
3     <th>Nome</th>
4     <th>Ruolo</th>
5   </tr>
6   <tr>
7     <td>Mario</td>
8     <td>Sviluppatore</td>
9   </tr>
10  <tr>
11    <td>Luigi</td>
12    <td>Designer</td>
13  </tr>
14 </table>
```

3.1.1 Unione di Celle (Spanning)

Spesso è necessario che una singola cella si estenda per occupare lo spazio di più righe o più colonne adiacenti. Questo si ottiene tramite due attributi applicabili ai tag `<td>` o `<th>`:

- `colspan="n"`: Estende la cella orizzontalmente per "n" colonne.
- `rowspan="n"`: Estende la cella verticalmente verso il basso per "n" righe.

Quando si utilizza `colspan` o `rowspan`, le celle che verrebbero normalmente occupate in quello spazio **devono essere omesse** nel codice HTML, altrimenti la griglia si "romperà", spingendo le celle in eccesso fuori dai bordi della tabella.

Each row must have the same number of `<td>` or `<th>` containers. If you want a given cell to cover several columns or rows:

Title	Artist	Year	Size (width x height)	
The Death of Marat	Jacques-Louis David	1793	162cm	128cm
Burial at Ornans	Gustave Courbet	1849	314cm	663cm

Notice that this row now only has four cell elements.

use the **colspan** or **rowspan** attributes

```

<table>
<tr>
<th>Title</th>
<th>Artist</th>
<th>Year</th>
<th colspan="2">Size (width x height)</th>
</tr>
<tr>
<td>The Death of Marat</td>
<td>Jacques-Louis David</td>
<td>1793</td>
<td>162cm</td>
<td>128cm</td>
</tr>
<tr>
<td>Burial at Ornans</td>
<td>Gustave Courbet</td>
<td>1849</td>
<td>314cm</td>
<td>663cm</td>
</tr>
</table>

```

Figura 3.1: Schema visivo di una tabella con una cella allungata orizzontalmente (`colspan="2"`).

3.2 Struttura Semantica Avanzata e Accessibilità

Per tabelle complesse, HTML5 fornisce tag strutturali che permettono di raggruppare logicamente le righe, migliorando l'accessibilità e facilitando l'applicazione di stili CSS mirati.

3.2.1 Raggruppamento delle Righe

- `<thead>`: Racchiude le righe di intestazione della tabella.
- `<tbody>`: Racchiude le righe contenenti i dati principali. Una singola tabella può contenere più elementi `<tbody>` (ad esempio per dividere i dati in categorie).
- `<tfoot>`: Racchiude le righe di riepilogo o i totali (solitamente posizionato alla fine, ma semanticamente può apparire prima del `<tbody>` nel codice sorgente).

Vantaggio in fase di stampa: Se una tabella è molto lunga e si estende su più pagine stampate, i browser intelligenti ripeteranno automaticamente il contenuto del `<thead>` e del `<tfoot>` in cima e in fondo a ogni pagina fisica.

3.2.2 Titolo e Gruppi di Colonne

- `<caption>`: Fornisce un titolo descrittivo alla tabella. Deve essere obbligatoriamente il primo elemento figlio subito dopo il tag di apertura `<table>`.
- `<colgroup>` e `<col>`: Permettono di applicare attributi o stili CSS a intere colonne verticali, senza dover assegnare classi a ogni singolo `<td>`.

```

1 <table>
2   <caption>Bilancio Annuale 2026</caption>
3   <colgroup>
4     <col style="background-color: lightgray;" <col style="background-color: white;" </colgroup>
5   <thead>
6     <tr>
7       <th>Mese</th>
8       <th>Entrate</th>
9     </tr>
10  </thead>
11  <tbody>
12    <tr>
13      <td>Gennaio</td>

```

```
14         <td>€ 5000</td>
15     </tr>
16 </tbody>
17 <tfoot>
18     <tr>
19         <th>Totale</th>
20         <th>€ 5000</th>
21     </tr>
22 </tfoot>
23 </table>
```

3.3 Stilizzazione delle Tabelle (CSS)

Di default, i browser renderizzano le tabelle senza bordi visibili e con spaziature minime, risultando difficili da leggere.

3.3.1 Il problema dei bordi doppi (border-collapse)

Se si applica un bordo sia alla tabella (<table>) che alle celle (<th>, <td>), il browser disegnerà i bordi in modo separato, lasciando uno spazio vuoto tra una cella e l'altra (creando un antiestetico effetto "doppio bordo").

La proprietà CSS `border-collapse: collapse;` applicata al tag <table> risolve questo problema: forza i bordi adiacenti a fondersi in una singola linea continua, creando il classico aspetto da foglio di calcolo professionale.

```
1  /*Esempio Pratico: Reset standard per tabelle */
2  table {
3      border-collapse: collapse; /* Fonde i bordi doppi */
4      width: 100%;             /* Estende la tabella per tutto il contenitore*/
5  }
6
7  th, td {
8      border: 1px solid black; /*Applica il bordo alle singole celle */
9      padding: 10px;          /* Distanza il testo dai bordi */
10     text-align: left;        /* Allinea il testo a sinistra (default th è centrato)*/
11 }
```

3.3.2 Strisce Zebra (Zebra Striping)

Una tecnica essenziale per migliorare la leggibilità delle tabelle con molte righe consiste nell'alternare il colore di sfondo delle righe (una chiara, una scura). Questo si realizza senza aggiungere classi HTML, sfruttando le pseudo-classi CSS.

```
1  /*Seleziona tutte le righe pari (even) all'interno del body della tabella*/
2  tbody tr:nth-child(even) {
3      background-color: #f2f2f2;
4  }
5
6  /*Effetto interattivo al passaggio del mouse sull'intera riga*/
7  tbody tr:hover {
8      background-color: #e0e0e0;
9  }
```

3.4 Introduzione ai Form HTML

I form (moduli) sono il meccanismo principale attraverso il quale gli utenti inviano dati al server web. Costituiscono l'elemento base per la creazione di applicazioni web interattive, permettendo operazioni come il login, la registrazione, l'inserimento di commenti o gli acquisti.

3.4.1 Il contenitore globale: l'elemento form

Tutti i controlli interattivi (campi di testo, pulsanti, menù a tendina) devono essere racchiusi all'interno del tag `<form>`. Questo elemento non ha una rappresentazione visiva autonoma, ma funge da "involucro" logico per i dati da inviare.

Gli attributi fondamentali del tag `<form>` sono due:

- **action:** Specifica l'URL dello script lato server (es. un file PHP o un endpoint API) che riceverà e processerà i dati inviati. Se omissso, i dati vengono ricaricati e inviati alla pagina corrente.
- **method:** Definisce il protocollo HTTP utilizzato per impacchettare e inviare i dati. I valori standard possibili sono `GET` o `POST`.

3.4.2 Metodi di Invio: GET vs POST

La scelta del metodo è una tipica domanda da esame e ha implicazioni cruciali sulla sicurezza e sul funzionamento dell'applicazione.

- **GET:** I dati del form vengono accodati direttamente all'URL della pagina di destinazione sotto forma di *query string*, risultando visibili in chiaro (es. `elabora.php?nome=Mario&eta=25`).
 - *Uso:* Ideale per richieste idempotenti che non alterano i dati sul server, come i motori di ricerca o il filtraggio di prodotti in un e-commerce.
 - *Limiti:* Fortemente sconsigliato per dati sensibili (come le password), limitato nella quantità di dati inviabili, i dati restano salvati nella cronologia del browser.
- **POST:** I dati vengono inviati all'interno del corpo (body) della richiesta HTTP, nascosti all'utente.
 - *Uso:* Ideale e obbligatorio per l'invio di dati sensibili, file allegati, o per compiere azioni che modificano lo stato del server (es. registrazione utente, salvataggio di un articolo).



Figura 3.2: Diagramma concettuale del flusso Client-Server che illustra visivamente la differenza tra una richiesta HTTP GET (con le variabili incapsulate nell'URL) e una richiesta HTTP POST (con le variabili all'interno del pacchetto HTTP).

3.5 I Controlli di Input Standard

L'elemento `<input>` è il tag più versatile per l'acquisizione dei dati. È un elemento vuoto (senza tag di chiusura) e la sua interfaccia e funzionalità cambiano radicalmente in base al valore assegnato all'attributo `type`.

3.5.1 Campi Testuali e Segreti

- `type="text"`: Crea un campo a riga singola per l'inserimento di testo generico.
- `type="password"`: Identico al campo testuale, ma maschera visivamente i caratteri digitati (solitamente con pallini o asterischi) per impedire la lettura a chi sta guardando lo schermo.

3.5.2 Scelte Multiple e Singole

- `type="radio"`: Crea un pulsante di opzione circolare. Si usa quando l'utente deve selezionare **una sola** opzione all'interno di un gruppo esclusivo di scelte. > **Attenzione per i progetti**: Per raggruppare i radio button in modo che la selezione di uno deselecti automaticamente gli altri, tutti i tag `<input>` del gruppo devono condividere l'esatto stesso valore per l'attributo `name`.
- `type="checkbox"`: Crea una casella di spunta quadrata. Si usa quando l'utente può selezionare **zero, una o più** opzioni indipendenti contemporaneamente.

3.5.3 Pulsanti di Azione

- `type="submit"`: Genera un pulsante che, se cliccato dall'utente, raccoglie tutti i dati validi e innesca l'invio (submit) del form verso l'URL specificato in `action`.
- `type="reset"`: Svuota istantaneamente tutti i campi del form, riportandoli ai valori predefiniti iniziali.

L'attributo **name** è assolutamente vitale per qualsiasi controllo di form. Quando il modulo viene inviato al server, i dati viaggiano sotto forma di coppie chiave/valore. Il `name` diventa la chiave (la variabile che il backend andrà a cercare e leggere), mentre il contenuto digitato o selezionato dall'utente diventa il `value`. Un input senza `name` viene semplicemente ignorato durante l'invio.

```
1 <form action="/login" method="POST">
2   <input type="text" name="username" placeholder="Inserisci il tuo nome">
3   <input type="password" name="pwd">
4
5   <input type="radio" name="genere" value="M"> Uomo
6   <input type="radio" name="genere" value="F"> Donna
7
8   <input type="submit" value="Accedi">
9 </form>
```

3.6 Evoluzione: I Tipi di Input in HTML5

HTML5 ha introdotto una vasta gamma di nuovi valori per l'attributo `type`. Questi nuovi input portano due grandi vantaggi: migliorano l'esperienza utente (specialmente sui dispositivi mobili, richiamando tastiere virtuali ad hoc) e delegano la validazione sintattica di base direttamente al browser, riducendo la necessità di complessi script JavaScript.

3.6.1 Input Dati Specifici

- `type="email"`: Forza la validazione del formato email (esige la presenza della @ e di un dominio valido). Su smartphone fa apparire la tastiera con la @ a portata di mano.
- `type="url"`: Richiede l'inserimento di un indirizzo web formalmente corretto (incluso il protocollo `http://` o `https://`).
- `type="number"`: Consente l'inserimento esclusivo di valori numerici, solitamente fornendo controlli a freccia per incrementare o decrementare il valore. Lavora in sinergia con gli attributi limitatori `min`, `max` e `step`.
- `type="tel"`: Ottimizzato per l'inserimento di numeri di telefono (richiama il tastierino numerico sui dispositivi mobili).

3.6.2 Controlli Interattivi e Selettori

- `type="range"`: Renderizza uno slider (un cursore scorrevole orizzontale) utile per selezionare un valore numerico approssimativo all'interno di un intervallo.
- `type="color"`: Invoca il *color picker* (selettore cromatico) nativo del sistema operativo dell'utente, restituendo il valore esadecimale del colore.
- `type="date"`, `type="time"`, `type="month"`: Generano controlli per il calendario (Datepicker) e selettori temporali nativi. Garantiscono l'assoluta precisione della formattazione della data prima dell'invio.

3.7 Elementi Strutturali e Accessibilità

Un form robusto e professionale non fa affidamento solo su `<input>`. L'HTML fornisce tag semantici specifici per menù, testi lunghi e, soprattutto, per garantire la massima accessibilità (A11y).

3.7.1 Menù a Tendina: select e option

L'elemento `<select>` genera un menù a discesa, mentre una serie di tag `<option>` annidati definiscono le singole voci selezionabili. L'attributo `name` va inserito sul `<select>`, mentre il `value` su ogni singola `<option>`.

Utilizzando l'attributo `selected` su un tag `<option>`, si forza il browser a mostrare quel valore come pre-selezionato al momento del caricamento della pagina.

```
1 <select name="nazione">
2   <option value="it">Italia</option>
3   <option value="fr" selected>Francia</option>
4   <option value="de">Germania</option>
5 </select>
```

3.7.2 Aree di Testo Multilinea: textarea

A differenza dell'input testuale a riga singola, il tag `<textarea>` permette l'inserimento di paragrafi multipli di testo (ideale per recensioni, commenti o messaggi di contatto). Essendo un tag contenitore, a differenza di `<input>`, richiede obbligatoriamente il tag di chiusura `</textarea>`.

3.7.3 L'importanza vitale del tag label

L'elemento `<label>` definisce un'etichetta di testo descrittiva per un campo di form. L'associazione logica avviene collegando il valore dell'attributo `for` della label con l'attributo `id` dell'input bersaglio.

Perché le label sono cruciali in sede d'esame e nel progetto?

Oltre a essere mandatorie per far comprendere la pagina agli screen reader degli ipovedenti, associare una `<label>` a un input (specialmente per checkbox e radio button) espande l'area "cliccabile". L'utente potrà cliccare direttamente sul testo dell'etichetta per spuntare la piccola casellina, migliorando drasticamente la comodità e l'usabilità (UX) dell'interfaccia.

```
1 <input type="checkbox" id="privacy-policy" name="accetta_privacy" value="si">
2 <label for="privacy-policy">Ho letto e accetto i termini di servizio</label>
```

3.7.4 Raggruppamento Logico: fieldset e legend

Quando un form contiene molti campi, è buona prassi architetturale suddividerlo in macro-sezioni tematiche (es. "Dati di Spedizione", "Dati di Fatturazione"). L'elemento `<fieldset>` crea un raggruppamento strutturale e visivo dei controlli, mentre il tag annidato `<legend>` fornisce una didascalia (titolo) integrata direttamente nel bordo del riquadro generato dal fieldset.

3.8 Controlli Specializzati e Upload File

Esistono input progettati per gestire dati non testuali o informazioni di sistema che non richiedono l'interazione diretta dell'utente, ma che sono vitali per la logica dell'applicazione.

3.8.1 Campi Nascosti (Hidden)

L'elemento `<input type="hidden">` crea un campo che è completamente invisibile all'utente sulla pagina web.

- **Scopo:** Viene utilizzato dagli sviluppatori per inviare al server dati di "stato" o di contesto (come l'ID del record che si sta modificando in un database, o un token di sicurezza) insieme al resto del form, senza che l'utente debba vederli o modificarli.

3.8.2 Caricamento File (File Upload)

L'elemento `<input type="file">` genera un controllo (solitamente un pulsante "Scegli file") che apre la finestra di dialogo del sistema operativo per permettere all'utente di selezionare un file dal proprio dispositivo da allegare al modulo.

Regola d'Oro per gli Esami e i Progetti:

Se un modulo contiene un `<input type="file">`, l'attributo `method` del `<form>` **deve** essere obbligatoriamente impostato su `POST`. Inoltre, è assolutamente mandatorio aggiungere l'attributo `enctype="multipart/form-data"` al tag `<form>`. Senza questa codifica speciale, il browser invierà al server solo il *nome* del file (come testo) e non il file binario vero e proprio!

```
1 <form action="/upload" method="POST" enctype="multipart/form-data">
2   <label>Carica la tua foto profilo:</label>
3   <input type="file" name="foto_utente">
4   <input type="submit" value="Invia File">
5 </form>
```

3.9 Pulsanti Avanzati (Button Controls)

Oltre ai classici `<input type="submit">` e `<input type="reset">`, l'HTML fornisce opzioni più flessibili per gestire i bottoni dell'interfaccia.

3.9.1 L'elemento button

Il tag `<button>` offre una libertà architetture netta superiore rispetto a `<input>`. Essendo un elemento contenitore (dotato di tag di apertura e chiusura `</button>`), al suo interno non è limitato a contenere solo testo, ma può ospitare tag HTML completi, come icone, immagini (`<img>`), o testo formattato con `<strong>` ed `<em>`.

Per definire il comportamento del `<button>`, si utilizza l'attributo `type` che accetta tre valori: `submit` (invia il form, comportamento di default), `reset` (svuota il form), oppure `button` (non fa nulla da solo, va agganciato a uno script JavaScript).

3.9.2 Input di tipo Image

Il tag `<input type="image">` si comporta esattamente come un bottone di submit, ma utilizza un'immagine personalizzata per la sua visualizzazione visiva. Richiede l'attributo `src` (come un normale tag immagine) per puntare al file grafico.

```
1 <input type="submit" value="Salva">
2
3 <button type="submit">
4    Salva Modifiche
5 </button>
```

3.10 Autocompletamento e Validazione Personalizzata

HTML5 ha introdotto strumenti per assistere l'utente durante la digitazione e per forzare regole di validazione sintattiche strette prima ancora che i dati partano verso il server.

3.10.1 L'elemento datalist

Il tag `<datalist>` permette di creare un menù a tendina ibrido. Viene collegato a un normale campo di testo per fornire all'utente un elenco di suggerimenti predefiniti per l'autocompletamento, pur lasciandolo libero di digitare un valore non presente in lista.

Il collegamento logico si effettua assegnando un ID univoco al `<datalist>` e inserendo quello stesso ID nell'attributo `list` dell'elemento `<input>` di testo associato.

```
1 <label>Cerca Città:</label>
2 <input type="text" name="citta" list="citta_suggestite">
3
4 <datalist id="citta_suggestite">
5   <option value="Roma">
6   <option value="Milano">
7   <option value="Napoli">
8 </datalist>
```

3.10.2 Validazione tramite Espressioni Regolari (Pattern)

Per campi come codici postali, codici fiscali o numeri di matricola, il semplice controllo sul tipo di dato non basta. L'attributo `pattern` accetta un'espressione regolare (Regex) che il valore digitato deve rigorosamente rispettare affinché il form possa essere inviato. Ad esempio: `pattern="[A-Z]{3}[0-9]{3}"` forzerebbe l'inserimento di 3 lettere maiuscole seguite da 3 numeri.

3.11 Elementi di Feedback e Output (HTML5)

Infine, HTML5 definisce dei tag semantici specifici per mostrare risultati o stati di avanzamento, progettati per essere aggiornati dinamicamente tramite JavaScript.

- `<progress>`: Rappresenta lo stato di completamento di un task in corso (es. una barra di caricamento per un file in upload o per l'installazione di un modulo).
- `<meter>`: Rappresenta una misurazione scalare all'interno di un range noto o un valore frazionario (es. lo spazio su disco occupato, o la rilevanza di un risultato di ricerca). Non va usato per indicare un progresso.
- `<output>`: Identifica esplicitamente un'area in cui verrà iniettato il risultato di un calcolo matematico o l'esito di un'azione dell'utente elaborata da uno script.

4 CSS Avanzato - Layout(05)

4.1 Il Flusso Normale (Normal Flow)

Il *Normal Flow* descrive il comportamento predefinito con cui il browser posiziona e renderizza gli elementi HTML all'interno della pagina, dall'alto verso il basso e da sinistra a destra.

4.1.1 Elementi a Blocco (Block-level) e Inline

Il flusso normale gestisce due macro-categorie strutturali di elementi:

- **Elementi Block-level** (es. `<p>`, `<div>`, `<h1>`): Occupano per impostazione predefinita l'intera larghezza del loro contenitore genitore. Iniziano sempre su una nuova riga e si impilano verticalmente uno sotto l'altro.
- **Elementi Inline** (es. `<a>`, `<span>`, `<strong>`, `<img>`): Non formano un blocco isolato e non forzano un ritorno a capo. Si dispongono orizzontalmente l'uno accanto all'altro all'interno della linea di testo, per poi andare a capo se lo spazio finisce (effetto *reflow* al ridimensionamento della finestra).

Manipolazione del Flusso: È possibile alterare la natura nativa di un elemento tramite la proprietà CSS `display`. Impostando `display: block`; su uno `<span>`, questo inizierà a comportarsi come un `<div>`, mentre `display: inline`; su un `<li>` lo disporrà orizzontalmente sulla stessa riga.

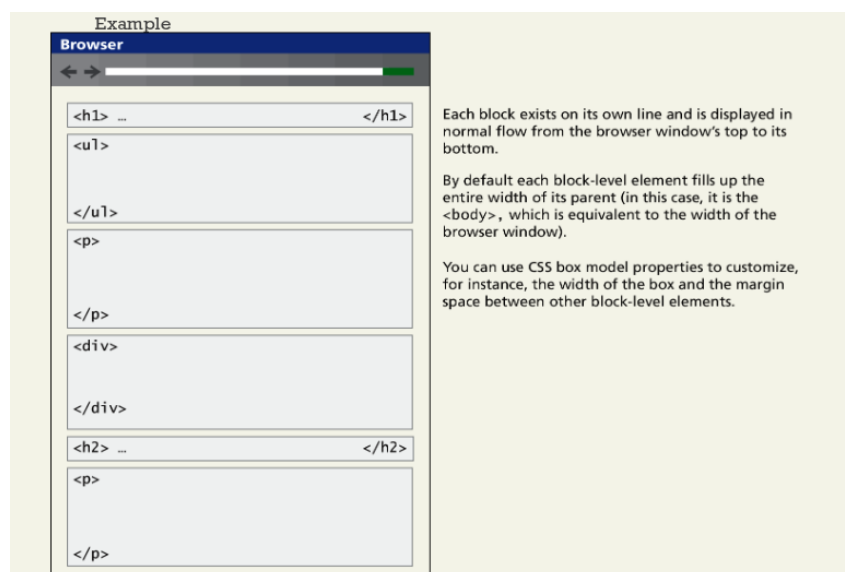


Figura 4.1: Schema visivo del "Normal Flow" (Browser view che mostra blocchi

4.2 Posizionamento degli Elementi (Positioning)

Oltre al flusso normale, il CSS offre la proprietà `position` per spostare esplicitamente un elemento, estraendolo parzialmente o totalmente dal suo contesto originale (ideale per sovrapposizioni e layout complessi).

Di base, ogni elemento nasce con `position: static`; (segue il Normal Flow). Cambiando questo valore, si sbloccano le proprietà direzionali `top`, `bottom`, `left` e `right`.

4.2.1 Posizionamento Relativo (Relative)

Un elemento con `position: relative`; viene spostato rispetto alla sua posizione originale nativa.

La caratteristica fondamentale e distintiva del posizionamento relativo è che **lo spazio originariamente occupato dall'elemento viene preservato**. Gli altri elementi circostanti non collassano per riempire il vuoto, comportandosi come se l'elemento si trovasse ancora nella sua posizione nativa.

```
1 /*Esempio Pratico: Posizionamento Relativo */
2 .box-spostato {
3   position: relative;
4   top: 20px; /* Spinge l'elemento verso il basso di 20px rispetto a dove dovrebbe essere */
5   left: 50px; /* Lo spinge verso destra di 50px*/
6 }
```

4.2.2 Posizionamento Assoluto (Absolute)

Un elemento con `position: absolute`; viene **completamente rimosso** dal flusso normale. Lo spazio che occupava originariamente scompare e viene riempito dagli elementi adiacenti, come se l'elemento fosse sparito.

La Regola del “Positioning Context” (Cruciale per i Progetti):

Un elemento assoluto calcola le sue coordinate (`top`, `left`, ecc.) rispetto al suo **primo antenato posizionato** (ovvero il primo genitore che ha un valore di `position` diverso da `static`). Se non trova alcun genitore posizionato risalendo l'albero DOM, prenderà come riferimento il documento principale (il viewport). La best practice per confinare e controllare un elemento assoluto è assegnare `position: relative`; al suo contenitore genitore diretto.

```
1 <figure style="position: relative; width: 300px;">
2   
3
4   <span style="position: absolute; top: 10px; right: 10px; background: red;">
5     NOVITA
6   </span>
7 </figure>
```

4.2.3 Posizionamento Fisso (Fixed)

Estremamente simile al posizionamento assoluto, ma l'elemento viene posizionato **esclusivamente rispetto al viewport del browser** (la finestra fisica visibile sullo schermo).

Un elemento `fixed` non scorrerà mai con la pagina; rimarrà incollato alla sua coordinata visiva anche quando l'utente effettua lo scroll verso il basso (ideale per menù di navigazione persistenti o banner laterali).

4.2.4 L'Indice Z (Z-Index)

Quando si estraggono gli elementi dal flusso normale (con `absolute`, `relative` o `fixed`), questi inevitabilmente finiranno per sovrapporsi l'uno all'altro. Per pilotare l'ordine di profondità, ovvero quale elemento stia “sopra” e quale “sotto”, si usa `z-index`.

- **Attivazione:** Lo `z-index` funziona **esclusivamente** su elementi che hanno la proprietà `position` esplicitamente impostata.
- **Valore:** Più il valore numerico è alto, più l'elemento sarà spinto in primo piano verso l'osservatore.
- **Valori Negativi:** Assegnando `z-index: -1`; , è possibile spingere un elemento posizionato dietro al normale testo o background del flusso principale.

Il Trabocchetto dell'Annidamento:

L'ordine di `z-index` è sempre vincolato al suo contenitore padre. Se la `<figure>` A ha `z-index: 1` e la `<figure>` B ha `z-index: 5`, la figura B coprirà sempre la figura A. Anche se un testo annidato dentro la figura A avesse `z-index: 9999`, non potrebbe mai superare la figura B, perché intrappolato nel livello di profondità del suo genitore.

4.3 Elementi Fluttuanti (Floats)

Storicamente, prima dell'avvento di moduli di layout dedicati, la proprietà `float` era l'unico modo per posizionare elementi (come intere colonne di testo o sidebar) uno di fianco all'altro.

La proprietà `float` estrae parzialmente l'elemento dal flusso normale, spingendolo il più possibile verso il margine sinistro (`float: left`;) o destro (`float: right`;) del suo contenitore. Il testo e gli elementi inline successivi “fluiscono” attorno all'elemento flottato.

Regola d'oro per l'esame: Un elemento a blocco a cui viene applicato un `float` deve rigorosamente avere una **larghezza (`width`) specificata**. Se non la si imposta, la larghezza diventerà `auto` (occupando implicitamente tutto lo spazio disponibile) e annullerà l'effetto di affiancamento, non lasciando spazio fisico per far fluire gli altri contenuti.

4.3.1 La Proprietà Clear

Cosa succede se vogliamo che un elemento (come un footer) *non* fluisca attorno a un elemento flottato, ma si posizioni forzatamente sotto di esso? Si utilizza la proprietà `clear`.

- `clear: left`;: Impedisce all'elemento di affiancarsi a blocchi flottati a sinistra.
- `clear: right`;: Impedisce l'affiancamento a destra.
- `clear: both`;: Forza l'elemento a scivolare sotto qualsiasi blocco flottato (sia a destra che a sinistra) che lo precede.

4.3.2 Il Problema del Contenitore (Containing Floats)

Questo è uno dei bug concettuali più comuni negli esercizi di layout. Se un genitore (es. un `<article>`) contiene *esclusivamente* elementi figli flottati, il genitore **collassa su se stesso**, come se fosse vuoto, e la sua altezza diventa 0px. Questo distrugge gli sfondi e i bordi del contenitore.

La Soluzione: Per forzare il contenitore a riconoscere l'ingombro dei suoi figli flottati ed espandersi per racchiuderli, la tecnica standard (e più semplice) è applicare la proprietà `overflow: auto;` al contenitore genitore.

4.4 Nascondere gli Elementi: Display vs Visibility

Gestire interfacce complesse richiede spesso di nascondere temporaneamente degli elementi (es. menù a tendina, popup, tooltip). Esistono due approcci diametralmente opposti:

1. **display: none;** Rimuove l'elemento *completamente* dal flusso del documento. Il browser si comporta come se l'elemento non esistesse: lo spazio fisico che occupava scompare e viene riempito dagli altri nodi.
2. **visibility: hidden;** Rende l'elemento totalmente trasparente (invisibile), ma **preserva intatto il suo ingombro fisico** (il suo Box Model) all'interno del flusso normale. Lo spazio rimane vuoto.

Una tecnica classica per creare *thumbnail* interattive (come gallerie di immagini) combina questi concetti: si posiziona in modo assoluto una versione ingrandita dell'immagine (inizialmente nascosta con `visibility: hidden;`) e la si fa apparire cambiando il valore in `visibility: visible;` innescando l'azione tramite la pseudo-classe `:hover` sul contenitore genitore.

4.5 Layout Moderni: Introduzione a Flexbox

Creare griglie e colonne usando i `float` o i posizionamenti assoluti genera codice fragile e calcoli matematici complessi per i margini. Per risolvere questo limite storico, CSS3 ha introdotto **Flexbox** (Flexible Box Layout), un sistema nativo progettato per distribuire lo spazio e allineare gli elementi lungo un singolo asse (una riga o una colonna).

L'architettura di Flexbox si basa su una relazione Padre-Figlio: si definisce un **Flex Container** (il genitore) che manipola il comportamento dei suoi **Flex Items** (i figli diretti).

4.5.1 Proprietà del Flex Container (Il Genitore)

Per attivare il motore Flexbox, basta una singola riga applicata al contenitore:

```
1 .contenitore {  
2   display: flex;  
3 }
```

Di default, questo trasformerà tutti i figli diretti (anche se blocchi come i `<div>`) in colonne disposte su una singola riga orizzontale.

Le direttive principali applicabili al genitore sono:

- **flex-direction:** Definisce l'asse principale di scorrimento.
 - `row` (default): Da sinistra a destra.
 - `column`: Dall'alto verso il basso.
- **flex-wrap:** Decide cosa succede se i figli sono troppi per lo spazio disponibile.
 - `nowrap` (default): I figli si stringono all'inverosimile per stare in una riga.
 - `wrap`: I figli vanno a capo su una nuova riga/colonna quando lo spazio finisce.

4.5.2 Allineamento e Distribuzione dello Spazio

Flexbox elimina la necessità di calcolare margini per centrare gli elementi, grazie a due potentissime proprietà del contenitore:

1. **justify-content** (Allineamento sull'Asse Principale, es. orizzontale in una **row**):
 - **flex-start** / **flex-end**: Ammassa i figli all'inizio o alla fine.
 - **center**: Centra i figli perfettamente.
 - **space-between**: Incolla il primo figlio all'inizio, l'ultimo alla fine, e distribuisce lo spazio vuoto in modo equo tra gli elementi centrali.
 - **space-around**: Applica uno spazio vuoto identico attorno a ogni singolo elemento.
2. **align-items** (Allineamento sull'Asse Trasversale, es. verticale in una **row**):
 - **stretch** (default): I figli si allungano per riempire tutta l'altezza del contenitore.
 - **center**: Centra gli elementi verticalmente (la soluzione definitiva per la centratura verticale nel web design!).

4.5.3 Proprietà dei Flex Items (I Figli)

È possibile controllare individualmente come ogni singolo figlio cresce o si restringe rispetto agli altri:

- **flex-basis**: Definisce la dimensione ideale di base del figlio prima che il layout distribuisca lo spazio residuo (funge da **width** o **height** ottimale).
- **flex-grow**: Un valore numerico (es. 1, 2) che indica la proporzione con cui l'elemento deve allargarsi per assorbire lo spazio vuoto extra. (Se l'Item A ha **flex-grow**: 2 e l'Item B ha 1, A prenderà il doppio dello spazio extra rispetto a B).
- **flex-shrink**: La velocità con cui l'elemento si restringe quando manca spazio fisico.

```
1 /*Esempio Pratico: Sintassi Shorthand per i figli */
2 .card-figlio {
3     /* flex-grow | flex-shrink | flex-basis*/
4     flex: 1 1 auto;
5 }
```

4.6 CSS Grid Layout (Griglie Bidimensionali)

Se Flexbox è stato progettato per gestire layout monodimensionali (una singola riga o una singola colonna alla volta), **CSS Grid** è il sistema definitivo per i layout bidimensionali, permettendo di governare simultaneamente sia le righe che le colonne.

Come per Flexbox, si attiva dichiarando un contenitore padre:

```
1 .griglia {
2     display: grid;
3 }
```

4.6.1 Definire la Struttura (Colonne e Righe)

Una volta attivata la griglia, devi specificare la sua architettura usando **grid-template-columns** e **grid-template-rows**.

L'Unità Frazionaria (fr):

CSS Grid introduce l'unità `fr` (fraction). Rappresenta una frazione dello spazio disponibile nel contenitore. Se dichiari `grid-template-columns: 1fr 2fr 1fr;`, crei tre colonne: la centrale sarà grande il doppio rispetto alle due laterali, adattandosi dinamicamente allo schermo.

```
1 /*Esempio Pratico: Creazione della Griglia */
2 .contenitore-griglia {
3   display: grid;
4   /* Crea 3 colonne uguali usando la funzione repeat() per brevità */
5   grid-template-columns: repeat(3, 1fr);
6   /* Spaziatura vuota tra le celle (sostituisce i margini sui figli)*/
7   gap: 15px;
8 }
```

Funzioni Avanzate per l'Esame: Spesso si usa la combinazione `repeat(auto-fit, minmax(200px, 1fr))`. Questa potente direttiva dice al browser: "Crea quante più colonne possibili larghe minimo 200px. Se avanza spazio, distribiscilo in modo equo (1fr)." È la base per le gallerie di immagini responsive senza usare le Media Queries!

4.6.2 Posizionamento Esplicito delle Celle

Di default, i figli si dispongono automaticamente uno in ogni cella disponibile (Implicit Grid). Puoi però forzare un elemento a occupare posizioni specifiche o espandersi su più righe/colonne usando le "linee" invisibili che separano le celle.

```
1 .elemento-speciale {
2   /*Parte dalla linea verticale 1 e si estende fino alla linea 3 (occupa 2 colonne) */
3   grid-column: 1 / 3;
4   /* Occupa la seconda riga*/
5   grid-row: 2;
6 }
```

Il Matrimonio Perfetto: Grid + Flexbox

La best-practice moderna nei progetti web è usarli in sinergia: si usa **CSS Grid** sul contenitore `<body>` o `<main>` per definire la macro-architettura della pagina (header, sidebar, contenuto, footer), e si usa **Flexbox** all'interno delle singole celle (es. dentro una card o nella navbar) per allineare al millimetro icone, testi e bottoni.

4.7 Approcci al Layout e Responsive Design

Fino agli anni 2000, i siti web venivano disegnati pensando a un'unica dimensione di schermo (solitamente i monitor dei PC). Con l'avvento degli smartphone, il design ha dovuto adattarsi. Esistono storicamente tre approcci concettuali:

1. **Layout Fisso (Fixed):** Le larghezze dei contenitori sono scritte in pixel rigidi (es. `width: 960px;`). Non si adatta agli schermi piccoli e obbliga lo scroll orizzontale. Oggi è considerato obsoleto e penalizzato dalla SEO.
2. **Layout Liquido (Fluid):** Tutte le larghezze sono espresse in percentuali (`width: 80%;`). Si adatta, ma su schermi giganti i testi diventano illeggibili per l'eccessiva lunghezza delle righe.
3. **Responsive Design:** L'approccio moderno. La pagina "risponde" alle modifiche dello schermo non solo stringendosi, ma **riorganizzando attivamente i propri elementi** (es. da 3 colonne su PC si passa a 1 singola colonna su smartphone, e il menù orizzontale si trasforma in un menù ad hamburger).

4.7.1 I Pilastrini del Responsive Design

Per superare l'esame e costruire un progetto valutabile, un layout responsive richiede quattro componenti obbligatorie:

4.7.1.1 1. Il Meta Tag Viewport

Da inserire rigorosamente nell'<head> dell'HTML. Senza questo tag, i dispositivi mobili “mentiranno” sulla loro risoluzione (fingendo di essere uno schermo desktop e rimpicciolendo tutto). Questo tag forza il browser mobile a usare i suoi veri pixel fisici.

```
1 <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

4.7.1.2 2. Immagini Fluide

Le immagini rischiano di “sfondare” i contenitori se sono più larghe dello schermo. Per risolvere, si applica globalmente questa regola CSS:

```
1 img {
2     max-width: 100%;
3     height: auto; /*Mantiene le proporzioni originali senza schiacciarle*/
4 }
```

4.7.1.3 3. Le Media Queries

Il cuore pulsante del Responsive Design. Le Media Queries permettono di creare blocchi di CSS “condizionali” che il browser legge ed esegue **solo** se le dimensioni dello schermo rispettano certi parametri.

Sintassi delle Media Queries:

Prestare estrema attenzione a non dimenticare le parentesi graffe esterne che racchiudono le regole.

```
1 /*Regole base per dispositivi mobili (Mobile First) */
2 body { font-size: 14px; }
3 .sidebar { display: none; } /* Nascondo la sidebar su smartphone*/
4
5 /*SE lo schermo è largo almeno 768px (Tablet in su), applica QUESTE regole: */
6 @media only screen and (min-width: 768px) {
7     body {
8         font-size: 16px; /* Ingrandisco il testo */
9     }
10    .sidebar {
11        display: block; /* Faccio riapparire la sidebar*/
12        float: left;
13        width: 250px;
14    }
15 }
```

4.8 Filtri, Transizioni e Animazioni

Il CSS moderno permette manipolazioni grafiche che un tempo richiedevano l'uso di Photoshop o pesanti script JavaScript.

4.8.1 Filtri (Filters)

La proprietà `filter` applica effetti grafici direttamente sugli elementi HTML (particolarmente utile sulle immagini).

```
1 .foto-vintage {
2     /*Rende l'immagine in bianco e nero e la sfoca leggermente*/
3     filter: grayscale(100%) blur(2px);
4 }
```

4.8.2 Transizioni (Transitions)

Le transizioni permettono di animare un cambiamento di stato (es. il passaggio del mouse :[hover](#)) in modo fluido e morbido nel tempo, anziché in modo istantaneo e “scattoso”.

Le 4 proprietà fondamentali (spesso combinate nella notazione shorthand [transition](#)):

- [transition-property](#): Quale regola CSS animare (es. [background-color](#), [transform](#), [opacity](#), o [all](#)).
- [transition-duration](#): Quanto dura l’animazione in secondi (es. [0.5s](#)).
- [transition-timing-function](#): La curva di accelerazione (es. [linear](#), [ease-in](#), [ease-out](#)).
- [transition-delay](#): Eventuale ritardo prima che parta l’animazione.

```
1 /*Esempio Pratico: Transizione Fluida su Hover */
2 .bottone {
3     background-color: blue;
4     /* Dico al bottone di animare qualsiasi cambio di stile in mezzo secondo*/
5     transition: background-color 0.5s ease-out;
6 }
7
8 .bottone:hover {
9     /*Il passaggio da blu a rosso non sarà istantaneo, ci metterà 0.5s*/
10    background-color: red;
11 }
```

5 Fondamenti del Web Dev(06/7)

5.1 Introduzione a JavaScript

JavaScript nasce come linguaggio di scripting lato client, orientato agli oggetti e a tipizzazione dinamica. A differenza di linguaggi tradizionali come Java o C++, in JavaScript anche le funzioni sono considerate veri e propri oggetti. Essendo debolmente tipizzato, le variabili possono essere convertite implicitamente da un tipo di dato all'altro con grande facilità.

Client-Side Scripting: Approccio in cui il codice viene scaricato ed eseguito localmente dal browser dell'utente, anziché demandare l'elaborazione a un server remoto.

L'esecuzione in locale riduce notevolmente il carico sui server e permette al browser di rispondere in modo estremamente rapido agli eventi scatenati dall'utente, garantendo un'esperienza fluida e interattiva, molto vicina a quella dei classici software desktop.

5.1.1 Svantaggi dell'Approccio Client-Side

Nonostante i vantaggi, l'esecuzione lato client porta con sé alcune criticità:

- Non vi è garanzia che l'utente abbia JavaScript abilitato (è necessario prevedere meccanismi di fallback).
- Applicazioni molto complesse possono diventare ostiche da mantenere e da debuggare.
- JavaScript non è *fault tolerant*: se il browser incontra codice HTML o CSS non valido tenta comunque di interpretarlo, ma un singolo errore di sintassi o a runtime in JavaScript interrompe immediatamente l'esecuzione dello script.
- Essendo un linguaggio in costante espansione (attraverso le nuove edizioni ECMAScript), le funzionalità più moderne potrebbero non godere di retrocompatibilità su browser obsoleti.

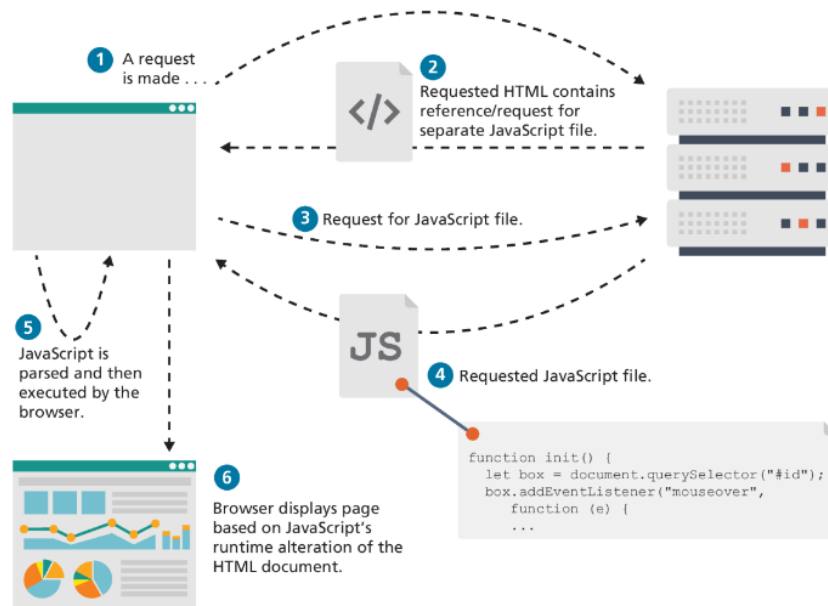


Figura 5.1: Schema visivo del ciclo di richiesta ed esecuzione JavaScript lato client

5.1.2 Inserimento del Codice in HTML

Così come il CSS, anche il codice JavaScript può essere integrato in una pagina in diverse modalità:

1. **Inline:** Direttamente all'interno degli attributi degli elementi HTML.
2. **Embedded:** All'interno di un apposito tag `<script>`.
3. **Esterno:** Tramite un file separato (generalmente `.js`) richiamato tramite l'attributo `src`. Questa è la pratica raccomandata in quanto mantiene separata la struttura dalla logica.

```
1 <head>
2   <script>
3     console.log("Script interno eseguito");
4   </script>
5
6   <script src="app.js"></script>
7 </head>
8 <body>
9   <button onClick="alert('Azione inline')">Clicca qui</button>
10
11   <noscript>
12     Per usufruire di tutte le funzionalità, abilita JavaScript nel tuo browser.
13   </noscript>
14 </body>
```

5.2 Variabili e Tipi di Dato

JavaScript suddivide i dati in due grandi famiglie: i **tipi primitivi** (dati semplici) e i **tipi di riferimento** (oggetti, array, funzioni).

5.2.1 Tipi Primitivi

I valori primitivi rappresentano dati basilari. I principali includono:

- **Boolean:** Esprime un valore di verità, **true** o **false**.
- **Number:** Rappresenta valori numerici, sia interi che decimali (salvati in formato floating point a 64-bit).
- **String:** Una sequenza testuale racchiusa tra apici singoli, doppi o backtick.
- **Null:** Rappresenta intenzionalmente l'assenza di un oggetto o di un valore.
- **Undefined:** Il valore di default assunto da variabili dichiarate ma non ancora inizializzate.
- **Symbol:** Un valore unico e immutabile (introdotto in ES6), spesso utilizzato come identificatore di proprietà per gli oggetti.

Comportamento Truthy e Falsy: In JavaScript, qualsiasi espressione valutata in un contesto booleano (come la condizione di un **if**) rivela un valore intrinseco. Sono considerati *false* esclusivamente: **false**, **null**, **undefined**, **0**, **NaN** e le stringhe vuote **""**. Tutti gli altri valori sono *truthy*.

5.2.2 Memoria: Primitivi vs Riferimento

Esiste una netta differenza architetturale nel modo in cui vengono memorizzati questi due gruppi di dati.

Le variabili che ospitano **tipi primitivi** contengono il valore effettivo direttamente all'interno della cella di memoria ad esse assegnata. Al contrario, le variabili che ospitano **tipi di riferimento** (come array o oggetti) non contengono i dati, ma un puntatore (o riferimento) all'area di memoria complessa in cui risiedono le informazioni reali.

```

1 let abc = 27;           | variables with primitive types
2 let def = "hello";     |
3
4 let foo = [45, 35, 25]; | variable with reference type
5                         | (i.e., array object)
6
7 let xyz = def;          | these new variables differ in important ways
8 let bar = foo;          | (see below)
9
10 bar[0] = 200;          | changes value of the first element of array

```

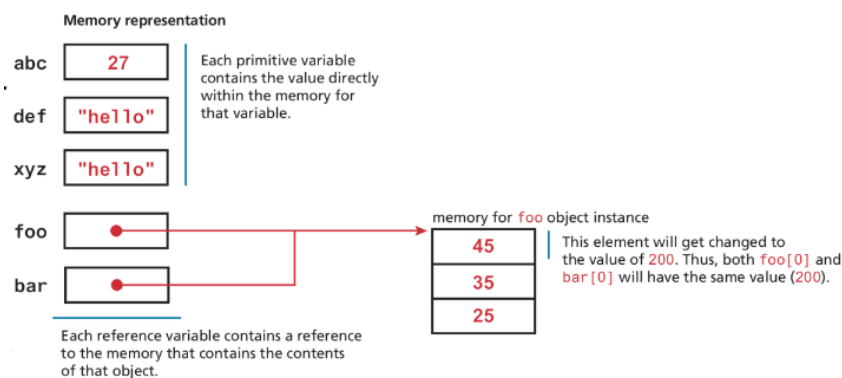


Figura 5.2: Schema della rappresentazione in memoria dei tipi primitivi vs reference

5.2.3 Dichiarazione delle Variabili: let vs const

In JavaScript moderno si abbandona l'uso di **var** in favore di due parole chiave più precise: **let** e **const**.

Attenzione a const con i tipi di riferimento:

La keyword **const** impedisce la *riassegnazione* della variabile a un altro valore o locazione di memoria. Tuttavia, se la variabile punta a un oggetto o a un array, le proprietà o gli elementi interni rimangono pienamente mutabili, poiché il puntatore in memoria non cambia.

```
1 // Dichiarazione e assegnazione di tipi primitivi
2 let punteggio = 10;
3 punteggio = 20; // Operazione consentita
4
5 const saluto = "Ciao";
6 // saluto = "Addio"; // ERRORE a runtime: riassegnazione negata
7
8 // Comportamento di const con tipi di riferimento
9 const arrayNumeri = [45, 35, 25];
10 arrayNumeri[0] = 99; // CONSENTITO: l'array diventa [99, 35, 25]
11
12 const utente = { nome: "Randy" };
13 utente.nome = "Ricardo"; // CONSENTITO: mutazione interna all'oggetto
```

5.2.4 Interazione Base e Output

Esistono vari metodi nativi per stampare messaggi e gestire input basilari:

- `console.log()`: Stampa messaggi nella console per sviluppatori. Indispensabile per interrogare variabili interattivamente.
- `alert(messaggio)`: Blocca l'esecuzione e mostra un pop-up modale con un messaggio.
- `prompt(messaggio)`: Richiede un input testuale all'utente tramite una finestra modale.
- `confirm(messaggio)`: Pone una domanda modale all'utente restituendo un valore booleano (Ok = **true**, Annulla = **false**).
- `document.write()`: Inietta output direttamente nel flusso del documento HTML (pratica ormai fortemente sconsigliata e soppiantata dall'uso delle API del DOM).

5.3 Strutture di Controllo e Flusso di Esecuzione

JavaScript condivide gran parte della sintassi per le strutture di controllo con linguaggi come C++ e Java, ma introduce peculiarità legate alla sua tipizzazione dinamica.

5.3.1 Condizionali e Operatore Ternario

Oltre ai classici costrutti **if...else if...else** e **switch**, JavaScript fa largo uso dell'operatore condizionale (o ternario) per assegnazioni rapide e compatte basate sulla valutazione di un'espressione.

```
1 // Sintassi Operatore ternario: (condizione) ? valore_se_vero : valore_se_falso;
2 let eta = 20;
3 let stato = (eta >= 18) ? "Maggiorenne" : "Minorenne";
```

Valori Truthy e Falsy: Essendo debolmente tipizzato, JavaScript forza la conversione di qualsiasi valore in un booleano quando viene valutato in un contesto condizionale (es. dentro un **if**). Sono considerati **Falsy** solo 6 valori specifici: **false**, **null**, **undefined**, **0**, **NaN** e la stringa vuota **""**. Qualsiasi altro valore (inclusi array, stringhe non vuote e oggetti vuoti) è intrinsecamente valutato come **Truthy**.

5.3.2 Iterazioni: Cicli e for...of

I cicli standard (**while**, **do...while**, **for**) mantengono la sintassi classica derivata dal C. Con l'introduzione di ES6, è stato aggiunto il ciclo **for...of**, particolarmente utile ed elegante per iterare direttamente sui valori di strutture dati iterabili come gli Array, senza dover gestire manualmente gli indici.

```
1 const anni = [1855, 1648, 1420];
2
3 // Iterazione classica basata sull'indice
4 for (let i = 0; i < anni.length; i++) {
5   console.log(anni[i]);
6 }
7
8 // Iterazione moderna basata sugli elementi con for...of
9 for (let anno of anni) {
10  console.log(anno);
11 }
```

5.3.3 Gestione degli Errori (try...catch)

Blocco dell'esecuzione: In JavaScript, quando l'interprete del browser incontra un errore a runtime (eccezione), interrompe bruscamente l'esecuzione sequenziale dello script.

Per prevenire il “crash” dell'applicazione, si utilizza il costrutto **try...catch**, che permette di circoscrivere il codice “rischioso” e “catturare” eventuali eccezioni, gestendole in modo controllato.

```
1 try {
2   // Tenta di eseguire una funzione non definita
3   funzioneInesistente("test");
4 } catch(err) {
5   // Se fallisce, cattura l'errore senza fermare l'intero programma
6   console.log("Eccezione intercettata: " + err);
7 }
```

5.4 Strutture Dati: Array, Oggetti e JSON

In JavaScript, Array e Oggetti rappresentano l'ossatura della gestione dei dati. Grazie alla flessibilità del linguaggio, possono contenere contemporaneamente tipi di dato eterogenei (numeri incrociati con stringhe, booleani e persino funzioni).

5.4.1 Array e Destrutturazione

Gli array sono liste ordinate, create tipicamente attraverso la **notazione letterale** con parentesi quadre `[]`, preferita rispetto al costruttore `new Array()`.

Array Destructuring: Una sintassi concisa (ES6) che permette di spaccettare rapidamente i valori posizionali da un array e assegnarli a variabili distinte in una singola istruzione.

```
1 const podio = ["Ferrari", "Mercedes", "Red Bull"];
2
3 // Assegnazione tradizionale manuale
4 let primo = podio[0];
5 let secondo = podio[1];
6
7 // Destrutturazione simultanea (ES6)
8 let [primoClassificato, secondoClassificato] = podio;
```

5.4.2 Oggetti (Object Literal)

A differenza dei linguaggi *class-based* classici, in JavaScript gli oggetti possono essere definiti “al volo” come semplici collezioni di coppie chiave-valore (chiamate *proprietà*). Il metodo di creazione nativo e raccomandato è la **notazione letterale** con parentesi graffe `{}`. Per accedere alle proprietà si può usare la *dot notation* (`obj.chiave`) o la *bracket notation* (`obj["chiave"]`).

```
1 const utente = {  
2   id: 1,  
3   nome: "Mario",  
4   indirizzo: {  
5     citta: "Roma",  
6     cap: "00100"  
7   }  
8 };
```

Object Destructuring e Operatore Spread (...)

Proprio come gli array, anche gli oggetti supportano l'estrazione rapida tramite *destructuring*. Inoltre, l'operatore **Spread** `...` è utilissimo per “spalmare” (copiare) le proprietà di un oggetto all'interno di un nuovo oggetto, effettuando una *shallow copy* (copia superficiale).

```
1 // Destrutturazione di un oggetto, anche su livelli annidati  
2 let { id, nome, indirizzo: { citta } } = utente;  
3  
4 // Spread Operator: espande le proprietà di 'utente' e ne aggiunge di nuove  
5 const utenteAggiornato = { ...utente, ruolo: "Admin" };
```

5.4.3 JSON (JavaScript Object Notation)

Il JSON è il formato standard *de facto* per l'interscambio di dati sul web (es. comunicazioni con Web API). Sebbene derivi visivamente dalla notazione letterale degli oggetti JS, possiede regole sintattiche rigorose.

Caratteristiche del JSON:

1. I nomi delle proprietà (le chiavi) **devono** essere necessariamente racchiusi tra doppi apici `"`.
2. Un JSON è testuale (una stringa). Perché il browser possa utilizzarlo come vero e proprio oggetto in memoria, il testo deve essere parsato, utilizzando tipicamente l'oggetto built-in del browser (es. `JSON.parse(stringa)`).

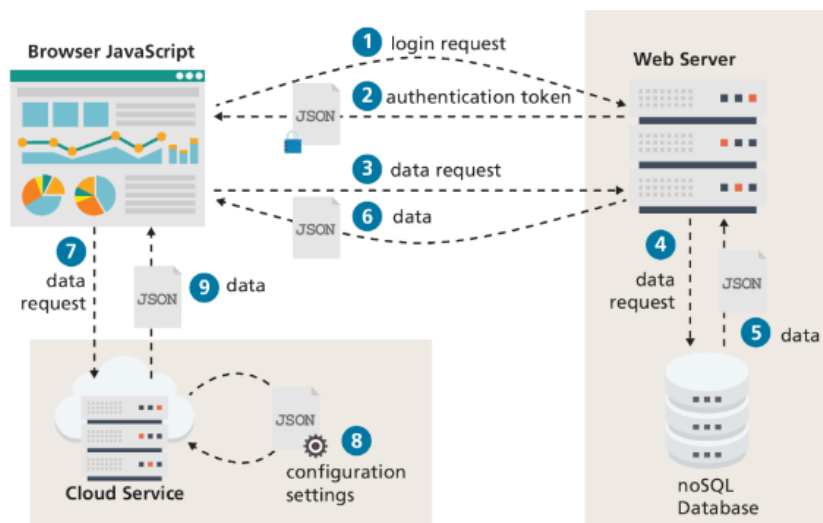


Figura 5.3: Schema visivo dell'impiego del JSON nello sviluppo web contemporaneo

5.5 Le Funzioni in JavaScript

Essendo JavaScript un linguaggio funzionale, le funzioni sono “cittadini di prima classe” (veri e propri oggetti) e costituiscono uno dei pilastri fondamentali dello sviluppo web. Non richiedono la specifica di un tipo di ritorno, né la tipizzazione dei parametri.

Esistono due modi principali per definire una funzione:

1. **Function Declaration (Dichiarazione):** La sintassi standard con un nome esplicito.
2. **Function Expression (Espressione):** La funzione viene assegnata a una variabile. Spesso si utilizzano funzioni anonime (senza nome).

```

1 // Function Declaration
2 function calcolaTotale(prezzo, quantita) {
3   return prezzo * quantita;
4 }
5
6 // Function Expression (Funzione anonima)
7 const calcolaSubtotale = function(prezzo, quantita) {
8   return prezzo * quantita;
9 };

```

5.5.1 Parametri: Default e Rest

JavaScript offre costrutti moderni (ES6) per gestire argomenti mancanti o liste di argomenti di lunghezza variabile:

- **Parametri di Default:** Permettono di assegnare un valore base se la funzione viene chiamata senza quell'argomento (evitando di ottenere NaN in operazioni matematiche).
- **Rest Operator (...):** Raggruppa un numero indefinito di argomenti in un array reale all'interno della funzione.

```

1 // Parametri di Default
2 function saluta(nome = "Ospite", saluto = "Ciao") {
3   console.log(saluto + " " + nome);
4 }
5 saluta(); // Output: "Ciao Ospite"
6
7 // Rest Parameters
8 function unisciStringhe(...stringhe) {

```

```
9 // 'stringhe' è un vero array contenente tutti i parametri passati
10 let risultato = "";
11 for (let str of stringhe) risultato += str + " ";
12 return risultato;
13 }
```

5.5.2 L'Hoisting: Il sollevamento delle variabili

Attenzione all'esame: L'interprete JavaScript, prima di eseguire il codice riga per riga, effettua una prima passata in cui "solleva" (hoisting) le dichiarazioni all'inizio del loro scope corrente.

- **Function Declarations:** Vengono sollevate **completamente** (firma e corpo). Puoi invocare la funzione in righe di codice precedenti alla sua effettiva dichiarazione.
- **Variabili (`let`, `var`, `const`):** Viene sollevata solo la *dichiarazione*, **non l'assegnazione**.
- **Function Expressions:** Poiché sono trattate come assegnazioni di variabili, se provi a invocarle prima della riga in cui sono definite, otterrai un `ReferenceError`.

5.5.3 Funzioni di Callback

Poiché le funzioni sono oggetti, possono essere passate come argomenti ad altre funzioni. Una funzione passata come parametro per essere eseguita in un secondo momento viene definita **Callback**.

```
1 const calcolaTasse = function(subtotale) {
2   return subtotale * 0.05;
3 };
4
5 function processaOrdine(prezzo, quantita, callbackTasse) {
6   let base = prezzo * quantita;
7   // Invoco la funzione passata come parametro
8   return base + callbackTasse(base);
9 }
10
11 // Passo il "riferimento" alla funzione, senza le parentesi ()
12 let totale = processaOrdine(50, 2, calcolaTasse);
```

5.6 Oggetti, Costruttori e contesto ('this')

Nei linguaggi orientati agli oggetti tradizionali le classi definiscono i comportamenti (metodi). In JavaScript, gli oggetti possono possedere proprietà che contengono funzioni.

All'interno di queste funzioni, la parola chiave **this** fa riferimento al **contesto di esecuzione**, ovvero l'oggetto stesso che sta invocando il metodo.

5.6.1 I Function Constructors

Prima dell'introduzione delle classi in ES6, il modo standard per creare "modelli" di oggetti multipli (istanze) era l'uso dei costruttori.

Function Constructor: Una normale funzione (per convenzione con l'iniziale Maiuscola) che viene invocata preceduta dalla keyword **new**.

Cosa succede "dietro le quinte" quando si usa **new**:

1. Viene creato un nuovo oggetto vuoto.
2. Il contesto del **this** viene associato a questo nuovo oggetto vuoto.
3. La funzione viene eseguita (assegnando le proprietà al **this**).
4. L'oggetto viene ritornato automaticamente.

```
1 function Cliente(nome, citta) {  
2   this.nome = nome;  
3   this.citta = citta;  
4   this.stampa = function() {  
5     return this.nome + " da " + this.citta;  
6   };  
7 }  
8  
9 const c1 = new Cliente("Mario", "Roma"); // Istanziamento
```

5.6.2 Arrow Syntax (Funzioni a Freccia)

Le *Arrow Functions* introducono una sintassi estremamente concisa per scrivere funzioni anonime.

Differenza Cruciale per il 'this': A differenza delle funzioni tradizionali, le funzioni a freccia **non hanno un proprio this**. Ereditano il contesto **this** dallo *scope lessicale* in cui sono state definite (ovvero dal loro genitore). Questo risolve molti problemi storici di perdita del contesto nei callback.

```
1 // Sintassi base  
2 const raddoppia = (x) => { return x * 2; };  
3  
4 // Ritorno implicito (su una riga, niente parentesi graffe o return)  
5 const triplica = x => x * 3;  
6  
7 // Per ritornare un oggetto letterale (implicitamente), serve avvolgerlo tra parentesi tonde  
8 const creaUtente = (id, nome) => ({ id: id, utente: nome });
```

5.7 Scope e Closures (Concetti Avanzati)

Lo **Scope** definisce la visibilità e il ciclo di vita di variabili e funzioni.

1. **Global Scope:** Variabili accessibili ovunque. Da limitare per evitare conflitti di nomi (namespace pollution).
2. **Function (Local) Scope:** Variabili definite dentro una funzione. Invisibili all'esterno.
3. **Block Scope:** Il blocco definito dalle parentesi graffe {} (es. **if**, **for**).

Differenza var vs let/const: Variabili dichiarate con **let** o **const** rispettano il Block Scope e muoiono alla fine del blocco. Le variabili dichiarate con **var** ignorano il blocco e "fuoriescono", diventando visibili nello scope superiore (locale o globale), generando spesso bug insidiosi nei cicli.

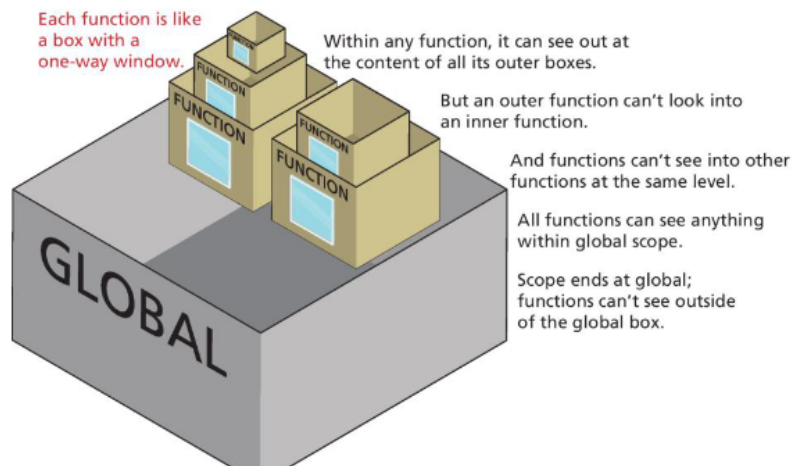


Figura 5.4: Schema visivo dei box “one-way window” dello Scope

5.7.1 Le Closures (Chiusure)

Questo è uno dei concetti più richiesti e complessi di JavaScript.

In condizioni normali, quando una funzione termina l'esecuzione, le sue variabili locali vengono distrutte (Garbage Collected). Tuttavia, se una funzione genitore *ritorna* al suo interno una funzione figlia, succede qualcosa di speciale.

Closure: È uno speciale oggetto logico che lega indissolubilmente una funzione al suo ambiente lessicale (il design-time scope). Permette a una funzione di continuare ad accedere alle variabili del suo scope genitore, **anche dopo** che il genitore ha terminato l'esecuzione.

```

1  function genitore() {
2      let segreto = "Password123"; // Variabile locale
3
4      function figlia() {
5          // La funzione interna accede alla variabile esterna
6          console.log("Il segreto è: " + segreto);
7      }
8
9      return figlia; // Ritorno il RIFERIMENTO alla funzione figlia
10 }
11
12 // Invocando genitore(), eseguo il codice e ottengo la funzione figlia.
13 // Normalmente, 'segreto' dovrebbe essere distrutta a questo punto.
14 const rivelaSegreto = genitore();
15
16 // Eppure, funziona ancora! La Closure ha preservato l'ambiente.
17 rivelaSegreto(); // Output: "Il segreto è: Password123"

```

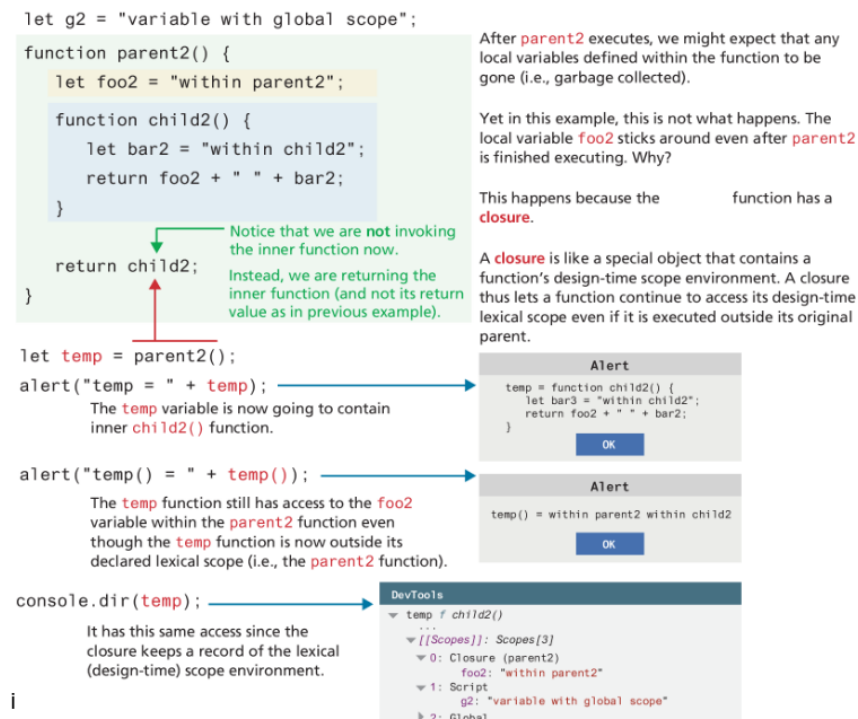


Figura 5.5: Struttura di una Closure visualizzata nel pannello DevTools del browser

5.8 Il Document Object Model (DOM)

Il **Document Object Model (DOM)** è un'interfaccia di programmazione che rappresenta un documento HTML o XML come una struttura ad albero. Ogni elemento, attributo e frammento di testo nel codice HTML diventa un **nodo** nell'albero del DOM, consentendo a JavaScript di accedere, modificare, creare o eliminare contenuti e stili dinamicamente.

Nodo (Node): L'unità base dell'albero del DOM. Esistono diversi tipi di nodi, i più comuni sono i *Element Nodes* (tag HTML), *Attribute Nodes* (es. `href`, `src`) e *Text Nodes* (il testo contenuto tra i tag).

La struttura gerarchica del DOM segue una logica di parentela: un nodo può avere un **Parent** (genitore), dei **Children** (figli) e dei **Siblings** (fratelli, ovvero nodi che condividono lo stesso genitore). Al vertice dell'albero risiede il nodo **Document**, che funge da radice (root).

5.8.1 Proprietà Fondamentali dei Nodi

Ogni oggetto nodo nel DOM possiede delle proprietà standard che permettono di identificarne la natura:

- **nodeType**: Un intero che indica il tipo di nodo (es. 1 per gli elementi, 3 per il testo).
- **nodeName**: Il nome del nodo (es. "DIV" o "P" per gli elementi).
- **nodeValue**: Il valore testuale contenuto (spesso **null** per i nodi elemento).

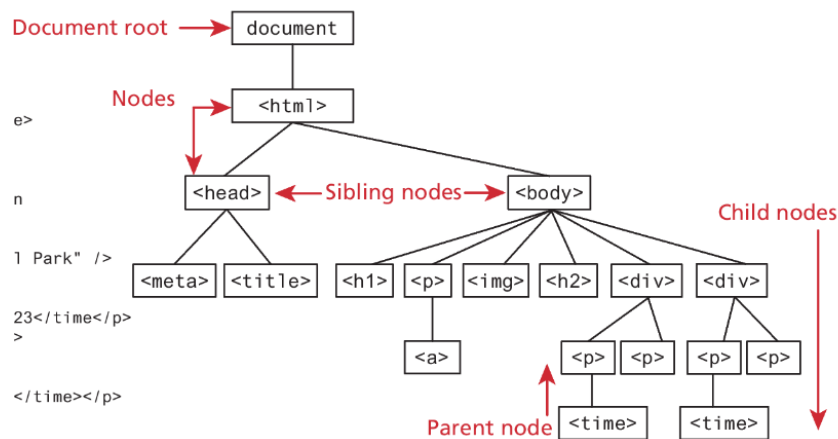


Figura 5.6: Schema visivo dell'albero del DOM con gerarchia parent/child/sibling

5.9 Accesso agli Elementi del DOM

Per manipolare la pagina, JavaScript deve prima “trovare” gli elementi corretti. Esistono metodi “tradizionali” e selettori moderni più flessibili.

5.9.1 Metodi di Selezione Tradizionali

Questi metodi sono altamente efficienti ma limitati a criteri di ricerca specifici:

- `document.getElementById("id")`: Restituisce il singolo elemento corrispondente all'ID univoco.
- `document.getElementsByTagName("tag")`: Restituisce una collezione (simile a un array) di tutti gli elementi di quel tipo.
- `document.getElementsByClassName("classe")`: Restituisce una collezione di elementi che possiedono la classe specificata.

5.9.2 Selettori Moderni (Query Selectors)

Introdotti per rispecchiare la flessibilità dei selettori CSS, sono oggi lo standard per la selezione degli elementi:

querySelector vs querySelectorAll

- `querySelector("#id .classe")`: Restituisce solo il **primo** elemento che soddisfa il selettore CSS fornito.
- `querySelectorAll(".classe")`: Restituisce una `NodeList` (una collezione statica) di **tutti** gli elementi che soddisfano il criterio.

```

1 // Selezione mirata con querySelector
2 const primoLink = document.querySelector("nav a.active");
3
4 // Selezione multipla
5 const tuttiIDiv = document.querySelectorAll("div.content");
6
7 // Iterazione su una NodeList
8 tuttiIDiv.forEach(div => {
9   console.log(div.textContent);
10 });

```

5.10 Manipolazione del Contenuto e degli Attributi

Una volta selezionato un elemento, è possibile modificarne il contenuto testuale, la struttura interna o gli attributi HTML.

5.10.1 Modifica del Testo e del Markup

Esistono tre proprietà principali per interagire con il contenuto di un nodo:

1. `textContent`: Accede al puro testo contenuto, ignorando qualsiasi tag HTML interno. È la scelta più sicura per inserire dati utente (evita attacchi XSS).
2. `innerHTML`: Permette di leggere o iniettare frammenti di codice HTML. Il browser eseguirà il parsing dei tag forniti.
3. `outerHTML`: Include anche il tag dell'elemento stesso nella stringa di markup.

Sicurezza e Performance: Usare `innerHTML` frequentemente può rallentare la pagina poiché costringe il browser a ricostruire parte dell'albero del DOM. Inoltre, iniettare stringhe non filtrate in `innerHTML` espone a rischi di sicurezza. Prediligi `textContent` quando non devi inserire tag.

5.10.2 Gestione degli Attributi

Gli attributi di un elemento (come `src`, `href`, `alt`, `disabled`) possono essere manipolati tramite metodi specifici:

- `element.getAttribute("attr")`: Legge il valore di un attributo.
- `element.setAttribute("attr", "valore")`: Crea o aggiorna un attributo.
- `element.removeAttribute("attr")`: Rimuove l'attributo.
- **Proprietà dirette**: Molti attributi comuni sono accessibili direttamente (es. `element.id`, `element.src`).

```
1 const img = document.querySelector("#logo");
2
3 // Cambio dinamico dell'immagine
4 img.setAttribute("src", "nuovo-logo.png");
5 img.alt = "Descrizione aggiornata";
6
7 // Manipolazione delle classi (classList API)
8 img.classList.add("evidenziato");
9 img.classList.toggle("nascosto");
```

5.11 Creazione Dinamica dei Nodi

Oltre a selezionare e modificare elementi esistenti, il DOM permette di creare da zero nuove strutture HTML e inserirle nell'albero. Questo processo richiede tipicamente tre passaggi: la creazione dell'elemento, la creazione del suo contenuto (se testuale), e l'inserimento nel documento.

5.11.1 Metodi di Manipolazione Strutturale

I metodi principali esposti dal nodo padre sono:

- `document.createElement("tag")`: Crea un nuovo nodo elemento vuoto in memoria (non ancora visibile sulla pagina).
- `document.createTextNode("testo")`: Crea un nodo testuale puro.
- `padre.appendChild(nuovoNodo)`: Aggiunge il nodo creato alla fine della lista dei figli del nodo padre.

- `padre.insertBefore(nuovoNodo, nodoRiferimento)`: Inserisce il nuovo nodo prima di un figlio specifico.
- `padre.removeChild(nodoFiglio)`: Elimina un nodo figlio.

```

1 // 1. Creazione del nodo elemento <p>
2 const p = document.createElement("p");
3
4 // 2. Creazione del nodo testo
5 const testo = document.createTextNode("Questo è un paragrafo dinamico");
6
7 // 3. Assemblaggio: inserisco il testo dentro il <p>
8 p.appendChild(testo);
9
10 // 4. Inserimento nel DOM: aggiungo il <p> dentro un div esistente
11 const container = document.getElementById("contenitore");
12 container.appendChild(p);

```

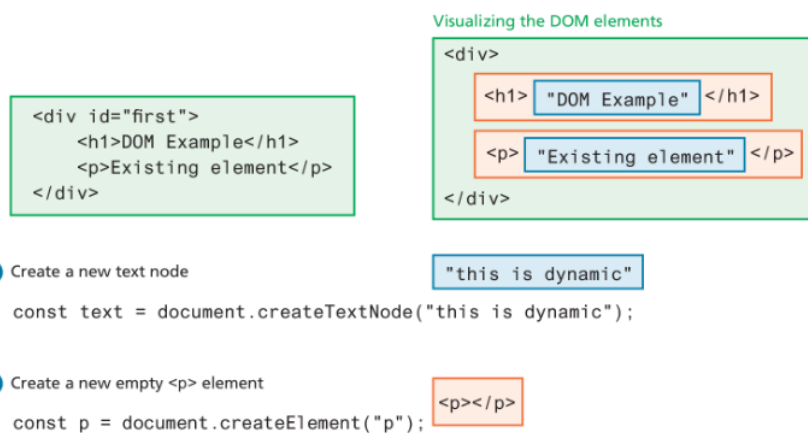


Figura 5.7: Schema visivo della creazione e inserimento dinamico di nodi nel DOM

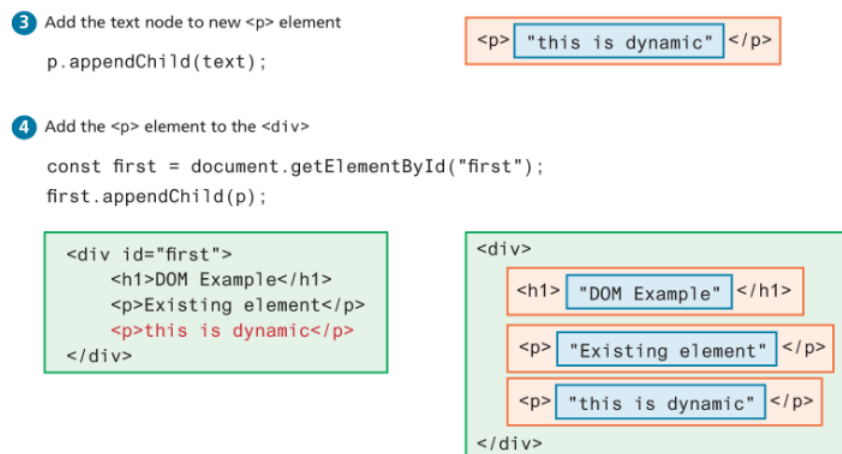


Figura 5.8: Schema visivo della creazione e inserimento dinamico di nodi nel DOM

5.12 DOM Timing: Quando eseguire lo script

Errore comune: Se lo script JavaScript cerca di selezionare e manipolare nodi HTML che il browser non ha ancora analizzato (parsato), restituirà errori come `null` o `undefined`. Il codice DOM non può operare finché l'albero non è pronto.

Per garantire che il codice venga eseguito al momento giusto (specialmente se importato nella `<head>`), è necessario attendere i corretti eventi di caricamento della pagina:

- **DOMContentLoaded:** Si attiva non appena il browser ha scaricato e analizzato completamente il documento HTML costruendo l'albero del DOM. **È l'evento raccomandato per la maggior parte degli script.**
- **load (su window):** Si attiva solo quando *tutta* la pagina (inclusi stili CSS pesanti, immagini e iframe) ha terminato il caricamento. Può risultare molto lento.

```
1 // Pratica standard per avviare l'applicazione in sicurezza
2 document.addEventListener("DOMContentLoaded", function() {
3   // Tutto il codice DOM va inserito qui
4   const btn = document.querySelector("#btn");
5   console.log("Il DOM è pronto!");
6 });
```

5.13 Gestione degli Eventi (Event Handling)

Un'interfaccia utente web è guidata dagli eventi (click, input da tastiera, scorrimento, ecc.). Gestire un evento significa “registrare” una funzione (chiamata *Event Handler* o *Callback*) che il browser eseguirà automaticamente non appena l'evento si verifica.

5.13.1 addEventListener

Il metodo moderno e raccomandato per associare eventi è `addEventListener()`.

A differenza dei vecchi attributi HTML (`onClick="..."`) o delle proprietà degli oggetti (`node.onclick = ...`), `addEventListener` permette di associare *più di un listener* allo stesso evento sul medesimo elemento senza sovrascriverli.

```
1 const bottone = document.querySelector("#invia");
2
3 // Metodo raccomandato: registrazione con funzione anonima (o freccia)
4 bottone.addEventListener("click", (e) => {
5   console.log("Bottone cliccato!");
6 });
```

5.13.2 L'Oggetto Event (e)

Quando un evento viene scatenato, il browser costruisce un **Oggetto Event** contenente tutti i dettagli dell'azione e lo passa automaticamente come primo argomento alla funzione di callback (convenzionalmente chiamato `e` o `event`).

Proprietà cruciali dell'oggetto event:

- **e.target:** Il nodo esatto che ha generato l'evento (l'elemento fisicamente cliccato).
- **e.type:** Il tipo di evento scatenato (es. `"click"`, `"keydown"`).
- **e.preventDefault():** Metodo fondamentale che blocca il comportamento di default del browser (es. impedisce a un link di ricaricare la pagina o a un form di essere inviato).

5.14 Propagazione degli Eventi: Capturing e Bubbling

Quando si verifica un evento su un nodo interno (es. uno `<span>` dentro un `<div>`), l'evento non colpisce solo quel nodo, ma si propaga attraverso la gerarchia del DOM in due fasi:

1. **Event Capturing (Discesa):** L'evento viaggia dal root (`<html>`) scendendo verso l'elemento bersaglio.
2. **Event Bubbling (Risalita):** È il comportamento di default. L'evento "bolle" dall'elemento bersaglio risalendo verso tutti i suoi genitori, attivando eventuali event listener registrati in ascolto anche su di essi.

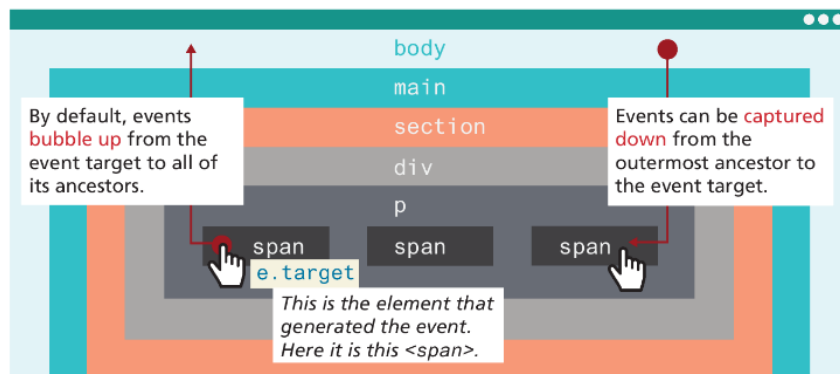


Figura 5.9: Schema visivo di Event Capturing ed Event Bubbling lungo i nodi del DOM

Fermare la propagazione: A volte il bubbling causa problemi attivando inavvertitamente gli handler dei contenitori padre. Chiamando `e.stopPropagation()` all'interno della callback, si "uccide" l'evento impedendogli di risalire ulteriormente.

5.14.1 Event Delegation (Delegazione degli Eventi)

Event Delegation: Una tecnica di ottimizzazione in cui, invece di assegnare un `addEventListener` a dozzine o centinaia di elementi figli (es. tutti gli `<li>` di una lista), si assegna un *singolo* event listener all'elemento padre (`<ul>`).

Sfruttando l'Event Bubbling, il click su un figlio risalirà fino al padre. Lì, usando `e.target`, possiamo determinare esattamente quale figlio ha scatenato l'evento. Questo risparmia memoria e funziona automaticamente anche per i nodi figli creati dinamicamente in seguito.

```
1 const listaPadre = document.querySelector("#miaLista");
2
3 listaPadre.addEventListener("click", function(e) {
4   // Controllo se l'elemento cliccato è effettivamente un LI
5   if (e.target && e.target.nodeName === "LI") {
6     console.log("Hai cliccato l'elemento: " + e.target.textContent);
7   }
8 });
```

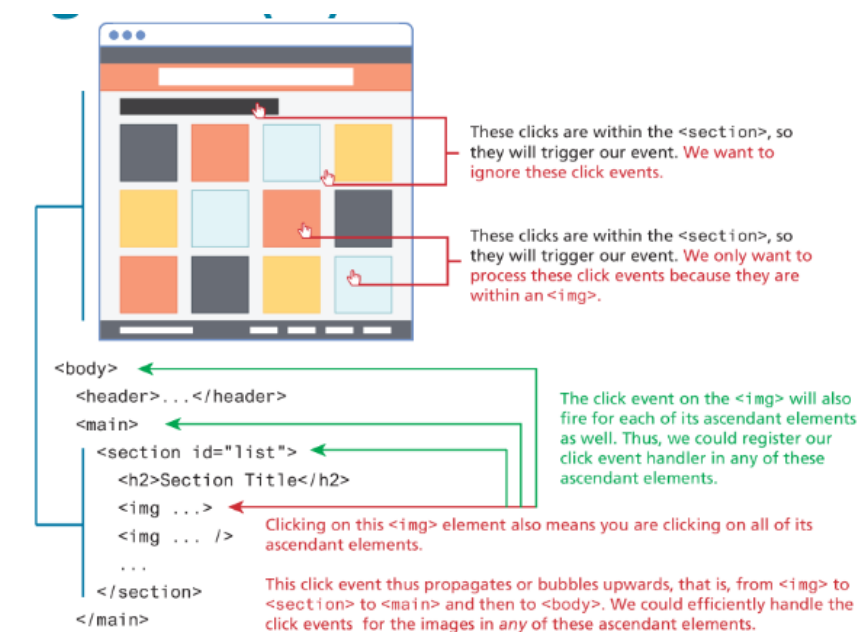


Figura 5.10: Esempio pratico visivo di Event Delegation applicato a una griglia di immagini

5.15 Accesso ai Dati Custom (Dataset)

Per collegare dati specifici a un elemento HTML senza usare attributi non standard o id complessi, HTML5 fornisce gli attributi `data-*`. Tramite JavaScript, questi valori sono facilmente accessibili (sia in lettura che in scrittura) attraverso la proprietà `dataset`.

```

1 <article id="prodotto" data-id="123" data-categoria="elettronica">
2   ...
3 </article>

1 // JavaScript
2 const articolo = document.querySelector("#prodotto");
3
4 // Lettura dei valori data-* (nota che il prefisso "data-" viene ommesso in JS)
5 console.log(articolo.dataset.id); // "123"
6 console.log(articolo.dataset.categoria); // "elettronica"
7
8 // Scrittura
9 articolo.dataset.disponibile = "true"; // Aggiunge data-disponibile="true"

```

5.16 Interazione coi Form e Validazione

JavaScript gioca un ruolo cruciale nel migliorare l'usabilità (UX) e nel pre-validare i form prima che i dati vengano inviati al server. Questo riduce le richieste inutili e dà un feedback immediato all'utente.

5.16.1 Eventi Chiave dei Form

- **focus**: L'utente seleziona o entra in un campo di input.
- **blur**: Il campo di input perde il focus (es. l'utente clicca fuori).
- **change**: Il valore di un `<input>`, `<select>` o `<textarea>` viene modificato (e il focus viene perso).
- **submit**: L'utente tenta di inviare il modulo.

5.16.2 Validazione Client-Side

La validazione si aggancia generalmente all'evento `submit` del tag `<form>` (non al pulsante). Se il controllo fallisce, è obbligatorio usare `e.preventDefault()` per impedire al browser di eseguire la richiesta HTTP verso il server.

```
1 const form = document.querySelector("#loginForm");
2
3 form.addEventListener("submit", (e) => {
4     const userField = document.querySelector("#username").value;
5
6     // Validazione base: controllo campo vuoto
7     if (userField === null || userField.trim() === "") {
8         e.preventDefault(); // BLOCCA L'INVIO DEL FORM
9         alert("Inserisci un nome utente valido!");
10    }
11 });
```

5.16.3 Regular Expressions (Espressioni Regolari)

Le Regex sono potenti pattern di ricerca usati estesamente per validare formati di testo complessi (email, codici fiscali, date). In JavaScript, una Regex letterale viene definita racchiudendola tra due slash `/`.

Le espressioni regolari usano caratteri letterali e **metacaratteri** (simboli con significati speciali come `^` inizio stringa, `$` fine stringa, `.` qualsiasi carattere, `+` uno o più, ecc.).

```
1 // Semplice regex pattern case sensitive in JavaScript
2 const pattern = /ran/;
3
4 // Troverà match nelle stringhe "randy" o "cranberry"
5 const isValid = pattern.test("randy"); // restituisce true
```

6 JavaScript - Addendum(08)

6.1 Variabili e Scope

In JavaScript, la dichiarazione delle variabili determina la loro visibilità (scope) e il loro comportamento a runtime. Esistono tre costrutti principali per dichiarare una variabile: `var`, `let` e `const`.

6.1.1 Differenze tra var, let e const

Scope (Ambito di visibilità): Definisce la porzione di codice in cui una variabile è accessibile. Può essere globale, a livello di funzione o a livello di blocco (delimitato da parentesi graffe `{ }`).

- **var:** Ha un *function scope*. Se dichiarata all'esterno di una funzione, diventa una variabile globale e crea una proprietà nell'oggetto globale (es. `window` nei browser). Permette la ridichiarazione senza generare errori, sovrascrivendo e mantenendo il valore precedente.
- **let:** Costrutto più moderno, ha un *block scope* (valido solo all'interno del blocco `{ }` in cui è definito). Non permette la ridichiarazione all'interno dello stesso blocco concettuale e non inquina l'oggetto globale.
- **const:** Ha un *block scope* come `let`, ma deve essere obbligatoriamente inizializzata al momento della dichiarazione. Non permette la riassegnazione del riferimento (è una costante), sebbene il contenuto di oggetti o array definiti come `const` possa essere mutato internamente.

Variabili non dichiarate

Se si assegna un valore a una variabile senza prima dichiararla (omettendo `var`, `let` o `const`), JavaScript la inizierà automaticamente come **variabile globale**. Questo è un comportamento profondamente prone a errori (error-prone).

L'uso di `var` è oggi considerato una pratica obsoleta nella maggior parte dei casi. L'adozione di `let` e `const` previene bug insidiosi legati allo scope e all'inquinamento dello spazio dei nomi globale.

```
1 // Esempio pratico let vs var
2 let x = 10;
3 if (x === 10) {
4   let x = 20; // Variabile completamente diversa, confinata nel blocco if
5   console.log(x); // Stampa 20
6 }
7 console.log(x); // Stampa 10
8
9 function varExample() {
10   var y = 10;
11   {
12     var y = 20; // È la stessa variabile y, lo scope di blocco viene ignorato
13     console.log(y); // Stampa 20
14   }
15   console.log(y); // Stampa 20
16 }
```

6.2 L'Oggetto Globale e la keyword `this`

6.2.1 L'Oggetto Globale

L'oggetto globale rappresenta lo scope globale dell'ambiente di esecuzione. Nel browser è esposto tramite `window`, mentre in un ambiente come Node.js tramite `global`. Per garantire interoperabilità tra ambienti, le versioni moderne di JavaScript mettono a disposizione la proprietà `globalThis`, che punta sempre al corretto oggetto globale. Le variabili globali diventano automaticamente proprietà accessibili su questo oggetto.

6.2.2 Il contesto `this`

Il significato della keyword `this` è dinamico e dipende dal contesto di esecuzione, ovvero da *come* e da *quale oggetto* la funzione viene invocata, e non da dove viene lessicalmente dichiarata.

This substitution: In modalità non stretta (non-strict mode), se una funzione viene invocata con un contesto `this` mancante (`undefined` o `null`), il motore JavaScript lo sostituisce automaticamente per farlo puntare all'oggetto globale (`globalThis`).

```
1 function getThis() {
2   return this;
3 }
4
5 const obj1 = { name: "Oggetto 1" };
6 obj1.metodo = getThis;
7
8 // Il this viene iniettato dall'oggetto invocante (obj1)
9 console.log(obj1.metodo());
10
11 function globalContext() {
12   this.prova = 999;
13 }
14 globalContext(); // Chiamata "scollegata", punta a globalThis
15 console.log(globalThis.prova); // Stampa 999
```

6.3 Operatori di Confronto e Type Coercion

JavaScript implementa due diverse filosofie di confronto: l'uguaglianza rigorosa e l'uguaglianza astratta. La distinzione è fondamentale per garantire affidabilità nella logica di controllo.

6.3.1 Strict Equality (`===` e `!==`)

Il controllo rigoroso verifica contemporaneamente sia il **valore** che il **tipo** degli operandi. Non viene mai effettuata alcuna conversione implicita.

- Tipi primitivi diversi tra loro restituiscono sempre **false**.
- `null === null` e `undefined === undefined` sono validi e restituiscono **true**.
- Un caso speciale riguarda `NaN` (Not a Number): questo valore non è mai uguale a nulla, nemmeno a se stesso (`NaN === NaN` restituisce **false**). Per verificare la presenza di un valore illegale si usa la funzione `isNaN()` o il pattern `x !== x`.
- Due oggetti, array o funzioni risultano uguali *solo* se condividono lo stesso esatto riferimento in memoria, a prescindere dall'uguaglianza dei loro valori interni.

6.3.2 Abstract Equality (== e !=)

L'operatore astratto tenta di conciliare tipi di dato discordanti tramite la **coercizione dei tipi** (type conversion) prima di effettuare il confronto.

Comportamenti anomali dell'Abstract Equality

A causa delle complesse regole di conversione automatica, l'uso di `==` può scatenare bug silenti. Ad esempio, nel confronto tra un booleano e un numero, il booleano viene prima trasformato (**true** diventa 1, **false** diventa 0), e successivamente comparato.

Salvo casi d'uso molto specifici (come il check contemporaneo di nullità: `valore == null` che intercetta sia `null` che `undefined`), è caldamente consigliato limitare l'uso dei confronti alle versioni strict (`===`).

```
1 // Esempi di uguaglianza rigorosa vs astratta
2 console.log(5 == "5"); // true (la stringa "5" viene convertita in numero 5 prima del test)
3 console.log(5 === "5"); // false (valutazione immediata: number vs string)
4
5 console.log(true == 1); // true
6 console.log(true === 1); // false
7
8 // Casi limite e inaspettati (Weird parts)
9 console.log(null == undefined); // true
10 console.log(null > 0); // false
11 console.log(null >= 0); // true (null viene prima forzato a 0 per operatori relazionali)
12 console.log([] == ![]); // true (un array vuoto è "truthy", ma subisce bizzarre coercizioni incrociate)
```

6.4 Funzioni e Dinamismo degli Argomenti

In JavaScript, le funzioni tradizionali (dichiarate con la keyword `function`) possiedono una flessibilità intrinseca riguardo al numero di parametri accettati. Non è necessario che il numero di argomenti passati durante l'invocazione corrisponda esattamente ai parametri definiti nella firma della funzione.

L'oggetto `arguments`: È una variabile locale, simile a un array (array-like object), disponibile all'interno di tutte le funzioni classiche. Contiene i valori di tutti gli argomenti passati alla funzione al momento della chiamata.

Sfruttando la proprietà `arguments.length`, è possibile creare funzioni in grado di gestire un numero variabile di parametri (funzioni variadiche).

```
1 // Esempio di funzione con numero di argomenti dinamico
2 function concatena() {
3   let risultato = "";
4   // Iteriamo sugli argomenti passati, anche se nessuno è dichiarato nella firma
5   for (let i = 0; i < arguments.length; i++) {
6     risultato += arguments[i] + " ";
7   }
8   return risultato;
9 }
10
11 console.log(concatena("JavaScript", "è", "flessibile")); // Stampa "JavaScript è flessibile "
```

6.4.1 Funzioni Predefinite (Globali)

Il linguaggio fornisce alcune funzioni globali pronte all'uso per operazioni comuni:

- `isNaN(valore)`: Valuta se l'argomento fornito è un valore "Not-a-Number" illegale. Ritorna **true** se il valore è NaN.
- `isFinite(valore)`: Determina se un numero è finito e legale (scartando `Infinity` o NaN).

- `parseInt(stringa, base)` e `parseFloat(stringa)`: Analizzano una stringa e tentano di estrarne un numero intero (specificando opzionalmente la base numerica) o in virgola mobile.
- `eval(stringa)`: Valuta ed esegue una stringa come se fosse codice sorgente JavaScript.

Attenzione a eval() L'uso di `eval()` è fortemente sconsigliato in contesti reali per gravi rischi di sicurezza (potrebbe eseguire codice malevolo iniettato dall'utente) e impatti negativi sulle performance.

6.5 Interazione Base: Finestre di Dialogo

JavaScript nel browser permette di interagire con l'utente tramite tre semplici finestre di dialogo (modali, bloccano l'esecuzione della pagina finché non vengono chiuse).

1. **Alert Box**: Mostra un messaggio informativo. L'utente può solo cliccare "OK".

```
1 alert("Operazione completata con successo!");
```

2. **Confirm Box**: Richiede all'utente di prendere una decisione. Ritorna `true` se l'utente clicca "OK", `false` se clicca "Annulla" (Cancel).

```
1 if (confirm("Sei sicuro di voler procedere?")) {  
2     // Esegui azione  
3 }
```

3. **Prompt Box**: Richiede un input testuale all'utente. Ritorna la stringa inserita se si clicca "OK", oppure `null` se si clicca "Annulla".

```
1 // Il secondo parametro è il valore di default opzionale  
2 let nome = prompt("Inserisci il tuo nome", "Utente");
```

6.6 Gestione degli Errori (Eccezioni)

Il blocco `try...catch` permette di gestire fluidamente gli errori a runtime, evitando che l'applicazione vada in crash (terminazione anomala).

- **try**: Delimita il blocco di codice in cui si sospetta possa verificarsi un errore.
- **catch (err)**: Contiene il codice da eseguire per gestire l'eccezione se si verifica un errore nel blocco `try`. La variabile `err` è l'oggetto eccezione generato.
- **throw**: Consente di generare (sollevare) un'eccezione personalizzata. Può lanciare qualsiasi tipo di dato: stringhe, numeri, booleani o oggetti strutturati.

```
1 let input = "testo";  
2 try {  
3     if (isNaN(input)) {  
4         throw "Errore: Input non numerico!"; // Lancio eccezione custom  
5     }  
6     let calcolo = input * 10;  
7 } catch (errore) {  
8     console.log("Intercettato: " + errore);  
9     // Gestione silenziosa o notifica all'utente  
10 }
```

6.7 Oggetti: Fondamenti

Gli oggetti in JavaScript sono collezioni dinamiche di proprietà, dove ogni proprietà è un'associazione chiave-valore. Una proprietà può contenere tipi primitivi, altri oggetti o funzioni (in tal caso, la proprietà prende il nome di **metodo**).

6.7.1 Creazione tramite Object_INITIALIZER

Il modo più rapido e diffuso per creare una singola istanza di un oggetto è l'inizializzatore (literal notation). Se la proprietà da inserire ha lo stesso nome della variabile che ne contiene il valore, è possibile usare una sintassi abbreviata (shorthand property).

```
1 let eta = 25;
2 const utente = {
3   nome: "Mario",
4   cognome: "Rossi",
5   eta, // Shorthand per eta: eta
6   contatto: { // Oggetto annidato
7     email: "mario@email.it"
8   }
9 };
```

6.7.2 Accesso alle Proprietà: Dot vs Associative Array Notation

Esistono due notazioni per leggere o scrivere le proprietà di un oggetto:

1. **Dot Notation** (`oggetto.proprietà`): La sintassi più comune, richiede che il nome della proprietà sia un identificatore JavaScript valido (niente spazi, non può iniziare con un numero, ecc.). Se si tenta di accedere a una proprietà inesistente, viene restituito `undefined`.
2. **Associative Array Notation** (`oggetto["proprietà"]`): Permette di accedere alle proprietà usando stringhe. È essenziale quando:

- Il nome della proprietà contiene caratteri speciali o spazi (es. `oggetto["nome composto"]`).
- La chiave da accedere è calcolata dinamicamente a runtime (contenuta in una variabile).

```
1 const auto = { marca: "Fiat", modello: "500" };
2
3 // Aggiunta dinamica di una proprietà
4 auto.anno = 2020;
5
6 // Accesso dinamico tramite variabile
7 let chiaveDinamica = "modello";
8 console.log(auto[chaveDinamica]); // Stampa "500"
```

6.7.3 Iterazione sulle Proprietà: il costrutto for...in

Il ciclo `for...in` è progettato specificamente per scorrere tutte le chiavi (proprietà) enumerabili di un oggetto.

Durante ogni iterazione del `for...in`, la variabile di controllo assume il valore del *nome* della chiave (stringa), non del suo contenuto. Per accedere al valore, è necessario usare la notazione ad array associativo `oggetto[chave]`.

```
1 const studente = {
2   matricola: 1234,
3   voto: 28,
4   lode: false
5 };
6
7 for (let chiave in studente) {
8   console.log("Proprietà: " + chiave + " -> Valore: " + studente[chave]);
9 }
```

6.8 Costruttori e Prototipi

Oltre agli Object Initializer (o literal), JavaScript mette a disposizione un secondo meccanismo fondamentale per la creazione di oggetti: le **funzioni costruttore**. Questo approccio è indispensabile quando si ha la necessità di creare molteplici istanze aventi la medesima struttura, simulando di fatto il concetto di “classe” tipico dei linguaggi orientati agli oggetti tradizionali.

6.8.1 Le Funzioni Costruttore

Una funzione costruttore non è altro che una normale funzione JavaScript, ma è progettata per essere invocata tramite l'operatore **new**. Al suo interno, la keyword **this** viene utilizzata per assegnare le proprietà e i metodi alla nascente istanza dell'oggetto.

Operatore new: Quando invocato su una funzione, crea dinamicamente un nuovo oggetto vuoto, imposta il suo costruttore e lega questo nuovo oggetto al contesto **this** all'interno del corpo della funzione.

```
1 // Esempio di Funzione Costruttore
2 function Automobile(marca, modello, anno) {
3   this.marca = marca;
4   this.modello = modello;
5   this.anno = anno;
6
7   // Metodo definito direttamente nel costruttore (meno efficiente)
8   this.mostraDettagli = function() {
9     console.log(this.marca + " " + this.modello);
10  };
11 }
12
13 let miaAuto = new Automobile("Fiat", "500", 2020);
14 miaAuto.mostraDettagli(); // Stampa "Fiat 500"
```

Definire i metodi direttamente all'interno del costruttore (come `mostraDettagli` nell'esempio sopra) implica che ogni singola istanza creata avrà la propria copia indipendente di quella funzione, consumando più memoria del necessario. La soluzione ottimale è utilizzare il **prototype**.

6.8.2 La Catena dei Prototipi (Prototype Chain)

L'ereditarietà in JavaScript non è basata sulle classi (fino a ES6, che introduce una sintassi “zuccherata” ma sotto il cofano rimane prototipale), bensì sui **prototipi**.

Ogni oggetto in JavaScript possiede un collegamento interno a un altro oggetto, chiamato il suo prototipo (accessibile tramite la proprietà `__proto__` o meglio tramite i metodi standard). Se si tenta di accedere a una proprietà o a un metodo che non esiste direttamente sull'oggetto in questione (Own Property), l'interprete risalirà la **catena dei prototipi** (Prototype Chain) cercando la proprietà nell'oggetto prototipo genitore, e così via, finché non la trova o raggiunge il fondo della catena (**null**).

La proprietà **prototype** è un attributo speciale presente su tutte le funzioni costruttore. Qualsiasi metodo o proprietà aggiunto all'oggetto `Costruttore.prototype` verrà condiviso e reso accessibile automaticamente a tutte le istanze create tramite quel costruttore.

```
1 function Studente(nome, matricola) {
2   this.nome = nome;
3   this.matricola = matricola;
4   this.voti = [];
5 }
6
```

```

7 // Aggiungiamo un metodo condiviso tramite prototype
8 Studente.prototype.aggiungiVoto = function(voto) {
9   this.voti.push(voto);
10 };
11
12 let studente1 = new Studente("Sara", 1234);
13 let studente2 = new Studente("Paolo", 5678);
14
15 // Entrambi condividono la stessa funzione in memoria
16 studente1.aggiungiVoto(30);
17 studente2.aggiungiVoto(28);

```

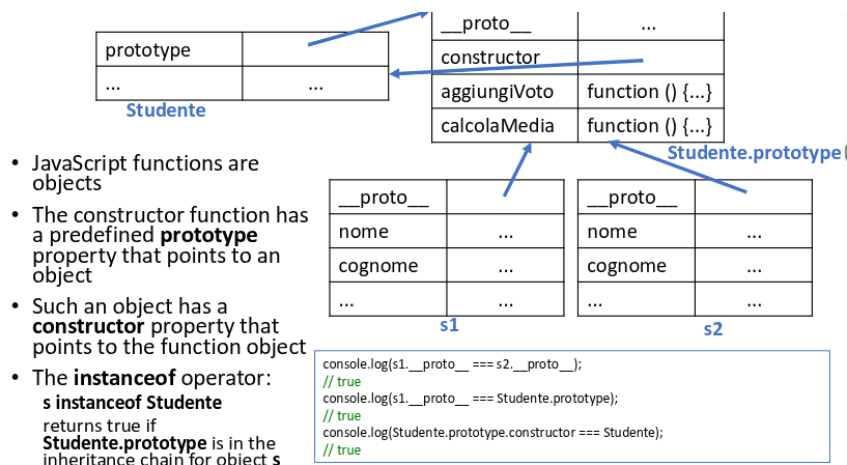


Figura 6.1: Schema visivo della Catena dei Prototipi tra istanze e Object.prototype

6.8.3 Manipolazione sicura dei Prototipi

Sebbene esista la proprietà speciale `__proto__`, la manipolazione diretta è considerata una cattiva pratica. JavaScript moderno offre metodi statici sull'oggetto globale `Object` per gestire in modo sicuro le catene ereditari:

- `Object.getPrototypeOf(oggetto)`: Restituisce il prototipo dell'oggetto.
- `Object.setPrototypeOf(oggetto, nuovoPrototipo)`: Sovrascrive il prototipo di un oggetto esistente.
- `Object.create(prototipo)`: Crea e restituisce un nuovo oggetto vuoto che eredita direttamente dall'oggetto passato come parametro.

6.9 Oggetti Predefiniti (Built-in Objects): Array

JavaScript offre svariati oggetti "built-in" pronti all'uso per gestire operazioni matematiche, date, testo e strutture dati. L'oggetto globale `Array` è una delle collezioni di dati più versatili del linguaggio.

Un Array è una sequenza ordinata di valori indicizzati. A differenza di linguaggi a tipizzazione statica, gli array in JavaScript possono contenere elementi di tipi misti nella stessa struttura e si ridimensionano dinamicamente in base alle necessità.

Creazione di un Array e ambiguità del costruttore

Usare `new Array()` può generare ambiguità se viene passato un solo numero intero come argomento, poiché invece di creare un array con quel numero come unico elemento, creerà un array *vuoto* con una lunghezza preimpostata pari a quel numero. È sempre consigliato preferire l'uso delle parentesi quadre (Array Literal).

```
1 let myArrayLiteral = ["Saab", "Volvo", "BMW"]; // Consigliato
2 let arrayMisto = [10, "Testo", { chiave: "valore" }, true];
3
4 // Aggiungere elementi espande automaticamente la lunghezza (length)
5 myArrayLiteral[3] = "Ferrari";
6 console.log(myArrayLiteral.length); // Stampa 4
```

6.9.1 Metodi fondamentali degli Array

Gli array possiedono numerosi metodi ereditati da `Array.prototype` che permettono di manipolarne il contenuto:

- **Modifica della coda e della testa:**

- `push(elemento)`: Aggiunge uno o più elementi alla *fine* e ritorna la nuova lunghezza.
- `pop()`: Rimuove l'ultimo elemento e lo restituisce.
- `unshift(elemento)`: Aggiunge uno o più elementi all'*inizio*.
- `shift()`: Rimuove e restituisce il primo elemento.

- **Manipolazione interna:**

- `splice(indice, quantiDaRimuovere, ...elementiDaAggiungere)`: Strumento potentissimo (coltellino svizzero) per rimuovere, sostituire o aggiungere elementi in qualsiasi posizione dell'array.
- `slice(inizio, fine)`: Estrae una porzione dell'array (sotto-array) senza modificare l'array originale (operazione *non distruttiva*).

- **Utilità:**

- `join(separatore)`: Unisce tutti gli elementi in un'unica stringa testuale, separandoli con il separatore fornito.
- `concat(altroArray)`: Fonde due o più array restituendone uno nuovo.

Un comportamento peculiare riguarda il metodo `sort()`. Per impostazione predefinita, esso converte temporaneamente gli elementi in stringhe e li ordina alfabeticamente. Se si esegue su un array di numeri, **40** verrà posizionato prima di **5** poiché il carattere "4" viene prima di "5". Per ordinare correttamente i numeri è obbligatorio passare una **funzione di comparazione** (compare function).

```
1 let numeri = [10, 5, 40, 25];
2
3 // Funzione di comparazione numerica ascendente
4 function comparaNumeri(a, b) {
5   return a - b;
6 }
7
8 numeri.sort(comparaNumeri);
9 console.log(numeri); // Stampa [5, 10, 25, 40]
```

6.10 Oggetti Predefiniti (Built-in Objects): Seconda Parte

Dopo aver analizzato gli Array, esploriamo gli altri principali oggetti globali forniti nativamente da JavaScript per la gestione di tipi primitivi complessi, date e calcoli matematici.

6.10.1 Boolean e Regole di Coercizione

In JavaScript, i valori booleani possono esistere sia come tipi primitivi (**true/false**) sia come "oggetti involucro" (wrapper objects) tramite `new Boolean()`. Tuttavia, la forma più comune e utile è l'uso di `Boolean(valore)` come **funzione di conversione** per forzare la trasformazione di un qualsiasi tipo di dato in un booleano primitivo.

Truthy e Falsy: Ogni valore in JavaScript ha una sua intrinseca natura booleana quando valutato in un contesto logico (es. in un `if`). I valori che si convertono in **true** sono detti *truthy*, quelli che diventano **false** sono *falsy*.

I valori **Falsy** in JavaScript sono rigorosamente sei:

1. **false**
2. **undefined**
3. **null**
4. **0** (e `-0`)
5. **NaN**
6. **""** (stringa vuota)

L'insidia degli Oggetti e Array vuoti

Tutti gli oggetti e gli array, anche se vuoti (`{}` o `[]`), sono sempre considerati **Truthy**. Tuttavia, a causa delle regole di uguaglianza astratta, `[] == false` restituisce **true** (poiché l'array vuoto subisce una conversione in stringa vuota, e poi in numero `0`, confrontato con **false** convertito in `0`). Per evitare impazzimenti, usa sempre `===`.

6.10.2 Date

JavaScript gestisce le date contando il numero di **millisecondi** trascorsi dal 1° Gennaio 1970 (Epoch time).

È possibile creare un'istanza dell'oggetto **Date** in 4 modi:

1. **new Date()**: Data e ora corrente.
2. **new Date(millisecondi)**: Passando i millisecondi dall'Epoch.
3. **new Date(stringa)**: Parsando una stringa, preferibilmente in formato ISO 8601 (es. `"2023-12-25T19:15:00"`). Formati diversi sono altamente sconsigliati per problemi di compatibilità tra browser.
4. **new Date(anno, mese, giorno, ore, minuti, secondi, ms)**: Passando i parametri numerici. *Nota: il mese parte da 0 (Gennaio = 0).*

```
1 let d = new Date(); // Data odierna
2 console.log(d.getFullYear()); // Es: 2024 (mai usare getYear)
3 console.log(d.getMonth()); // Es: 9 (Corrisponde a Ottobre)
4 console.log(d.toLocaleDateString()); // Restituisce una stringa formattata secondo il locale del sistema
```

6.10.3 Math

L'oggetto **Math** si differenzia dagli altri perché **non è un costruttore**. Non si crea un'istanza di **Math** (es. `new Math()` genera errore), ma si usano direttamente le sue proprietà e metodi statici per calcoli matematici.

- **Costanti:** `Math.PI`, `Math.E`
- **Metodi di arrotondamento:** `Math.ceil(x)` (arrotonda per eccesso), `Math.floor(x)` (per difetto), `Math.round(x)` (all'intero più vicino).
- **Utilità:** `Math.random()` (genera un numero pseudo-casuale tra 0 compreso e 1 escluso), `Math.max()`, `Math.min()`.

6.10.4 Number e String

Come per **Boolean**, anche **Number** e **String** possono fungere da costruttori (creando scomodi oggetti wrapper) o da utilissime funzioni di conversione universale.

Quando chiami un metodo su un tipo primitivo testuale (es. `"ciao".toUpperCase()`), JavaScript crea silenziosamente e temporaneamente un oggetto `String` wrapper, esegue il metodo, e poi distrugge l'oggetto.

- **Metodi Number:** `num.toFixed(n)` formatta un numero mantenendo esattamente `n` cifre decimali (restituendo una stringa).
- **Metodi String:** `split(separatore)` per suddividere una stringa in un array, `substring(inizio, fine)` per estrarne una parte, `toUpperCase()/toLowerCase()`.

6.11 Elementi di JavaScript Moderno (ES6+)

L'aggiornamento ECMAScript 6 (2015) ha rivoluzionato JavaScript, introducendo una sintassi più pulita e funzionalità avanzate per la gestione dei dati e delle strutture di controllo.

6.11.1 Il ciclo `for...of`

Mentre il `for...in` itera sulle *chiavi* di un oggetto, il `for...of` itera direttamente sui **valori** di oggetti *iterabili* (come Array, Stringhe, Map e Set).

Gli Oggetti standard (`{}`) non sono nativamente iterabili con `for...of` e generano un `TypeError`. Per iterare sui valori o sulle coppie chiave-valore di un oggetto tramite `for...of`, si devono usare i metodi statici `Object.keys()`, `Object.values()` o `Object.entries()`.

```
1 let array = [10, 20, 30];
2 for (let valore of array) {
3   console.log(valore); // Stampa 10, 20, 30
4 }
5
6 // Iterazione sicura su un oggetto
7 let utente = { nome: "Mario", ruolo: "Admin" };
8 for (let [chiave, valore] of Object.entries(utente)) {
9   console.log(chiave + ": " + valore);
10 }
```

6.11.2 Destructuring Assignment (Assegnamento Destrutturato)

Il destructuring permette di estrarre in modo rapido ed elegante valori da Array o proprietà da Oggetti, assegnandoli a variabili distinte in una singola riga di codice.

```
1 // Destrutturazione Array
2 let [a, b, c] = [1, 2, 3];
3
4 // Scambio di variabili elegante senza variabile d'appoggio temporanea
5 [a, b] = [b, a];
6
7 // Saltare elementi o raccogliere i rimanenti (rest pattern)
8 let [primo, , terzo, ...rimanenti] = [10, 20, 30, 40, 50];
9 console.log(rimanenti); // Stampa [40, 50]
10
11 // Destrutturazione Oggetti (le chiavi devono corrispondere ai nomi delle variabili)
12 let obj = { k1: 42, k2: 24 };
13 let { k1, k2 } = obj;
```

6.11.3 Template String Literals e Default Parameters

Le **Template Strings** sostituiscono le faticose concatenazioni fatte con il `+`. Usando i backtick (```), permettono l'interpolazione diretta di variabili ed espressioni tramite il prefisso ``${}``.

I **Default Parameters** permettono di definire un valore di fallback direttamente nella firma della funzione, da usare se l'argomento non viene passato o risulta `undefined`.

```
1 function saluta(nome, grado = "Novizio") {  
2   // Interpolazione multi-riga tramite template literal  
3   return `Benvenuto ${nome},  
4     il tuo grado attuale è: ${grado}.`;   
5 }  
6  
7 console.log(saluta("Alice")); // Usa "Novizio" come default
```

6.11.4 Classi e Costruttori

Sebbene JavaScript rimanga un linguaggio basato sui prototipi (Prototype-based), ES6 ha introdotto la keyword **class**. Questa fornisce una sintassi “zuccherata” (syntactic sugar) molto più vicina ai linguaggi orientati agli oggetti classici (come Java o C++), nascondendo la complessità della manipolazione diretta dei prototipi.

- **Costruttore:** La funzione di inizializzazione si chiama esplicitamente **constructor**. Può essere omessa se non necessaria.
- **Metodi:** All'interno del blocco **class** {}, i metodi vengono definiti senza l'uso della parola chiave **function**.
- **Ereditarietà:** Si utilizza la parola chiave **extends** per derivare una classe da un'altra.
- **Super:** La keyword **super**() viene utilizzata all'interno del costruttore della sottoclasse per invocare il costruttore della classe padre. Può essere usata anche per richiamare metodi del padre (es. **super.toString()**).

Tutto il codice all'interno di una dichiarazione di classe viene eseguito implicitamente in **Strict Mode**. Inoltre, a differenza delle funzioni classiche, le classi non subiscono l'hoisting: devono essere dichiarate prima di essere istanziate.

```
1 class Studente {  
2   constructor(nome, corso) {  
3     this.nome = nome;  
4     this.corso = corso;  
5     this.voti = [];  
6   }  
7  
8   aggiungiVoto(voto) {  
9     this.voti.push(voto);  
10  }  
11 }  
12  
13 // Ereditarietà (Single inheritance)  
14 class StudenteLavoratore extends Studente {  
15   constructor(nome, corso, azienda) {  
16     super(nome, corso); // Invoca il costruttore di Studente  
17     this.azienda = azienda;  
18   }  
19  
20   mostraProfilo() {  
21     return `${this.nome} studia ${this.corso} e lavora presso ${this.azienda}`;  
22   }  
23 }  
24  
25 let w1 = new StudenteLavoratore("Gino", "Informatica", "Google");  
26 console.log(w1.mostraProfilo());
```

6.12 Strict Mode

Introdotta in ES5, lo Strict Mode è una direttiva che istruisce il motore JavaScript a eseguire il codice in un contesto molto più rigoroso, trasformando molti “errori silenziosi” in eccezioni esplicite (throw errors).

Attivazione: Si abilita inserendo la stringa letterale `"use strict"`; all'inizio di un file JavaScript (applica la regola a tutto il file) o all'inizio di una funzione (applica la regola solo a quella funzione).

Cosa proibisce lo Strict Mode

- L'uso di variabili non precedentemente dichiarate (previene la creazione accidentale di variabili globali).
- L'uso dell'istruzione `with` (fortemente deprecata).
- La presenza di parametri duplicati nella firma di una funzione (es. `function f(a, a)` genera errore).

6.13 Arrow Functions (Funzioni a Freccia)

Le Arrow Functions (ES6) offrono una sintassi estremamente compatta per scrivere espressioni di funzione. Sono particolarmente utili quando si devono passare funzioni anonime come argomenti (callbacks), ad esempio nei metodi degli array come `forEach`, `map` o `filter`.

Regole Sintattiche Fondamentali:

1. La parola chiave `function` viene eliminata, sostituita dalla freccia `=>` posta tra i parametri e il corpo.
2. Se c'è un **singolo parametro**, le parentesi tonde `()` possono essere omesse. (Se zero parametri, le `()` sono obbligatorie).
3. Se il corpo della funzione è composto da una **singola espressione**, le parentesi graffe `{ }` e la parola chiave `return` possono essere omesse (Ritorno Implicito).

```
1 // Funzione tradizionale anonima passata come callback
2 let array = [10, 20, 30];
3 array.forEach(function(elemento) {
4     console.log(elemento * 2);
5 });
6
7 // Stessa operazione con Arrow Function
8 array.forEach(elemento => console.log(elemento * 2));
9
10 // Arrow function con parametri multipli e ritorno implicito
11 const somma = (a, b) => a + b;
12 console.log(somma(5, 7)); // Stampa 12
13
14 // Arrow function senza parametri
15 const generaNumero = () => Math.random() * 10;
```

6.14 Spread Operator (...)

L'operatore di espansione (Spread Operator) permette di "spacchettare" gli elementi di un iterabile (come un Array o una Stringa) o le proprietà di un Oggetto (introdotto per gli oggetti in ES2018) in luoghi in cui sono previsti zero o più argomenti o elementi.

6.14.1 Spread sugli Array e Stringhe

È utilissimo per concatenare array, estrarre caratteri da una stringa, o creare copie superficiali (Shallow Copy).

```
1 let a1 = [10, 20, 30];
2 let a2 = [1, ...a1, 100]; // Spacchetta a1 dentro a2
3 console.log(a2); // Stampa [1, 10, 20, 30, 100]
4
5 // Shallow copy di un array
6 let copia = [...a1];
7
```

```
8 // Spacchettare una stringa in un array di caratteri
9 let caratteri = [..."Ciao"];
10 console.log(caratteri); // Stampa ['C', 'i', 'a', 'o']
```

6.14.2 Spread sugli Oggetti

Permette di clonare oggetti o fondere più oggetti in uno nuovo.

Regola della Sovrascrittura: Se durante la fusione (merge) di due oggetti tramite spread operator esistono proprietà con lo stesso nome, il valore dell'ultimo oggetto espanso (quello più a destra) sovrascriverà i precedenti.

Attenzione: Lo spread operator applicato agli oggetti copia solo le proprietà "proprie" (Own Properties) enumerabili, ignorando la catena dei prototipi e i metodi ereditati. Inoltre, esegue una copia superficiale (shallow copy): se una proprietà contiene un riferimento a un altro oggetto, verrà copiato il riferimento, non l'oggetto interno.

```
1 let studenteBase = { nome: "Mario", media: 25 };
2 let dettagliExtra = { corso: "Web", media: 28 }; // Proprietà "media" duplicata
3
4 // Clonazione e Merge
5 let profiloCompleto = { ...studenteBase, ...dettagliExtra, attivo: true };
6
7 // L'ultima proprietà inserita vince sui conflitti di nome
8 console.log(profiloCompleto);
9 // Stampa { nome: "Mario", media: 28, corso: "Web", attivo: true }
```


7 Intro al lato server - PHP

7.1 Introduzione allo Sviluppo Server-Side

L'hosting di base dei file è gestito tramite un web server, ma lo sviluppo server-side è un processo più avanzato: coinvolge l'uso di tecnologie di programmazione (come PHP) per creare script in grado di generare contenuti dinamicamente elaborando input, interfacciandosi con database, file system, email e altri servizi software.

7.1.1 Architettura Client vs Server

Esecuzione Client-Side: Il browser invia una richiesta per una risorsa (es. un file JavaScript). Il server restituisce il file sorgente così com'è. Il browser esegue poi lo script localmente sfruttando le proprie risorse.

Esecuzione Server-Side: Il browser richiede l'esecuzione di una risorsa dinamica (es. un file PHP). Il web server interpreta ed esegue il codice internamente (comunicando con database o altri servizi se necessario) e restituisce al browser unicamente l'output finale dell'elaborazione, solitamente in formato HTML puro.

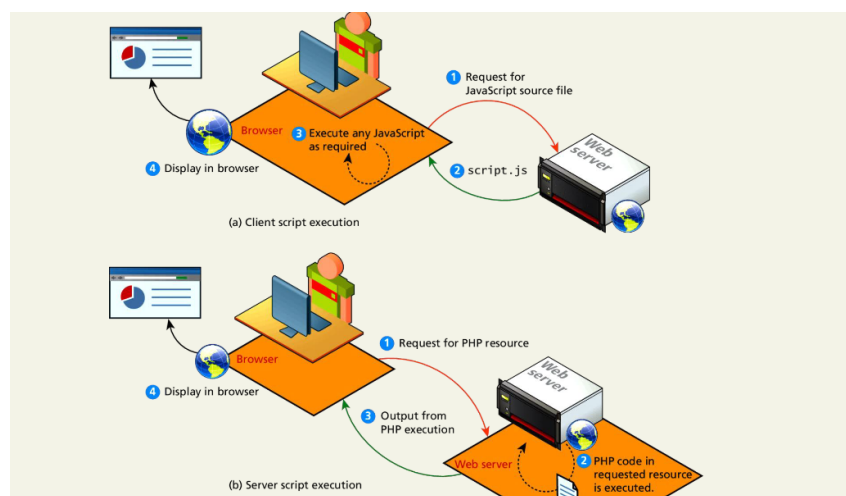


Figura 7.1: Comparing Client and Server Scripts

7.1.2 Tecnologie Server-Side Comuni

Nel panorama dello sviluppo backend esistono diverse tecnologie, ciascuna con le proprie peculiarità:

- **JSP (Java Server Pages):** Utilizza Java ed è fortemente orientato agli oggetti. Viene compilato tramite un compilatore JIT (Just-In-Time) garantendo esecuzioni rapide e natura cross-platform. È molto diffuso in ambienti intranet e sistemi enterprise di grandi dimensioni.

- **Node.js:** Permette di eseguire JavaScript sul server, unificando il linguaggio tra frontend e backend. A differenza di altre soluzioni, integra nativamente le funzionalità di web server, eliminando la stretta necessità di software esterni come Apache o IIS.
- **PHP:** Linguaggio a tipizzazione dinamica che può essere incorporato direttamente nell'HTML, pur supportando moderni paradigmi orientati agli oggetti (classi, ereditarietà).

I file PHP prima dell'esecuzione vengono compilati in una rappresentazione intermedia chiamata *opcodes* (analogamente al byte-code di Java). L'acronimo originale era *Personal Home Pages*, ma oggi è diventato l'acronimo ricorsivo "PHP: Hypertext Preprocessor".

7.2 Il Web Server e l'Ambiente di Esecuzione

7.2.1 Responsabilità del Web Server

Un web server (come Apache) funge da intermediario primario ed è responsabile di numerose operazioni critiche per il funzionamento di un'applicazione web:

- Gestione delle connessioni HTTP in ingresso.
- Smistamento delle richieste per risorse statiche e dinamiche.
- Controllo dei permessi e degli accessi.
- Compressione e crittografia dei dati trasferiti.
- Gestione di domini multipli, cookie, mantenimento dello stato della sessione e upload dei file.

7.2.2 Architettura LAMP e Apache

Lo sviluppo PHP si appoggia tipicamente allo stack software **LAMP**: Linux (Sistema Operativo), Apache (Web Server), MySQL (Database) e PHP. Per gli ambienti di sviluppo locali, si utilizzano frequentemente pacchetti preconfigurati multipiattaforma come XAMPP o MAMP.

Apache Daemon e Moduli: Apache viene eseguito come un *daemon* (un processo silente in background che attende il verificarsi di un evento, come l'arrivo di una richiesta su una determinata porta di rete). Quando riceve una richiesta, utilizza dei *moduli* (o handler, estensioni compilate spesso in C) per decidere come processarla.

Nel caso di una richiesta per una pagina PHP, il flusso attraversa vari moduli di Apache (come `mod_auth` per l'autenticazione, `mod_ssl` per la sicurezza) prima di essere intercettato da `mod_php5` (o versioni successive), che rappresenta l'interfaccia diretta tra Apache e l'ambiente PHP.

Le prestazioni e la gestione della concorrenza in Apache dipendono dalla sua configurazione: può operare in modalità **multi-process** (preforked), dove ogni richiesta genera un nuovo processo Apache distinto, oppure in modalità **multi-threaded** (worker), dove un singolo processo Apache gestisce la concorrenza allocando thread differenti per le varie richieste.

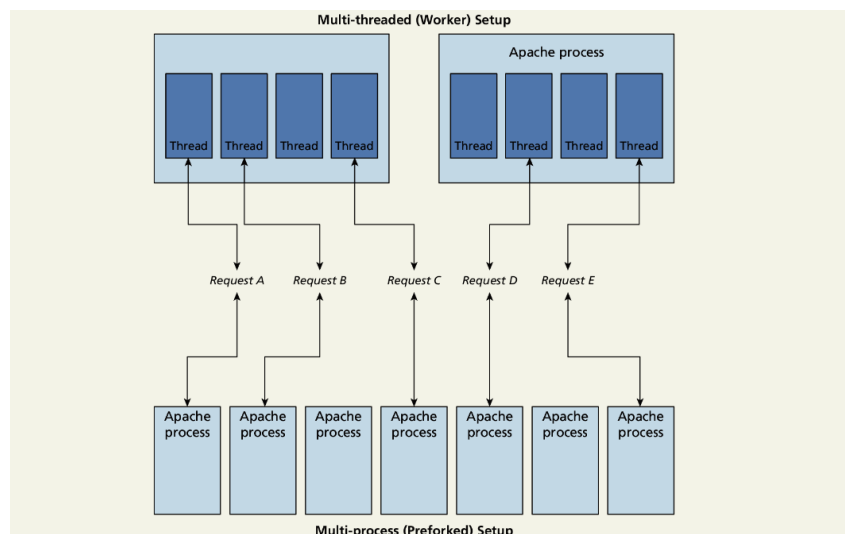


Figura 7.2: Multi-threaded vs Multi-process Setup

7.2.3 Internals di PHP

Il nucleo di PHP è scritto in linguaggio C ed è logicamente suddiviso in tre componenti principali:

1. **Core PHP:** Modulo base che definisce le funzionalità essenziali dell'ambiente (gestione delle variabili, manipolazione di stringhe, array, logica matematica e orientamento agli oggetti).
2. **Extension Layer:** Livello che fornisce le interfacce per comunicare con servizi esterni, offrendo le librerie necessarie per connettersi a database MySQL, gestire FTP, servizi SOAP o processare file XML.
3. **Zend Engine:** Il cuore esecutivo di PHP. Si occupa di ricevere in input il file PHP richiesto, compilarlo nel formato intermedio (*opcodes*) e procedere alla sua esecuzione.

7.3 Fondamenti e Sintassi di PHP

PHP è un linguaggio a tipizzazione dinamica e debole (le variabili possono cambiare tipo di dato nel tempo). Condivide gran parte della sintassi di base (cicli, condizionali, assegnazioni) con JavaScript e linguaggi C-like, ma se ne distanzia per la gestione delle variabili, delle stringhe e dell'inclusione dei file.

7.3.1 Tag e Integrazione con HTML

Il codice PHP può essere incorporato direttamente all'interno di un file HTML (che dovrà avere estensione `.php`). L'interprete esegue solo ciò che è racchiuso negli appositi tag, ignorando e inviando al client tutto il resto (es. markup HTML) come testo puro.

- **Tag standard:** Il codice deve essere inserito tra `<?php` e `?>`.
- **Tag short-echo:** Per stampare rapidamente una variabile o un'espressione, si utilizza `<?= "Testo"?>`, che equivale esattamente a `<?php echo "Testo"; ?>`.

Best Practice Architetture: Esistono due approcci principali per unire HTML e PHP. Il primo è “mescolare” logica e presentazione nello stesso file (utile per piccoli script). Il secondo, fortemente raccomandato per progetti complessi ed esami, prevede la **separazione** della logica (in file di classi o controller PHP puri) dalla presentazione (file HTML che includono il PHP solo per stampare i dati).

7.3.2 Variabili, Costanti e Commenti

In PHP, le variabili sono definite dinamicamente e non richiedono parole chiave come `let` o `var` per la dichiarazione, ma **devono sempre essere precedute dal simbolo del dollaro (\$)**.

- **Commenti:** PHP supporta i commenti a riga singola con `//` o `#`, e i commenti multilinea con `/* . . . */`.
- **Tipi di dato supportati:** Boolean, Integer, Float, String, Array (molto flessibili in PHP) e Object.

Costanti: A differenza delle variabili, il valore di una costante non può essere modificato dopo la sua definizione. Si dichiarano utilizzando la funzione `define()` e, per convenzione, si scrivono in maiuscolo. **Non** richiedono il prefisso `$` quando vengono richiamate.

```
1 # Esempio di dichiarazione variabili e costanti
2 $count = 42;
3 $username = "Mario";
4
5 define("DATABASE_LOCAL", "localhost");
6 // Connessione ipotetica utilizzando la costante
7 $db = new mysqli(DATABASE_LOCAL, "user", "password", "db_name");
```

7.3.3 Output e Manipolazione delle Stringhe

Per inviare l'output al browser, si utilizza principalmente il costrutto `echo`. La concatenazione tra stringhe non avviene con il segno `+` (come in JS), ma tramite il **punto (.)**.

Singoli vs Doppi Apici * Doppi apici (" "): Consentono l'interpolazione (parsing) diretta delle variabili al loro interno. Le sequenze di escape (es. `\n`, `\"`) vengono interpretate.

- **Singoli apici (' '):** Vengono trattati come stringhe letterali pure. Le variabili scritte al loro interno non vengono sostituite dal loro valore.

```
1 $firstName = "Pablo";
2 $lastName = "Picasso";
3
4 // Concatenazione standard con il punto
5 echo "<em>" . $firstName . " " . $lastName . "</em>";
6
7 // Interpolazione con doppi apici (stesso risultato)
8 echo "<em> $firstName $lastName </em>";
9
10 // Utilizzo dell'escape per stampare attributi HTML
11 echo "<img src=\"\$id.jpg\" alt=\"\$firstName\">";
```

Approfondimento: printf() > Per formattare l'output in modo complesso (es. mostrare un numero preciso di decimali), PHP offre `printf()`, derivata dal C. Utilizza dei *placeholder* (segnaposto) all'interno di una stringa, che vengono sostituiti dalle variabili passate come argomenti.

- `%s` per le stringhe.
- `%d` per gli interi.

- `%.2f` per i float (es. forza la stampa a 2 cifre decimali, utile per i prezzi).

7.4 Strutture di Controllo e Inclusione File

7.4.1 Condizionali e Cicli

La sintassi per `if...else`, `switch...case`, `while`, `do...while` e `for` è pressoché identica a quella di JavaScript e Java.

Sintassi Alternativa per i Template Quando si mescolano grossi blocchi di HTML con strutture di controllo PHP, le parentesi graffe `{ }` possono rendere il codice illeggibile. PHP fornisce una sintassi alternativa dove l'apertura della graffa è sostituita dai due punti `:`, e la chiusura da keyword specifiche come `endif;`, `endfor;`, `endwhile;`.

```
1 <?php if ($userStatus == "loggedin"): ?>
2   <a href="account.php">Il tuo Profilo</a>
3   <a href="logout.php">Esci</a>
4 <?php else: ?>
5   <a href="login.php">Accedi</a>
6 <?php endif; ?>
```

7.4.2 L'Inclusione dei File (Include e Require)

Una delle funzionalità più potenti e basilari di PHP per organizzare il codice (evitando duplicazioni per header, footer, o connessioni al DB) è l'inclusione di file esterni. È l'equivalente di fare "copia e incolla" del contenuto di un file in un altro prima dell'esecuzione.

Esistono 4 costrutti principali:

1. `include "file.php";`
2. `include_once "file.php";`
3. `require "file.php";`
4. `require_once "file.php";`

Differenza cruciale tra Include e Require * `include`: Se il file specificato non viene trovato, PHP emette un *Warning* (avviso) ma **continua** l'esecuzione dello script.

- **`require`:** Se il file non viene trovato, PHP emette un *Fatal Error* e **interrompe immediatamente** l'esecuzione dello script. (Utilizzare `require` per file critici come le configurazioni del database). I suffissi `_once` assicurano che il file non venga incluso più di una volta, prevenendo errori di ridefinizione di funzioni o classi.

Scope dei file inclusi: Le variabili definite nel file principale prima della riga dell'inclusione saranno accessibili all'interno del file incluso. Se l'inclusione avviene all'interno di una funzione, il codice del file incluso erediterà lo scope di quella specifica funzione (variabili locali).

7.5 Funzioni e Ambito delle Variabili

In PHP, l'utilizzo delle funzioni è fondamentale per non scrivere tutto il codice nel flusso principale dello script e per massimizzare la riusabilità. La sintassi per la definizione e la chiamata delle funzioni è strettamente derivata dal C e risulta molto

familiare per chi proviene da JavaScript.

7.5.1 Definizione e Sintassi Base

Una funzione può restituire un valore tramite la parola chiave **return** oppure limitarsi a eseguire un blocco di codice (producendo ad esempio un output HTML) senza restituire nulla.

Best Practice: È buona norma documentare sempre le funzioni spiegando i parametri richiesti e il tipo di dato restituito, utilizzando commenti multilinea `/** ... */` posizionati immediatamente prima della dichiarazione.

```
1  /**
2   * Questa funzione restituisce l'ora di sistema.
3   * Se $showSeconds è true, include anche i secondi.
4   */
5  function getNiceTime($showSeconds = true) {
6      if ($showSeconds) {
7          return date("H:i:s");
8      } else {
9          return date("H:i");
10     }
11 }
12
13 // Chiamata della funzione
14 echo getNiceTime(); // Usa il valore di default (true)
15 echo getNiceTime(false); // Sovrascrive il valore di default
```

7.5.2 Parametri e Valori di Default

Come mostrato nell'esempio precedente, PHP permette di assegnare dei **valori di default** ai parametri direttamente nella firma della funzione. Se la funzione viene invocata senza passare un argomento per quel parametro, l'interprete utilizzerà automaticamente il valore predefinito, evitando errori a runtime.

7.5.3 Passaggio per Valore vs Passaggio per Riferimento

Questo è uno dei concetti architetturali più critici di PHP per quanto riguarda la gestione della memoria e la manipolazione sicura dei dati.

Passaggio per Valore (Comportamento Default): Quando si passa una variabile a una funzione, PHP ne crea una **copia** locale in memoria. Qualsiasi modifica apportata al parametro all'interno della funzione non avrà alcun effetto sulla variabile originale definita all'esterno.

Passaggio per Riferimento: Aggiungendo una **e commerciale (&)** prima del nome del parametro nella dichiarazione della funzione, si passa un riferimento alla cella di memoria della variabile. Le modifiche effettuate dentro la funzione si rifletteranno direttamente e permanentemente sulla variabile originale.

```
1  // Esempio di passaggio per Valore
2  $initial = 15;
3  function changeByValue($arg) {
4      $arg += 300;
5  }
6  changeByValue($initial);
7  echo "Valore: " . $initial; // Stampa 15 (l'originale è intatto)
8
9  // Esempio di passaggio per Riferimento
10 function changeByReference(&$arg) {
11     $arg += 300;
12 }
13 changeByReference($initial);
14 echo "Riferimento: " . $initial; // Stampa 315 (l'originale è stato alterato)
```

7.6 Lo Scope (Ambito) delle Variabili

Le regole di visibilità delle variabili in PHP presentano differenze sostanziali rispetto a JavaScript, specialmente per quanto concerne l'interazione tra lo scope globale e quello locale.

7.6.1 Function Scope

Qualsiasi variabile definita all'interno di una funzione (inclusi i parametri stessi passati per valore) possiede **function scope**: esiste ed è accessibile esclusivamente durante l'esecuzione di quella specifica funzione. Al termine dell'esecuzione, la variabile viene distrutta.

Isolamento rigido dello Scope Globale

A differenza di altri linguaggi, in PHP le variabili create nel corpo principale dello script (scope globale) **non sono accessibili per impostazione predefinita** all'interno delle funzioni. Cercare di leggere o manipolare una variabile esterna senza dichiararla esplicitamente all'interno della funzione genererà un warning e non produrrà alcun risultato valido.

7.6.2 La Keyword `global`

Sebbene l'uso di variabili globali all'interno delle funzioni sia spesso sconsigliato per questioni di design, in casi in cui risulti inevitabile, PHP richiede l'utilizzo esplicito della parola chiave `global` per forzare l'accesso allo scope esterno.

```
1 $count = 56; // Variabile dichiarata nello scope globale
2
3 function testScope() {
4     // echo $count; // ERRORE: variabile non definita nel function scope
5
6     global $count; // Importa il riferimento alla variabile globale
7     echo $count;   // Ora stampa correttamente 56
8 }
9
10 testScope();
```


8 PHP - Arrays e Superglobals

8.1 Array in PHP

In PHP, un array è una struttura dati dinamica implementata come una **mappa ordinata**, che associa ogni valore a una specifica chiave. Rispetto ad altri linguaggi, gli array in PHP sono estremamente flessibili: possono crescere o ridursi dinamicamente senza doverne dichiarare la dimensione a priori.

8.1.1 Chiavi e Valori

Chiavi (Keys): Possono essere numeri interi o stringhe e non devono essere necessariamente sequenziali. Se non dichiarate esplicitamente, partono da 0 e si incrementano di 1. Non bisogna mai mescolare tipi di chiavi ambigui (es. l'intero 1 e la stringa "1").

Valori (Values): Non hanno alcuna restrizione di tipo. Possono contenere primitive (stringhe, numeri), oggetti o persino altri array.

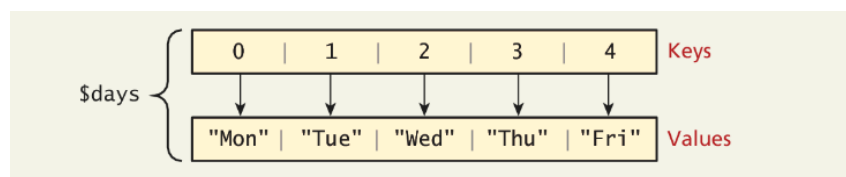


Figura 8.1: Arrays Key Value mapping

```
1 // Definizione di un array con chiavi implicite (sequenziali, base 0)
2 $days = array("Mon", "Tue", "Wed", "Thu", "Fri");
3 $days_alt = ["Mon", "Tue", "Wed", "Thu", "Fri"]; // Sintassi breve alternativa
4
5 // Aggiunta dinamica di elementi
6 $days[] = "Sun"; // Aggiunge automaticamente in coda
7 $days[7] = "Sat"; // Salta un indice (l'indice 6 sarà non definito/NULL)
8
9 // Array Associativo (mappatura esplicita con chiavi stringa)
10 $forecast = array("Mon" => 40, "Tue" => 47, "Wed" => 52);
11 echo $forecast["Tue"]; // Stampa 47
```

8.1.2 Array Multidimensionali

Essendo i valori non tipizzati, è possibile inserire array all'interno di altri array, creando strutture dati complesse. L'accesso agli elementi interni avviene concatenando la notazione a parentesi quadre.

```
1 $cart = array();
2 $cart[] = array("id" => 37, "title" => "Burial at Ornans");
3 $cart[] = array("id" => 63, "title" => "Starry Night");
4
5 // Accede al secondo array (indice 1), e ne estrapola il valore con chiave "title"
6 echo $cart[1]["title"]; // Stampa "Starry Night"
```

8.1.3 Iterazione degli Array

Se un array ha chiavi intere e puramente sequenziali, si possono usare i classici cicli **for**, **while** o **do...while**. Tuttavia, per gli array associativi o con chiavi “saltate”, si utilizza il costrutto nativo **foreach**, studiato appositamente per gli iteratori.

```
1 // Iterazione estraendo solo i valori
2 foreach ($forecast as $value) {
3     echo $value . "<br>";
4 }
5
6 // Iterazione estraendo contestualmente sia le chiavi che i valori
7 foreach ($forecast as $key => $value) {
8     echo "Giorno: " . $key . " - Temperatura: " . $value;
9 }
```

8.1.4 Operazioni Comuni e Ordinamento

Controllo ed Eliminazione:

- Per eliminare un elemento specifico si usa la funzione `unset($array[indice])`.
- Per controllare se un valore esiste per una determinata chiave (molto utile negli array non sequenziali) si usa `isset($array[indice])`. Restituisce **true** se il valore esiste, **false** altrimenti.

Approfondimento sull'ordinamento: Se utilizzi la funzione `sort($array)`, PHP ordinerà i valori dell'array ma **disturberà** l'associazione originale delle chiavi, riassegnando nuovi indici numerici sequenziali a partire da 0. Per ordinare l'array in base ai valori mantenendo intatta l'associazione tra la chiave e il suo valore, devi obbligatoriamente usare la funzione `asort($array)`.

8.2 Array Superglobali (\$_GET, \$_POST, \$_SERVER)

PHP fornisce speciali array associativi predefiniti chiamati **Superglobali**. Sono definiti “super” perché sono costantemente in scope ovunque nel codice (anche all'interno delle funzioni senza dover usare la keyword `global`) e catturano automaticamente i dati provenienti dal client e dall'ambiente.

8.2.1 \$_GET e \$_POST

Questi array gestiscono i dati inviati dal client (browser) verso il server.

- **\$_GET**: Raccoglie i dati passati visibilmente tramite la Query String dell'URL (es. `pagina.php?utente=Mario`).
- **\$_POST**: Raccoglie i dati inviati in modo invisibile nel corpo della richiesta HTTP (es. invio di moduli HTML).

Gestione Dati Multipli nei Form HTML

Se in un form crei un gruppo di checkbox con lo stesso nome (es. `name="day"`), PHP, leggendo l'array `$_GET` o `$_POST`, sovrascriverà i valori conservando solo l'ultimo checkbox selezionato. Per far capire a PHP che stai inviando un set multiplo, devi **obbligatoriamente** dichiarare il nome nel form HTML usando la notazione ad array: `name="day[]"`. Lato server, riceverai un vero e proprio array iterabile di scelte.

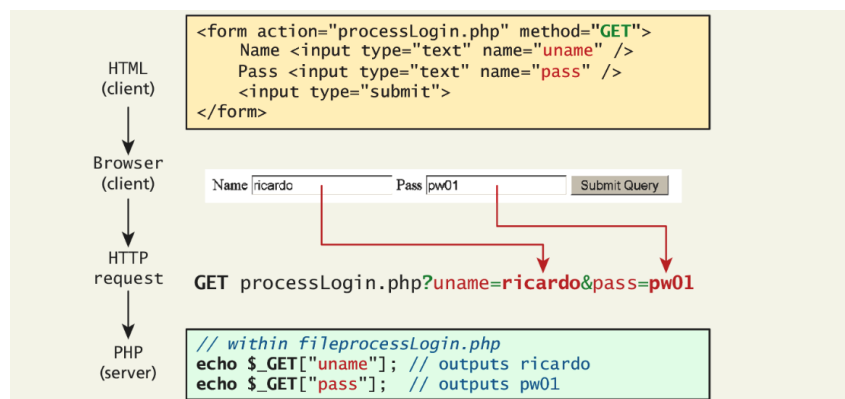


Figura 8.2: \$_GET and \$_POST HTTP request flow

```

1 // Controllo di base sulla ricezione dei dati
2 if ($_SERVER["REQUEST_METHOD"] == "POST") {
3     // Si verifica prima che i parametri esistano per evitare warning
4     if (isset($_POST["uname"]) && isset($_POST["pass"])) {
5         echo "Login in corso per: " . $_POST["uname"];
6     }
7 }

```

8.2.2 L'array \$_SERVER

`$_SERVER` contiene informazioni generate dal Web Server (es. Apache) riguardo gli header della richiesta, i percorsi e lo script in esecuzione.

Chiavi principali:

- **Ambiente:** `SERVER_NAME` (il dominio), `SERVER_ADDR` (IP del server), `DOCUMENT_ROOT` (cartella radice).
- **Richiesta:** `REQUEST_METHOD` (se la pagina è stata chiamata via GET o POST), `REMOTE_ADDR` (l'indirizzo IP dell'utente connesso).
- **Header Client:** `HTTP_USER_AGENT` (OS e Browser dell'utente), `HTTP_REFERER` (l'URL della pagina precedente che conteneva il link a quella attuale).

Attenzione alla Sicurezza (Spoofing)

Tutti gli header generati dal lato client (es. `HTTP_REFERER` e `HTTP_USER_AGENT`) possono essere facilmente falsificati (forged) da un utente malintenzionato. Mai basare meccanismi di sicurezza cruciali o logiche di autenticazione in modo esclusivo su queste chiavi!

8.3 Gestione degli Upload: L'Array \$_FILES

Quando un utente invia un file tramite un modulo web, PHP gestisce i dati ricevuti popolando l'array superglobale `$_FILES`.

8.3.1 Requisiti del Form HTML

Per poter inviare un file al server, il form HTML deve **obbligatoriamente** rispettare tre requisiti:

1. Utilizzare il metodo HTTP `POST`.
2. Includere l'attributo `enctype="multipart/form-data"` (necessario per codificare i dati binari).
3. Contenere un campo di input di tipo file: `<input type="file" name="file1"/>`.

8.3.2 Struttura di `$_FILES`

Per ogni file caricato, `$_FILES` crea un array associativo la cui chiave principale corrisponde all'attributo `name` del form HTML (es. `$_FILES['file1']`). Questo elemento è a sua volta un array contenente le seguenti chiavi essenziali:

- **name**: Il nome originale del file sul client (inclusa l'estensione).
- **type**: Il MIME type del file (es. `image/png`).
- **tmp_name**: Il percorso completo e temporaneo in cui il server ha salvato il file appena ricevuto.
- **size**: La dimensione del file in byte.
- **error**: Un codice intero che indica l'esito dell'upload. Se l'upload ha successo, il valore è `UPLOAD_ERR_OK` (che equivale a 0).

Validazione e Sicurezza dell'Upload

Limitare la dimensione o il tipo di file tramite attributi HTML (come il campo hidden `MAX_FILE_SIZE`) o controlli JavaScript è utile per la UX, ma è **facilmente aggirabile** dall'utente. La validazione critica (controllo della dimensione massima, verifica del MIME type nativo e dell'estensione) **deve** essere sempre implementata lato server tramite PHP.

8.3.3 Spostare il File Definitivamente

Al termine dell'esecuzione dello script PHP, i file temporanei caricati vengono distrutti. Se si desidera conservare l'upload, è necessario spostarlo manualmente in una cartella permanente del server utilizzando la funzione `move_uploaded_file()`.

```
1 $fileToMove = $_FILES['file1']['tmp_name'];
2 $destination = "./uploads/" . $_FILES["file1"]["name"];
3
4 // Verifica preventivamente gli errori prima di spostare
5 if ($_FILES['file1']['error'] === UPLOAD_ERR_OK) {
6     if (move_uploaded_file($fileToMove, $destination)) {
7         echo "File caricato e salvato con successo!";
8     } else {
9         echo "Errore durante lo spostamento del file.";
10    }
11 }
```

8.4 Interazione con il File System (I/O)

PHP offre due paradigmi principali per la lettura e la scrittura dei file fisici sul server: l'approccio a Stream (basato sul C) e l'approccio All-In-Memory.

8.4.1 Stream Access (Accesso Sequenziale)

Questo metodo legge il file un frammento (o una riga) alla volta.

- **Vantaggio**: È estremamente efficiente dal punto di vista della memoria (RAM), rendendolo l'unica scelta sensata per l'elaborazione di file di testo o log di dimensioni enormi.

- **Funzioni principali:** `fopen()` per aprire il flusso, `fgets()` per leggere una riga, `fwrite()` per scrivere, e `fclose()` per chiudere e liberare la risorsa.

```
1 // Lettura C-style tramite Stream
2 $f = fopen("log.txt", "r");
3 $lineNumber = 1;
4
5 while ($line = fgets($f)) {
6     echo $lineNumber . ": " . $line . "<br>";
7     $lineNumber++;
8 }
9 fclose($f);
```

8.4.2 All-In-Memory Access

Questo metodo carica l'intero contenuto del file direttamente nella memoria (RAM) di PHP.

- **Vantaggio:** Estremamente facile da implementare e ideale per elaborare, in poche righe di codice, configurazioni o dati CSV/delimitati. Non adatto per file di diversi gigabyte.

Funzioni Core per il File System in Memoria:

- `file($filename)`: Legge l'intero file e restituisce un array, dove ogni elemento corrisponde a una riga esatta del file.
- `file_get_contents($filename)`: Legge l'intero file e lo incapsula in un'unica enorme variabile stringa.
- `file_put_contents($filename, $string)`: Prende una stringa e la scrive in un file, creandolo se non esiste.

Esempio Classico (File CSV): Spesso nei progetti web (o esami) si richiede di leggere un file delimitato da virgole. L'approccio migliore è usare `file()` per ottenere l'array delle righe, iterare con un `foreach`, e usare `explode(", $line)` per "spaccare" la singola riga in un array di campi utilizzabili (es. autore, titolo, anno).

```
1 // Elaborazione di un file CSV in memoria
2 $paintings = file("paintings.txt") or die("Errore: File non trovato");
3
4 foreach ($paintings as $painting) {
5     // Es: "Picasso, The Actor, 1904" diventa ["Picasso", " The Actor", " 1904"]
6     $fields = explode(",", $painting);
7
8     $artist = $fields[0];
9     $title = $fields[1];
10    // Elaborazione dei dati estratti...
11 }
```


9 PHP - Classi e Oggetti

9.1 Programmazione Orientata agli Oggetti (OOP) e UML

A partire dalla versione 5.0, PHP supporta a pieno i paradigmi della programmazione orientata agli oggetti (OOP), offrendo costrutti sintattici simili a quelli di linguaggi enterprise come Java o C++.

Classe: Il modello astratto (blueprint) che definisce la struttura di un'entità, delineandone le proprietà e i metodi. **Oggetto (o Istanza):** La variabile concreta creata a partire da una classe. Ogni oggetto mantiene il proprio stato interno (i valori delle proprietà) in modo indipendente dagli altri oggetti generati dalla stessa classe. **Proprietà (o Attributi):** Le variabili definite all'interno di una classe che rappresentano lo stato dell'oggetto. **Metodi:** Le funzioni definite all'interno di una classe che espongono e definiscono il comportamento dell'oggetto.

Per progettare l'architettura software a oggetti si utilizza frequentemente l'**UML (Unified Modeling Language)**. In un diagramma delle classi UML, una classe viene rappresentata da un rettangolo diviso in tre sezioni logiche: nome della classe, attributi e metodi. La visibilità di ciascun elemento viene indicata con dei prefissi visivi (+ per le proprietà/metodi pubblici, - per quelli privati).

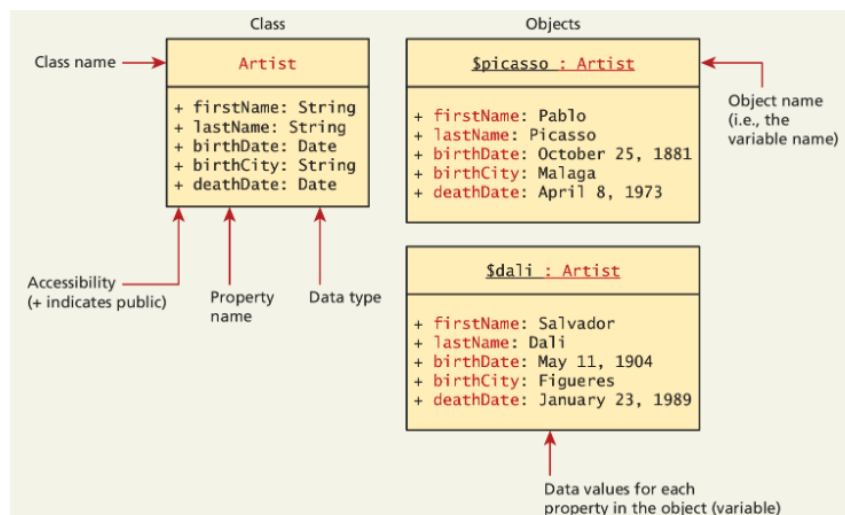


Figura 9.1: UML Class Diagram Example

9.2 Definizione di Classi e Oggetti in PHP

In PHP, la dichiarazione di una classe avviene tramite la parola chiave **class**. Per creare un'istanza (oggetto) in memoria a partire da tale definizione si utilizza la parola chiave **new**.

9.2.1 Visibilità e Incapsulamento

PHP definisce tre modificatori di visibilità per controllare l'accesso a proprietà e metodi:

- **Public:** L'elemento è accessibile da ovunque, sia dall'interno dei metodi della classe che dall'esterno (tramite l'istanza dell'oggetto).
- **Private:** L'elemento è rigidamente protetto ed è accessibile esclusivamente dai metodi definiti all'interno della classe stessa.
- **Protected:** L'elemento è accessibile dall'interno della classe stessa e da tutte le classi che la ereditano (sottoclassi).

L'operatore Freccia (→)

A differenza di linguaggi come Java o JavaScript che usano la notazione a punto (.), in PHP per accedere ai membri di un oggetto (proprietà o metodi) si utilizza l'operatore freccia →. Attenzione: quando si accede a una proprietà tramite la freccia, **non** bisogna inserire il simbolo del dollaro davanti al nome della proprietà. Esempio corretto: `$oggetto->variabile` (sbagliato: `$oggetto->$variabile`).

9.2.2 Il Riferimento all'Oggetto Corrente: \$this

All'interno dei metodi di una classe, è fondamentale avere un modo per riferirsi all'istanza specifica che sta invocando l'operazione in quel preciso momento. PHP mette a disposizione la pseudo-variabile `$this`, che funge da puntatore all'oggetto corrente.

Senza l'utilizzo di `$this`, PHP tratterebbe le variabili dichiarate all'interno dei metodi come semplici variabili locali, perdendo totalmente il collegamento con lo stato e le proprietà dell'oggetto istanziato.

9.2.3 Costruttori

Il costruttore è un metodo speciale che viene eseguito automaticamente dal motore PHP nel momento esatto in cui viene istanziato un nuovo oggetto tramite **new**. In PHP, il costruttore ha un nome fisso e riservato: `__construct()` (preceduto da due trattini bassi). La sua funzione primaria è quella di inizializzare le proprietà dell'oggetto passando i dati di partenza.

```
1 class Artista {
2     // Definizione delle proprietà con relativa visibilità
3     public $nome;
4     public $cognome;
5     private $idArtista;
6
7     // Metodo Costruttore
8     function __construct($nomeInput, $cognomeInput, $idInput) {
9         // $this si riferisce all'istanza appena creata in memoria
10        $this->nome = $nomeInput;
11        $this->cognome = $cognomeInput;
12        $this->idArtista = $idInput;
13    }
14
15    // Metodo pubblico
16    public function getNomeCompleto() {
17        return $this->nome . " " . $this->cognome;
18    }
19 }
20
21 // Creazione dell'oggetto invocando implicitamente __construct()
22 $pittore = new Artista("Pablo", "Picasso", 123);
23
24 // Chiamata a un metodo pubblico dell'istanza
25 echo $pittore->getNomeCompleto();
```

9.3 Funzionalità Avanzate OOP in PHP

Oltre ai concetti base di istanziazione e incapsulamento, PHP implementa caratteristiche avanzate tipiche dei linguaggi orientati agli oggetti maturi, permettendo di creare architetture software robuste e modulari.

9.3.1 Membri Statici (Static)

Membri Statici: Proprietà o metodi dichiarati con la parola chiave **static**. Appartengono alla classe nel suo complesso e non alle singole istanze (oggetti). Ciò significa che possono essere richiamati o letti senza dover prima creare un oggetto in memoria con **new**.

Poiché i metodi statici non operano su una specifica istanza, al loro interno **non** è possibile utilizzare la pseudo-variabile **\$this**. Per fare riferimento a una proprietà o a un metodo statico dall'interno della classe stessa, si utilizza la parola chiave **self** seguita dall'operatore di risoluzione di scope (i due punti doppi **::**). Dall'esterno, si usa il prefisso **NomeClasse::**.

```
1 class Configurazione {
2     public static $versione = "1.0";
3
4     public static function getVersion() {
5         // Uso di self:: per accedere a un membro statico interno
6         return self::$versione;
7     }
8 }
9
10 // Accesso diretto senza istanziare l'oggetto
11 echo Configurazione::getVersion();
```

9.3.2 Ereditarietà (Inheritance)

L'ereditarietà permette a una classe (sottoclasse o classe figlia) di ereditare tutte le proprietà e i metodi (pubblici e protetti) di un'altra classe (superclasse o classe padre). In PHP, l'ereditarietà è singola: una classe può estendere **una sola** classe padre alla volta.

La parola chiave **extends**

Per implementare l'ereditarietà si usa **extends**. La classe figlia può ridefinire (overriding) i metodi della classe padre per cambiarne o estenderne il comportamento.

Se la classe figlia ha bisogno di estendere la logica originale definita nella classe padre (ad esempio, per non riscrivere da zero l'intero costruttore base), deve utilizzare la parola chiave **parent::** per richiamare i metodi originali.

```
1 class LoggerBase {
2     protected $livello;
3
4     function __construct($livelloInput) {
5         $this->livello = $livelloInput;
6     }
7 }
8
9 class FileLogger extends LoggerBase {
10     private $file;
11
12     function __construct($filename, $livelloInput) {
13         // Richiama esplicitamente il costruttore della classe padre
14         parent::__construct($livelloInput);
15         $this->file = fopen($filename, "a");
16     }
17 }
```

9.3.3 Classi Astratte e Interfacce

Per definire contratti rigorosi e pattern architetturali (ad esempio nei logger o nei sistemi di connessione al DB), PHP sfrutta le classi astratte e le interfacce.

Classe Astratta (`abstract class`): Una classe che **non può mai essere istanziata** direttamente con `new` (genera un *Fatal Error*). Serve esclusivamente come struttura di base per altre classi. Può contenere sia metodi standard (con implementazione reale) sia metodi astratti (dichiarati ma privi di corpo), che le classi figlie saranno rigorosamente obbligate a implementare.

Interfaccia (`interface`): Un “contratto” puro e astratto. Contiene solo le firme dei metodi (tutti rigorosamente pubblici e senza alcuna implementazione). Una classe dichiara di aderire a un’interfaccia usando la parola chiave `implements`. A differenza dell’ereditarietà con `extends`, una classe può implementare **molteplici** interfacce contemporaneamente (separate da virgola).

Approfondimento Pratico: Usa l’ereditarietà (o le classi astratte) quando c’è una relazione gerarchica forte “è un” (un `FileLogger` è un `Logger Astratto`). Usa le interfacce per definire comportamenti o capacità aggiuntive e trasversali (es. un oggetto che è “Serializzabile” o “Autenticabile”).

9.3.4 Polimorfismo e Type Checking (`instanceof`)

Quando si progetta un sistema a oggetti e le funzioni accettano parametri generici, è spesso necessario verificare a runtime se un determinato oggetto deriva da una specifica classe o se implementa una certa interfaccia, specialmente prima di invocare un metodo critico.

L’operatore `instanceof`

PHP fornisce l’operatore nativo `instanceof` per testare l’ereditarietà e le interfacce. Restituisce `true` (stampato come “1”) se l’oggetto a sinistra appartiene alla classe a destra, o se la classe a destra è una superclasse o un’interfaccia presente in un punto qualsiasi della sua catena di ereditarietà.

```
1 // Presupponendo le definizioni di FileLogger, Logger (interfaccia) e AbstractLogger
2 $fl = new FileLogger("out.txt", 1);
3
4 // Verifica del tipo
5 if ($fl instanceof FileLogger) {
6     echo "L'oggetto è esattamente un FileLogger.";
7 }
8
9 // Questo restituirà true se FileLogger estende AbstractLogger
10 if ($fl instanceof AbstractLogger) {
11     echo "L'oggetto fa parte della famiglia AbstractLogger.";
12 }
```

10 Databases(12)

10.1 Interazione tra PHP e Database

10.1.1 Il Ruolo dei Database nello Sviluppo Web

Nello sviluppo web moderno, l'utilizzo di un database permette di implementare uno dei principi fondamentali dell'ingegneria del software: **separare ciò che varia da ciò che rimane invariato**.

In un'applicazione dinamica, il contenuto (i dati) varia continuamente in base alle interazioni e alle richieste dell'utente (spesso tramite parametri `GET` o `POST`), mentre il markup (il design e la struttura HTML) rimane costante. Il codice PHP funge da intermediario logico: determina quali dati mostrare, interroga il database tramite un'API specifica e inietta i risultati all'interno della struttura della pagina.

10.1.2 Transazioni e Integrità dei Dati

Quando si effettuano modifiche al database (operazioni di `UPDATE`, `INSERT` o `DELETE`), è fondamentale gestire la possibilità di fallimenti hardware, di rete o software per evitare che il database rimanga in uno stato parziale e incoerente.

Transazione: Una sequenza di operazioni sul database che viene trattata come un'unità logica singola e indivisibile. Garantisce che le operazioni vengano eseguite tutte con successo in blocco, oppure che nessuna di esse venga applicata, mantenendo la coerenza dei dati.

Un classico esempio è un processo di acquisto e-commerce: scrittura del record dell'ordine, controllo della carta di credito, rimozione dell'oggetto dall'inventario e notifica di spedizione. Se il server si disconnette o l'inventario non è aggiornabile, l'intero processo deve essere interrotto e i passaggi precedenti annullati in modo pulito.

10.1.2.1 Tipologie di Transazioni

- **Transazioni Locali:** Gestite interamente dal singolo DBMS (Database Management System). Sono le più semplici da implementare e coprono la stragrande maggioranza dei casi d'uso nello sviluppo web standard.
- **Transazioni Distribuite:** Coinvolgono server multipli e sistemi indipendenti sui quali si ha un controllo limitato. Sono notevolmente più complesse da orchestrare.

10.1.2.2 Comandi SQL per le Transazioni Locali

Le transazioni a livello di SQL si basano su tre comandi fondamentali:

```
1  /* Avvia la transazione: le modifiche successive sono "in sospeso" */
2  START TRANSACTION;
3
4  INSERT INTO ordini ...
5  UPDATE inventario ...
6
```

```
7  /* Se tutto il blocco logico ha successo, si rendono definitive le modifiche */
8  COMMIT;
9
10 /* In caso di errore, si annullano le modifiche ripristinando lo stato iniziale */
11 ROLLBACK;
```

DDL vs DML

Il Data Manipulation Language (DML, come i comandi visti sopra) è pienamente supportato dalle transazioni. Al contrario, i comandi di Data Definition Language (DDL), utilizzati per creare o modificare la struttura delle tabelle, in molti DBMS non possono essere annullati con un **ROLLBACK**.

10.1.3 API PHP per la Gestione dei Database

Per comunicare in modo programmatico con un database, PHP utilizza un'API (Application Programming Interface). Nel contesto specifico di MySQL, PHP ha storicamente offerto diverse opzioni:

10.1.3.1 L'estensione MySQL originale

Era l'estensione base di tipo procedurale. Oggi è **completamente deprecata** ed è incompatibile o sconsigliata per qualsiasi versione di MySQL successiva alla 4.1.3.

10.1.3.2 L'estensione MySQLi (MySQL Improved)

Nata come evoluzione diretta dell'estensione originale.

- Fornisce una doppia interfaccia: supporta sia un approccio **procedurale** che uno **orientato agli oggetti (OOP)**.
- Permette di sfruttare tutte le funzionalità più recenti ed esclusive dell'ecosistema MySQL.

10.1.3.3 PDO (PHP Data Objects)

Introdotta a partire da PHP 5.1, è un'API rigorosamente ed esclusivamente **orientata agli oggetti**.

- Fornisce un **layer di astrazione**: grazie a un set di classi uniforme, permette di interfacciarsi con molteplici tipologie di database (es. PostgreSQL, Oracle, SQL Server), semplicemente cambiando il driver di connessione, senza dover riscrivere la logica delle query.
- L'unico vero limite è l'impossibilità, in rari casi, di utilizzare funzionalità estremamente specifiche o di nicchia native solo in MySQL.

Quale API preferire per i progetti?

PDO è lo standard de facto per applicazioni scalabili o che potrebbero prevedere una futura migrazione del motore database. **MySQLi** resta un'ottima scelta se il progetto è fortemente ancorato a MySQL e si desidera mantenere script procedurali, anche se l'approccio a oggetti garantisce un codice più pulito e manutenibile.

10.2 Il Ciclo di Vita dell'Interazione con il Database

Indipendentemente dall'API scelta (PDO o MySQLi), l'interazione tra un'applicazione PHP e un database segue un algoritmo standard composto da cinque fasi sequenziali e imprescindibili:

1. Connessione al server database.
2. Gestione di eventuali errori di connessione.
3. Esecuzione della query SQL.
4. Elaborazione dei risultati (Result Set).
5. Rilascio delle risorse e chiusura della connessione.

Best Practice: Separazione delle Credenziali

Non hard-codare mai le credenziali del database direttamente negli script operativi. È buona norma definire costanti in un file separato (es. `config.php`) da includere dove necessario, facilitando il passaggio dagli ambienti di sviluppo a quelli di produzione.

```
1 // Esempio config.php
2 define('DBHOST', 'localhost');
3 define('DBNAME', 'bookcrm');
4 define('DBUSER', 'testuser');
5 define('DBPASS', 'mypassword');
```

10.2.1 Connessione e Gestione degli Errori

La fase di connessione è critica e deve sempre prevedere un blocco di controllo degli errori per evitare che l'applicazione mostri all'utente stack trace o blocchi l'esecuzione in modo non gestito.

Approccio PDO (Orientato agli Oggetti): PDO utilizza un blocco **try-catch** per intercettare le eccezioni generate durante il tentativo di connessione.

```
1 try {
2     // Definizione della stringa di connessione (DSN)
3     $connString = "mysql:host=" . DBHOST . ";dbname=" . DBNAME;
4
5     // Istanziamento dell'oggetto PDO
6     $pdo = new PDO($connString, DBUSER, DBPASS);
7
8     // Impostazione della modalità di errore su Exception
9     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
10
11 } catch (PDOException $e) {
12     // Intercettazione e blocco dell'esecuzione in caso di errore
13     die("Errore di connessione: " . $e->getMessage());
14 }
```

Approccio MySQLi (Procedurale): MySQLi utilizza funzioni condizionali standard per verificare lo stato dell'oggetto di connessione restituito.

```
1 $connection = mysqli_connect(DBHOST, DBUSER, DBPASS, DBNAME);
2
3 // mysqli_connect_errno() restituisce il codice dell'ultimo errore
4 if (mysqli_connect_errno()) {
5     die("Connessione fallita: " . mysqli_connect_error());
6 }
```

10.2.2 Esecuzione delle Query

Una volta stabilita la connessione, l'esecuzione varia a seconda che la query restituisca dei dati (come le **SELECT**) o effettui solo manipolazioni (come **UPDATE**, **INSERT**, **DELETE**).

Se la query **non** restituisce dati, l'interesse principale è sapere quante righe sono state modificate. In PDO si utilizza il metodo `exec()`, mentre in MySQLi si usa la funzione `mysqli_query()` seguita da `mysqli_affected_rows()`.

```
1 // Esempio PDO per query senza ritorno di dati
2 $sql = "UPDATE Categories SET CategoryName='Web' WHERE CategoryName='Business'";
3 $count = $pdo->exec($sql);
4 echo "<p>Aggiornate $count righe</p>";
```

10.3 Sicurezza: Prevenzione e Prepared Statements

10.3.1 La vulnerabilità SQL Injection

Integrare input utente (provenienti da `$_GET` o `$_POST`) direttamente in una stringa SQL espone l'applicazione alla vulnerabilità più critica e diffusa del web: l'**SQL Injection**.

Come funziona l'attacco

Se costruiamo la query così: `SELECT * FROM Users WHERE uname='$user'` Un hacker potrebbe inserire nel campo username la stringa: `' ; TRUNCATE TABLE Users; #`

La query risultante diventerebbe: `SELECT * FROM Users WHERE uname=''; TRUNCATE TABLE Users; #`

Il simbolo `#` commenta il resto della query originale, forzando il database a eseguire l'istruzione malevola `TRUNCATE`, cancellando tutti i dati.

Il metodo tradizionale (ma ormai insufficiente) per difendersi consisteva nel **sanitizzare** l'input usando funzioni come `mysqli_real_escape_string()` o `$pdo->quote()`. Questo approccio non copre tutti i casi (ad esempio, gli input numerici attesi come interi, dove i singoli apici non bloccano l'injection: `id = 9999 OR 1=1`).

10.3.2 I Prepared Statements (Istruzioni Prepare)

La soluzione moderna, sicura e standard per risolvere alla radice le vulnerabilità di injection sono i **Prepared Statements**.

Prepared Statement: Una query SQL pre-compilata inviata al server database in cui i parametri dinamici sono sostituiti da segnaposto (placeholder). I valori reali vengono inviati successivamente in un pacchetto separato.

I vantaggi sono due:

1. **Sicurezza assoluta:** Il database tratta i valori inviati strettamente come dati (stringhe, interi) e non li concatenerà mai come parte del codice SQL eseguibile. La sanitizzazione è automatica.
2. **Performance:** Se la stessa query deve essere eseguita più volte in un ciclo (es. multipli `INSERT`), il database la compila una sola volta, ottimizzando drasticamente i tempi di esecuzione.

10.3.2.1 Implementazione con PDO

PDO supporta due tipologie di placeholder: **posizionali** (il punto di domanda `?`) o **nominativi** (parole precedute dai due punti `:nome`). I nominativi sono caldamente consigliati perché rendono il codice molto più leggibile, specie in query lunghe.

```
1 /* Metodo PDO con Parametri Nominativi (Raccomandato) */
2
3 // 1. Preparazione della query con segnaposto
4 $sql = "INSERT INTO books (ISBN, Title, Year) VALUES (:isbn, :titolo, :anno)";
5 $statement = $pdo->prepare($sql);
6
7 // 2. Binding (Associazione) dei valori reali ai segnaposto
```

```
8 $statement->bindValue(':isbn', $_POST['isbn']);
9 $statement->bindValue(':titolo', $_POST['title']);
10 $statement->bindValue(':anno', $_POST['year']);
11
12 // 3. Esecuzione
13 $statement->execute();
```

10.3.2.2 Implementazione con MySQLi

MySQLi supporta solo i segnaposto posizionali (?) e richiede di specificare esplicitamente il tipo di dato di ogni variabile passata tramite una stringa di binding (**s** per stringa, **i** per intero, **d** per double, **b** per blob).

```
1 /* Metodo MySQLi con placeholder posizionali */
2
3 $id = $_GET['id'];
4 $sql = "SELECT Title, CopyrightYear FROM Books WHERE ID = ?";
5
6 // 1. Preparazione
7 if ($statement = mysqli_prepare($connection, $sql)) {
8
9     // 2. Binding: 'i' specifica che $id deve essere trattato come Integer
10    mysqli_stmt_bind_param($statement, 'i', $id);
11
12    // 3. Esecuzione
13    mysqli_stmt_execute($statement);
14 }
```

10.4 Elaborazione dei Risultati (Result Set)

Dopo aver eseguito una query di selezione (**SELECT**), il database non restituisce direttamente i dati utilizzabili, ma un *Result Set*, ovvero un puntatore ai dati recuperati. È necessario iterare su questo set riga per riga per estrarre le informazioni in PHP.

10.4.1 Fetching con PDO

In PDO, il metodo più rapido e comune per iterare sui risultati è `fetch()`. Di default, restituisce un array misto in cui i dati sono accessibili sia tramite indice numerico (0, 1, 2...) che tramite chiave associativa (il nome esatto della colonna nel database).

```
1 $sql = "SELECT ID, CategoryName FROM Categories ORDER BY CategoryName";
2 $result = $pdo->query($sql);
3
4 // Iterazione del result set riga per riga
5 while ($row = $result->fetch()) {
6     echo $row['ID'] . " - " . $row['CategoryName'] . "<br/>";
7 }
```

10.4.1.1 Fetch in un Oggetto (Object Mapping)

Una funzionalità avanzata di PDO è la capacità di popolare istanze di classi personalizzate con i dati estratti tramite `fetchObject()`. Affinché la mappatura avvenga automaticamente, i nomi delle colonne della query **devono coincidere esattamente** (in modo case-sensitive) con le proprietà della classe PHP.

```
1 class Book {
2     public $id;
3     public $title;
4     public $copyrightYear;
5 }
6
7 $statement = $pdo->prepare("SELECT id, title, copyrightYear FROM Books WHERE id = ?");
8 $statement->bindValue(1, $_GET['id']);
9 $statement->execute();
10
```

```

11 // Il risultato viene mappato direttamente in un'istanza della classe 'Book'
12 $book = $statement->fetchObject('Book');
13
14 echo "Titolo: " . $book->title . " (" . $book->copyrightYear . ")";

```

10.4.2 Fetching con MySQLi (Prepared Statements)

Con MySQLi procedurale e l'uso essenziale dei prepared statements, l'approccio è differente: si devono dichiarare a priori e “legare” (bind) le variabili PHP in cui il sistema riverserà i risultati per ogni fetch.

```

1 $sql = "SELECT Title, CopyrightYear FROM Books WHERE ID = ?";
2 if ($statement = mysqli_prepare($connection, $sql)) {
3     mysqli_stmt_bind_param($statement, 'i', $id);
4     mysqli_stmt_execute($statement);
5
6     // Binding delle variabili di output
7     mysqli_stmt_bind_result($statement, $title, $year);
8
9     // Ad ogni fetch(), $title e $year vengono aggiornate con i dati della riga corrente
10    while (mysqli_stmt_fetch($statement)) {
11        echo "$title ($year)<br/>";
12    }
13 }

```

10.5 Chiusura della Connessione e Rilascio Risorse

Una volta terminate le operazioni logiche sul database per la singola richiesta HTTP, è fondamentale liberare la memoria allocata dai *Result Set* e chiudere esplicitamente la connessione.

Gestione Memoria: Mantenere connessioni attive inutilmente o trattenere in memoria grandi set di dati post-elaborazione rallenta il server e, ad alti carichi, può causare colli di bottiglia o esaurimento delle risorse.

```

1 // --- Approccio MySQLi ---
2 // Libera la memoria specifica del result set
3 mysqli_free_result($result);
4 // Chiude il tunnel di connessione al server MySQL
5 mysqli_close($connection);
6
7 // --- Approccio PDO ---
8 // Elimina l'istanza dell'oggetto PDO, chiudendo implicitamente la connessione
9 $pdo = null;

```

10.6 Gestione di File e Immagini con il Database

Esistono due pattern architetturali diametralmente opposti per l'associazione di file binari (come immagini o documenti) ai record di un database.

10.6.1 1. Salvataggio del Percorso (Consigliato)

Il file binario caricato (es. tramite form multipart) viene intercettato da PHP (attraverso `$_FILES`), rinominato e spostato nel filesystem del web server (es. nella cartella `/images/`). Nel database viene archiviata **esclusivamente la stringa testuale del percorso** associato (es. `/images/983412824.jpg`).

- **Vantaggi:** Il database rimane leggero e i backup rapidi. Il web server serve i file statici con la massima efficienza senza dover invocare l'interprete PHP.
- **Uso:** Il percorso estratto tramite query viene semplicemente stampato nell'attributo `src` di un tag `<img />`.

10.6.2 2. Salvataggio come Dati BLOB (Binary Large Object)

Il file viene tradotto nel suo contenuto binario puro tramite funzioni come `file_get_contents()` e salvato integralmente in una cella del database preconfigurata con il tipo di dato `BLOB`.

- **Svantaggi:** Aumenta esponenzialmente le dimensioni del database, degrada le prestazioni e rallenta i tempi di esportazione/importazione.

Il Problema degli Header HTTP per i file BLOB

Per visualizzare un'immagine memorizzata come BLOB nel browser, non basta stampare i dati con un `echo`. Se lo si facesse, il browser tratterebbe i dati binari come testo, stampando incomprensibili stringhe di caratteri ASCII codificati in ISO/UTF-8. È tassativo utilizzare uno script PHP intermedio (es. `getImage.php?id=1`) che estragga i dati e imposti **gli Header HTTP MIME corretti** immediatamente prima dell'invio del flusso.

```
1 // Esempio logico di estrazione BLOB (es. getImage.php)
2 $statement = $pdo->prepare("SELECT ImageContent FROM TravelImage WHERE ImageID = :id");
3 $statement->bindParam(':id', $_GET['id']);
4 $statement->execute();
5 $result = $statement->fetch(PDO::FETCH_ASSOC);
6
7 if ($result) {
8     // FONDAMENTALE: forza il browser a interpretare lo stream di byte come un'immagine
9     header("Content-type: image/jpeg");
10
11     // Stampa il contenuto binario
12     echo $result["ImageContent"];
13 }
```


11 Validazione(13)

11.1 Validazione dei Dati

La validazione è un processo fondamentale nello sviluppo web per garantire l'integrità, la correttezza e la sicurezza dei dati inseriti dagli utenti. Un'architettura robusta prevede che la validazione avvenga obbligatoriamente su due livelli:

1. **Lato Client:** Migliora la *User Experience* (UX) fornendo feedback immediati senza dover ricaricare la pagina (es. tramite HTML5 o JavaScript). Tuttavia, è facilmente aggirabile da un utente malintenzionato o disabilitando gli script.
2. **Lato Server:** È la vera linea di difesa. Qualsiasi dato in ingresso (da form, URL o API) deve essere considerato potenzialmente non sicuro o errato e va rigorosamente validato prima di essere elaborato o salvato nel database.

11.1.1 Controllo dell'Input in PHP: `isset()` vs `empty()`

Quando i dati arrivano al server tramite le superglobali (`$_GET` o `$_POST`), il primo passo è verificare la loro presenza e il loro stato. Esiste una differenza cruciale tra le funzioni `isset()` ed `empty()`.

La funzione `isset()`

Verifica unicamente se una variabile è stata dichiarata e se il suo valore è diverso da `null`.

- Un parametro inviato vuoto (es. `?name=`) restituisce `true`.
- Un parametro con un solo spazio (es. `?name=%20`) restituisce `true`.
- Restituisce `false` solo se il parametro non è presente nella query string o nel form.

La funzione `empty()` e il problema dello zero

Verifica se una variabile è considerata "vuota". In PHP, sono considerati vuoti (e quindi `empty()` restituisce `true`):

- Una stringa vuota `""`
- Il parametro non impostato (equivale a `!isset()`)
- Il valore numerico `0` o la stringa `"0"`

Quest'ultimo punto è critico: se lo `0` è un valore logicamente valido per il dominio applicativo (es. "quantità di figli: 0"), utilizzare `empty()` per validare la presenza del campo scatterà erroneamente l'input.

Per verificare specificamente che un dato sia di tipo numerico, si utilizza la funzione `is_numeric($valore)`, che accetta sia interi che stringhe numeriche valide.

11.1.2 Criteri di Validazione Lato Server

Un sistema di validazione server-side completo dovrebbe testare l'input contro diverse categorie di regole:

- **Tipo di dato:** È un intero? È una stringa? È un booleano?

- **Lunghezza:** La stringa rispetta un numero minimo e massimo di caratteri?
- **Formato:** Il dato ha la struttura corretta? (es. un'email valida, un codice fiscale). Questo controllo si effettua quasi esclusivamente tramite *Espressioni Regolari (Regex)*.
- **Range:** Se è un numero, è compreso tra i limiti consentiti (es. età compresa tra 18 e 120)?
- **Valori specifici:** Il valore appartiene a una lista chiusa di opzioni valide? (Utilizzando la funzione PHP `in_array($valore, $arrayDiValoriConsentiti)`).
- **Unicità:** Il dato esiste già a sistema? (Richiede un'interrogazione al database, ad esempio per verificare se uno username è già registrato).

11.1.3 Validazione Lato Client con HTML5

HTML5 ha introdotto attributi nativi che delegano al browser una prima validazione semantica e formale, bloccando l'invio del form (`submit`) se i requisiti non sono soddisfatti.

- `required`: Rende il campo obbligatorio.
- `type="number"`, `type="email"`, `type="url"`: Impongono restrizioni di formato intrinseche.
- `min` e `max`: Definiscono il range per i campi numerici o per le date.
- `pattern`: Permette di definire un'espressione regolare direttamente nell'HTML che il valore deve rispettare per essere considerato valido.
- `title`: Se associato a un campo con `pattern`, il browser mostrerà il contenuto del `title` nel tooltip di errore per spiegare all'utente quale formato è richiesto.

```
1 <input type="text" name="country" pattern="[A-Za-z]{3}" title="Inserisci un codice nazione di 3 lettere" required />
```

Disattivare la validazione HTML5 Durante lo sviluppo, per testare la validazione lato server, è molto utile poter bypassare temporaneamente i blocchi del browser. È possibile farlo aggiungendo l'attributo booleano `novalidate` al tag `<form>` (disattiva la validazione per tutto il modulo) oppure l'attributo `formnovalidate` direttamente sul bottone di invio `<input type="submit">`.

11.1.4 Esempi Pratici di Validazione PHP

Nel dettaglio, analizziamo alcuni pattern classici di validazione estratti da casi d'uso reali affrontati a lezione (es. un form per l'ordine di prodotti o spedizioni).

11.1.4.1 Validazione Numerica, Range e Multipli

Quando si accetta un input quantitativo, non basta verificare che la stringa in ingresso sia composta da numeri, ma spesso è necessario applicare logiche di business specifiche (quantità minima, massima, o vincoli di imballaggio).

```
1 $ne = $_GET["ne"]; // Parametro: Numero di uova (Number of eggs)
2
3 // 1. is_numeric: Verifica che l'input sia considerabile un numero
4 // 2. $ne % 6 != 0: Verifica che sia un multiplo di 6 (il resto della divisione deve essere 0)
5 // 3. Verifica i limiti di range (minimo 6, massimo 60)
6 if (!is_numeric($ne) || $ne % 6 != 0 || $ne < 6 || $ne > 60) {
7     echo "Errore: Il numero di uova deve essere 6, 12, 18, ..., 60";
8     die(); // Interrompe bruscamente l'esecuzione dello script
9 }
```

11.1.4.2 Validazione tramite Whitelist (Insiemi Chiusi)

Per campi a scelta limitata (es. province, metodi di spedizione o indirizzi predefiniti), l'approccio più sicuro è verificare che l'input dell'utente coincida esattamente con uno dei valori presenti in un array strettamente controllato dal server (la cosiddetta *Whitelist*).

```
1 $sa = $_GET["sa"]; // Parametro: Indirizzo di spedizione (Shipping address)
2 $indirizzi_consentiti = ["via Diotisalvi", "via Mazzini", "Corso Italia"];
3
4 // in_array() verifica se il valore testuale di $sa è presente nell'array
5 if (!in_array($sa, $indirizzi_consentiti)) {
6     echo "Errore: Spediamo solo ai seguenti indirizzi: ";
7     foreach($indirizzi_consentiti as $addr) {
8         echo $addr . " ";
9     }
10    die();
11 }
```

11.1.5 Espressioni Regolari (Regex) e preg_match()

Per validare formati testuali specifici e rigidi (codici fiscali, CAP, email, numeri di targa), le funzioni standard di PHP non bastano. Si ricorre quindi alle Espressioni Regolari.

****preg_match(\$pattern, stringa) **** : la funzione PHP nativa che esegue una ricerca nella stringa in input per trovare una corrispondenza strutturale con il \$pattern (l'espressione regolare). Restituisce 1 (vero) se c'è un match, 0 (falso) altrimenti.

Sintassi del Pattern: In PHP, l'espressione regolare deve essere sempre racchiusa tra delimitatori, tipicamente lo slash (/). I simboli ^ e \$ indicano rispettivamente l'inizio e la fine della stringa, imponendo che l'intero valore fornito dall'utente rispetti il formato, e non solo una sua sottostringa.

```
1 $pc = $_GET["pc"]; // Parametro: Codice Postale (Postal Code)
2
3 // Regex per un CAP italiano: esattamente 5 cifre numeriche da 0 a 9
4 $pattern_cap = "/^[0-9]{5}$/";
5
6 if (!preg_match($pattern_cap, $pc)) {
7     echo "Errore: IL codice postale (CAP) deve contenere esattamente 5 cifre.";
8     die();
9 }
```

11.1.6 Best Practice: Architettura della Validazione a “Early Exit”

Quando si struttura la pagina PHP (solitamente quella definita nell'attributo `action` del `<form>`), è buona norma seguire un flusso logico sequenziale detto **“Early Exit”** (Uscita anticipata). Questo approccio mantiene il codice estremamente pulito, leggibile ed evita infiniti annidamenti di blocchi `if/else`.

1. **Verifica Presenza:** Controllare subito se i campi obbligatori in `$_GET` o `$_POST` sono vuoti usando `empty()`.
2. **Sanitizzazione:** Ripulire preventivamente l'input, ad esempio eliminando gli spazi bianchi iniziali e finali con `trim()`.
3. **Controlli Specifici:** Applicare i controlli mirati (`is_numeric`, `in_array`, `preg_match`).
4. **Interruzione Immediata:** Appena uno step fallisce, stampare il messaggio di errore e usare `die()` o `exit()` per bloccare il parsing del resto del file.
5. **Successo e Query:** Se il codice sopravvive e arriva in fondo alla lista dei controlli senza mai innescare un `die()`, significa che i dati sono puliti. Solo a questo punto si istanzia la connessione al Database.

Il limite di die() in produzione

Nelle slide e negli script di base, usare `die("Errore")` è comodo e rapido. Tuttavia, in ottica di esame e di User Experience, bloccare il rendering restituendo una pagina bianca con una riga di testo nudo non è ottimale. Una tecnica avanzata e spesso richiesta è salvare le stringhe di errore in un array PHP (es. `$errori[] = "CAP errato";`) per poi re-iniettarle dinamicamente nel markup HTML del form, evidenziando in rosso i campi da correggere.

12 PHP - Gestione degli errori(14)

12.1 Gestione degli Errori e delle Eccezioni in PHP

La gestione degli errori è una componente critica per lo sviluppo di applicazioni web robuste. In PHP, il sistema di error handling distingue storicamente tra **Errori** (problemi procedurali o di configurazione) ed **Eccezioni** (oggetti orientati agli oggetti utilizzati per gestire flussi anomali a runtime).

12.1.1 Tipologie di Problemi e Classificazione

Quando un'applicazione incontra un comportamento inatteso, PHP classifica l'evento in diverse categorie:

- **Errori Previsti (Expected Errors):** Situazioni che si sa potrebbero fallire, come input utente scorretti o un database temporaneamente irraggiungibile. Vanno intercettati proattivamente.
- **Errori di Programmazione:** Variabili non dichiarate, chiamate a funzioni inesistenti o errori di sintassi.
- **Notices e Warnings:** Segnalazioni non bloccanti. Un *Notice* avvisa di un comportamento potenzialmente errato (es. usare una variabile non inizializzata), mentre un *Warning* segnala un problema più serio (es. file mancante in un `include`), ma **non** interrompe l'esecuzione della pagina.
- **Fatal Errors:** Errori gravi che arrestano immediatamente l'esecuzione dello script (es. istanziare una classe inesistente o tentare di richiedere un file essenziale tramite `require` che non si trova).

12.1.2 Configurazione del Reporting (Sviluppo vs Produzione)

PHP offre un controllo estremamente granulare su quali errori mostrare e come registrarli. Queste impostazioni possono essere modificate nel file `php.ini` o dinamicamente all'inizio dello script tramite le funzioni `error_reporting()` e `ini_set()`.

Ambiente di Produzione

Regola d'oro della sicurezza: non mostrare **mai** gli errori in chiaro all'utente finale in un ambiente di produzione. I messaggi di errore espongono dettagli sull'architettura del server, percorsi dei file e nomi delle tabelle del database. In produzione `display_errors` deve essere impostato su `Off`, mentre `log_errors` su `On` per scrivere silenziosamente i problemi in un file di log.

Approccio in fase di Sviluppo: Al contrario, quando si sviluppa localmente, è fondamentale vedere tutto. Impostare `error_reporting(E_ALL)` e `display_errors = On` garantisce che non sfugga alcun warning o notice.

12.1.3 Eccezioni e il Costrutto Try-Catch

Le eccezioni sono oggetti derivati dalla classe base `Exception`. Vengono “lanciate” (**throw**) quando si verifica una condizione anomala e “catturate” (**catch**) in un blocco di codice progettato per gestire l'emergenza, evitando il crash dell'applicazione.

Costrutto Try-Catch: Meccanismo che permette di isolare una porzione di codice potenzialmente problematica (nel blocco **try**). Se si verifica un'eccezione, l'esecuzione salta immediatamente al blocco **catch** corrispondente.

L'oggetto eccezione catturato fornisce metodi utilissimi per il debug:

- `$e->getMessage()`: La descrizione testuale del problema.
- `$e->getCode()`: Un eventuale codice numerico di errore.
- `$e->getFile()` e `$e->getLine()`: Il file e la riga esatta in cui si è generata l'eccezione.

```

1  try {
2      // Codice che potrebbe generare un'eccezione
3      $db = new PDO($dsn, $user, $password);
4
5      // Possiamo lanciare eccezioni personalizzate
6      if ($input_non_valido) {
7          throw new Exception("Valore di input non accettato");
8      }
9  }
10 } catch (Exception $e) {
11     // Gestione pulita dell'errore
12     echo "Si è verificato un problema: " . $e->getMessage();
13 } finally {
14     // Opzionale: codice eseguito SEMPRE, sia in caso di successo che di errore
15     echo "Operazione conclusa.";
16 }
```

12.1.4 La Rivoluzione di PHP 7: L'Interfaccia Throwable

Fino alla versione 5.x, PHP aveva una limitazione fastidiosa: i classici *Fatal Error* non potevano essere intercettati con un blocco try-catch, causando l'arresto brutale dello script.

Con l'introduzione di **PHP 7**, l'architettura è stata ripensata. È stata introdotta l'interfaccia base **Throwable**. Sia la classe **Exception** che la nuova classe **Error** implementano questa interfaccia. Questo significa che molti errori fatali (come un errore di sintassi valutato a runtime o un errore di tipo mancante) ora lanciano un oggetto di classe **Error**, che può essere tranquillamente catturato.

Catture Multiple

È possibile concatenare più blocchi **catch** per gestire in modo diverso eccezioni applicative ed errori di sistema nativi.

```

1  try {
2      // Tentativo di chiamata a un metodo su una variabile non definita o null (Fatal Error)
3      echo $variabile_inesistente->metodoFittizio();
4  }
5  } catch (Exception $e) {
6      // Cattura eccezioni standard (es. errori di database o lanciate dal programmatore)
7      echo "Eccezione Applicativa: " . $e->getMessage();
8  }
9  } catch (Error $e) {
10     // Cattura gli errori fatali del motore PHP che prima avrebbero fatto crashare il sito
11     echo "Errore Fatale di PHP: " . $e->getMessage() . " alla riga " . $e->getLine();
12 }
```

13 JavaScript - Operazioni Asincrone(15)

13.1 Asincronia in JavaScript

JavaScript è un linguaggio orientato agli eventi (event-based). Il codice non viene eseguito in modo rigidamente sequenziale, ma in risposta a specifici eventi, come l'interazione dell'utente (click su un pulsante), lo scadere di un timer o la ricezione di dati dalla rete.

13.1.1 Callbacks ed Eventi

Callback: Una funzione che viene passata come argomento a un'altra funzione. La funzione ricevente “richiama” (calls back) la prima nel momento in cui si verifica l'evento atteso o si conclude un'operazione.

L'uso delle callback è il metodo basilare e storico per gestire le operazioni asincrone in JavaScript.

Delegare l'esecuzione a una callback permette di non bloccare il thread principale del programma, mantenendo il browser reattivo mentre si attende il verificarsi di un evento esterno.

Esempio: Timer e Callback

```
1 let counter = 0;
2
3 function mycallback() {
4   console.log("Counter: " + counter++);
5   if(counter === 5) {
6     clearTimeout(timerhandle); // Interrompe il timer
7   }
8 }
9
10 // La callback viene inserita in coda per l'esecuzione ogni 1000 ms
11 let timerhandle = setInterval(mycallback, 1000);
```

13.2 AJAX e XMLHttpRequest

13.2.1 L'approccio AJAX

AJAX (Asynchronous JavaScript and XML): Un insieme di tecniche di sviluppo lato client utilizzate per creare applicazioni web interattive e dinamiche. Consente di inviare e recuperare dati dal server in modo asincrono, senza interferire con il comportamento della pagina in uso.

Il vantaggio principale dell'approccio AJAX è l'eliminazione del ricaricamento completo del documento HTML. Mentre la richiesta viene gestita in background dal server, l'interfaccia utente rimane pienamente reattiva. Una volta ricevuta la risposta, tramite JavaScript viene aggiornata dinamicamente solo la porzione del DOM interessata dai nuovi dati.

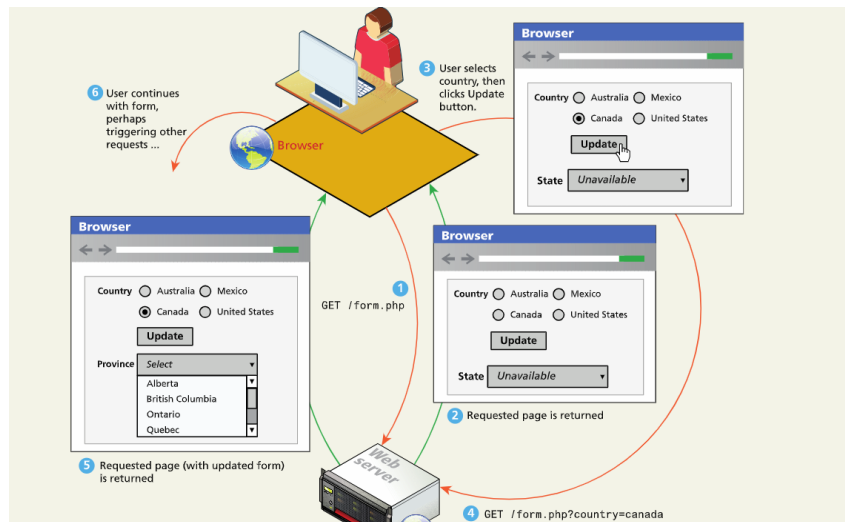


Figura 13.1: Schema “Normal Request Response Loop” vs “The solution” per mostrare visivamente la differenza di ciclo vitale tra sincrono e AJAX.

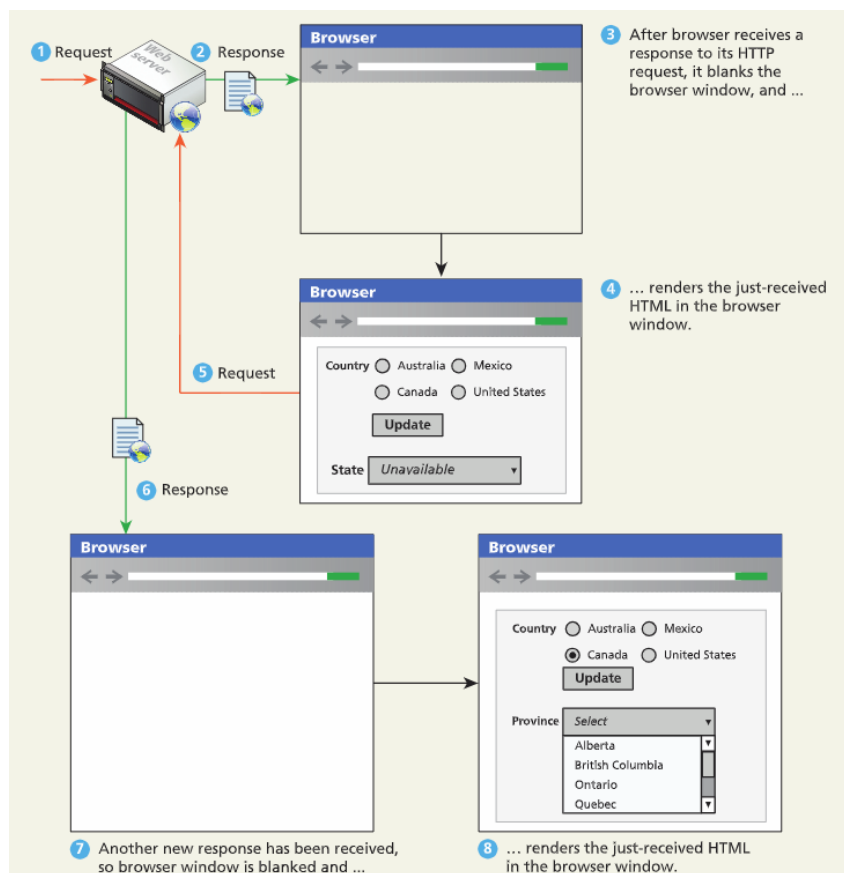


Figura 13.2: Problema

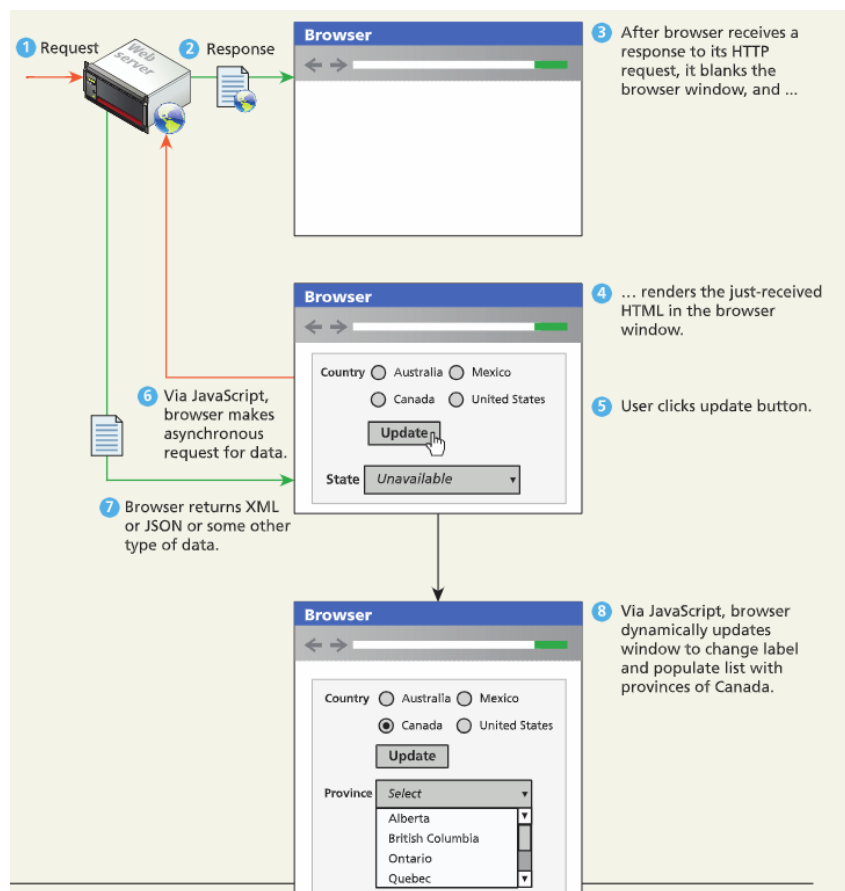


Figura 13.3: Soluzione

13.2.2 L'oggetto XMLHttpRequest

Per effettuare una richiesta HTTP asincrona si utilizza storicamente l'oggetto `XMLHttpRequest`. I dati scambiati possono essere formattati in XML, JSON o testo semplice.

Callback Hell (Pyramid of Doom)

L'uso intensivo di `XMLHttpRequest` e callback per gestire flussi asincroni complessi e propedeutici l'uno all'altro causa un annidamento eccessivo del codice. Questo rende la logica illeggibile, difficile da mantenere e soggetta a errori di gestione, un problema noto come "Callback Hell".

Flusso operativo di XMLHttpRequest

```
1  const x = new XMLHttpRequest();
2  // Inizializza la richiesta specificando metodo e URL
3  x.open("GET", "http://localhost/api/data");
4
5  // Definisce l'azione da eseguire al completamento del trasferimento
6  x.onload = function() {
7    if(x.status === 200) {
8      console.log("Risposta: " + x.response);
9    } else {
10     console.log("Errore: " + x.statusText);
11    }
12  };
13
14 // Definisce l'azione in caso di fallimento della rete
15 x.onerror = function() {
16   console.log("Errore di rete");
17 };
18
19 // Invia effettivamente la richiesta (il metodo ritorna immediatamente)
20 x.send();
```

Per operazioni particolarmente lunghe (es. upload/download di file di grandi dimensioni), è possibile monitorare l'avanzamento registrando una funzione handler sull'evento **onprogress**, che fornisce informazioni sui byte caricati rispetto al totale.

13.3 Promises e Fetch API

13.3.1 Le Promises

L'introduzione delle **Promises** ha rivoluzionato la gestione dell'asincronia in JavaScript, risolvendo di fatto il problema del *Callback Hell* e offrendo una sintassi molto più lineare e gestibile.

Promise: È un oggetto che rappresenta il risultato futuro (il completamento o il fallimento) di un'operazione asincrona e il suo relativo valore.

Una Promise si trova sempre in uno di questi tre stati:

- **Pending:** Stato iniziale, l'operazione non è ancora conclusa.
- **Fulfilled:** L'operazione è stata completata con successo.
- **Rejected:** L'operazione è fallita (es. errore di rete).

Il passaggio dallo stato *pending* a uno degli stati definitivi (*settled*) è irreversibile. Una volta risolta o rigettata, il valore della Promise non cambierà più.

Per associare delle azioni ai possibili esiti di una Promise, si utilizzano dei metodi concatenabili:

- **then()**: Specifica l'handler da eseguire in caso di successo (*fulfilled*).
- **catch()**: Specifica l'handler per gestire l'errore (*rejected*).
- **finally()**: Specifica un handler che verrà eseguito a prescindere dall'esito (utile per operazioni di pulizia e reset dell'interfaccia).

Esempio: Creazione e utilizzo di una Promise

```
1 // Creazione di una Promise
2 let promessa = new Promise((resolve, reject) => {
3   setTimeout(() => {
4     let successo = Math.random() < 0.5;
5     if(successo) {
6       resolve("Operazione completata con successo!");
7     } else {
8       reject("Errore di connessione al server.");
9     }
10  }, 2000);
11 });
12
13 // Consumo della Promise
14 promessa
15   .then(risultato => console.log("Successo: " + risultato))
16   .catch(errore => console.log("Fallimento: " + errore))
17   .finally(() => console.log("Operazione terminata, chiudo la connessione."));
```

13.3.2 La Fetch API

La funzione `fetch` è la versione moderna di `XMLHttpRequest`. Essendo basata nativamente sulle Promises, permette di effettuare richieste di rete in modo rapido, pulito e facilmente concatenabile.

Il doppio passaggio di Fetch

Un errore comune è pensare che `fetch` restituisca immediatamente i dati pronti. In realtà, `fetch` restituisce una prima Promise che si risolve non appena vengono ricevuti gli **header HTTP** della risposta. Per estrarre il corpo della risposta (ad esempio un JSON o un testo), è necessario chiamare un metodo come `.json()` o `.text()` sull'oggetto `Response`. Questi metodi restituiscono a loro volta una **seconda Promise**.

Esempio: Richiesta GET con Fetch

```
1 fetch("https://api.example.com/data")
2   .then(response => {
3     // Controllo esplicito dello status HTTP (fetch non rigetta per errori 404/500)
4     if (!response.ok) {
5       throw new Error("Errore HTTP: " + response.status);
6     }
7     // Restituisce la seconda Promise per il parsing del JSON
8     return response.json();
9   })
10  .then(data => {
11    // I dati parsati sono finalmente pronti per essere manipolati
12    console.log("Dati ricevuti:", data);
13  })
14  .catch(error => {
15    // Cattura errori di rete o eccezioni lanciate nei 'then' precedenti
16    console.error("Problema nella richiesta:", error.message);
17  });
```

13.4 Async/Await

Introdotte con ES2017, le parole chiave `async` e `await` forniscono una sintassi molto più chiara e leggibile per scrivere codice asincrono basato sulle Promise, facendolo sembrare strutturalmente simile al codice sincrono tradizionale.

13.4.1 La keyword `async`

async: Antepoendo `async` alla dichiarazione di una funzione, si garantisce che questa restituisca sempre una **Promise**.

- Se la funzione ritorna un valore normale (non-Promise), questo viene automaticamente avvolto in una Promise risolta (*fulfilled*).
- Se la funzione lancia un'eccezione, viene restituita una Promise rigettata (*rejected*).

13.4.2 La keyword `await`

await: Può essere utilizzata **solo** all'interno di funzioni definite come `async`. Mette in pausa l'esecuzione della funzione asincrona finché la Promise associata non passa a uno stato definitivo (*settled*).

- Se la Promise viene risolta, `await` restituisce direttamente il risultato.
- Se la Promise viene rigettata, `await` lancia un'eccezione, che può essere intercettata con un blocco `try/catch`.

L'uso di `try/catch` combinato con `await` permette di gestire gli errori asincroni con gli stessi costrutti utilizzati per il normale codice sincrono, eliminando la necessità di concatenare i `.catch()`.

Esempio: Fetch API convertita in Async/Await

```
1  async function loadData() {
2      try {
3          console.log("Inizio richiesta...");
4
5          // await attende la risoluzione degli header HTTP
6          let response = await fetch("https://api.example.com/data");
7
8          // Controllo manuale dello status
9          if (!response.ok) {
10             throw new Error("Errore HTTP: " + response.status);
11          }
12
13          // await attende il completamento del download e il parsing del body
14          let data = await response.json();
15
16          console.log("Dati elaborati:", data);
17      } catch (e) {
18          // Intercetta sia errori di rete sia eccezioni lanciate nel try
19          console.error("Si è verificato un problema:", e.message);
20      }
21  }
```

13.5 Il Modello di Esecuzione in JavaScript

13.5.1 L'Event Loop e la Gestione dei Task

JavaScript è intrinsecamente **single-threaded**, il che significa che può eseguire una sola operazione alla volta. La gestione della concorrenza è affidata a un meccanismo chiamato **Event Loop**.

Event Loop: Un ciclo continuo e infinito (`while (true)`) che monitora una coda di operazioni (Task Queue). Se la coda contiene dei task, li preleva uno alla volta e li esegue; se la coda è vuota, attende l'arrivo di nuovi task.

I task vengono inseriti nella coda quando si verificano eventi, ad esempio: l'utente preme un pulsante, scade un timer (es. `setTimeout`), o arriva una risposta dalla rete.

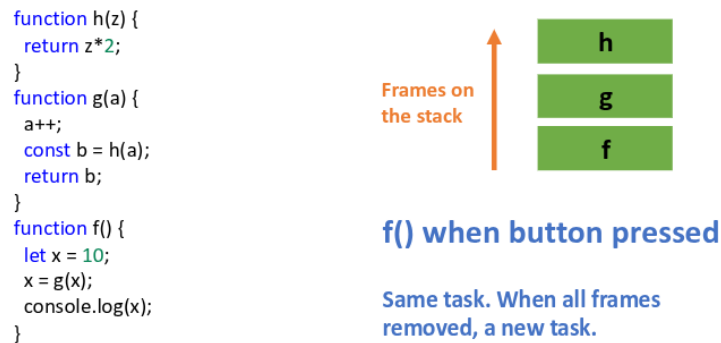


Figura 13.4: Schema del Call Stack (Frames on the stack) per le chiamate sincrone e della Task Queue

13.5.2 Sincronicità e Blocco dell'Interfaccia

- L'esecuzione di una funzione sincrona (e di tutte le sotto-funzioni che essa chiama) costituisce **un singolo task**.
- Finché il task corrente non ha terminato la propria esecuzione (svuotando il call stack), l'Event Loop **non può** passare al task successivo.

Blocco del Browser (Unresponsive GUI)

Se una funzione esegue un'operazione estremamente pesante o un ciclo molto lungo (es. **for** infinito), l'intero thread principale si blocca. Gli altri task nella coda (come i click dell'utente o i timer) non verranno processati e l'interfaccia grafica del browser smetterà di rispondere ("freezing").

Il comportamento reale di `setTimeout`

Anche se impostiamo un timer con ritardo 0 ms, la sua callback non interrompe il codice attualmente in esecuzione. Viene semplicemente inserita nella coda dei task e verrà eseguita solo quando il task corrente (e i task precedenti in coda) saranno completati.

```
1 console.log('Task corrente: inizio');
2
3 setTimeout(() => {
4   console.log('Callback eseguita (Nuovo Task)');
5 }, 0);
6
7 // Simuliamo un blocco sincrono lungo
8 for(let i = 0; i < 1e9; i++) {}
9
10 console.log('Task corrente: fine');
11
12 /* OUTPUT PREVISTO:
13 1. Task corrente: inizio
14 2. Task corrente: fine
15 3. Callback eseguita (Nuovo Task)
16 */
```

Analogamente ai timer, le funzioni gestite tramite `.then()` delle Promise (o il codice dopo un `await`) non vengono eseguite istantaneamente a metà del task corrente, ma vengono schedate per un'esecuzione successiva nel momento in cui il thread si libera, garantendo un ordine prevedibile delle operazioni.

14 Canvas(17)

14.1 HTML5 Canvas: Fondamenti

Canvas: L'elemento HTML `<canvas>` fornisce uno spazio bitmap dipendente dalla risoluzione che può essere utilizzato per renderizzare grafici, grafica di giochi, arte o altre immagini visive in tempo reale tramite script (generalmente JavaScript).

La dimensione dell'area di disegno è controllata esclusivamente dagli attributi `width` e `height`. Sebbene sia possibile applicare regole CSS al canvas (come bordi e margini), le operazioni di disegno interne ereditano il contesto del canvas stesso e alcune proprietà (come colori e font) seguono la normale sintassi CSS.

14.1.1 Inizializzazione e Supporto

Per utilizzare programmaticamente un canvas, è necessario recuperare il suo “contesto” di disegno (per il 2D si utilizza `2d`), eseguire azioni su di esso e infine renderizzarle.

Attenzione: È buona prassi prevedere un testo alternativo all'interno dei tag `<canvas>` per i browser obsoleti e verificare il supporto lato codice prima di eseguire le operazioni di disegno.

```
1 <canvas id="myCanvas" width="200" height="200">
2   Il tuo browser non supporta HTML5 Canvas.
3 </canvas>
```

```
1 try {
2   const canvas = document.getElementById('myCanvas');
3   const context = canvas.getContext('2d');
4   // Logica di disegno...
5 } catch (e) {
6   alert("HTML5 Canvas non è supportato nel tuo browser.");
7 }
```

14.2 Disegno di Percorsi e Trasformazioni

14.2.1 Gestione dei Path

Tutte le forme vettoriali (ad eccezione dei rettangoli che hanno metodi dedicati come `fillRect`) richiedono la definizione di percorsi (path).

Operazioni principali sui Path:

- `beginPath()`: Inizializza un nuovo percorso disconnesso dai precedenti.
- `moveTo(x, y)`: Sposta il cursore alla coordinata indicata senza tracciare linee.
- `lineTo(x, y)`: Crea un segmento lineare dalla posizione corrente alla coordinata specificata.
- `closePath()`: Traccia una retta dalla posizione corrente al punto di origine iniziale del path.
- `stroke()`: Rende effettivamente visibile il percorso tracciandone i contorni.
- `fill()`: Riempie l'area interna definita dal path.

```

1 // Esempio di disegno linea
2 context.beginPath();
3 context.moveTo(100, 100); // Origine del segmento
4 context.lineTo(130, 180); // Destinazione
5 context.lineWidth = 10;
6 context.strokeStyle = '#00ff00'; // Colore CSS
7 context.lineCap = 'round'; // Stili terminali: butt (default), round, square
8 context.stroke();

```

14.2.2 Trasformazioni del Contesto spaziale

Per scrivere codice scalabile e riutilizzabile, la documentazione raccomanda di **disegnare sempre le forme all'origine (0,0)** e applicare successivamente trasformazioni al sistema di coordinate per posizionarle, scalarle o ruotarle.

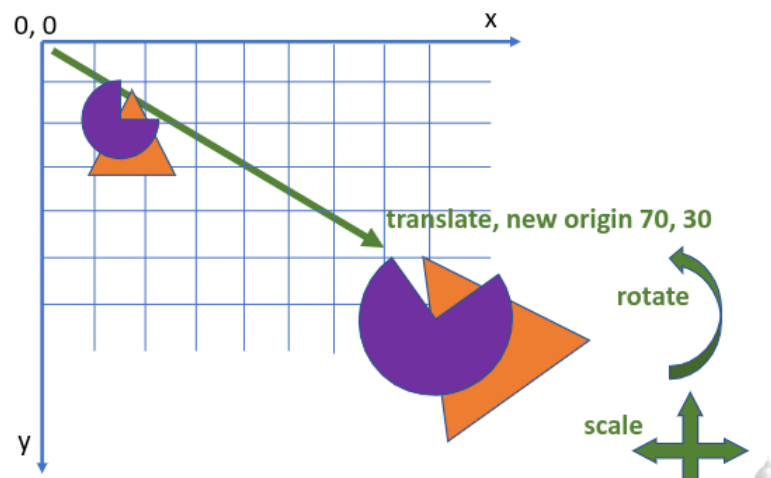


Figura 14.1: Applying Transformations

Le trasformazioni principali includono:

- `translate(x, y)`: Sposta fisicamente l'origine degli assi del canvas.
- `rotate(angle)`: Ruota l'intero sistema di coordinate. L'angolo deve essere espresso in **radianti** (es. 1.57 per 90 gradi). La rotazione avviene sempre perno sull'origine corrente.
- `scale(x, y)`: Moltiplica l'unità di misura degli assi. Se applicato *prima* di posizionare l'oggetto corretto (o se l'oggetto non è disegnato nell'origine), influenzerà anche le sue coordinate spaziali allontanandolo/avvicinandolo.

14.2.3 Isolamento dello Stato: Save e Restore

Le trasformazioni geometriche e l'assegnazione di stili alterano il contesto globale e si accumulano istruzione dopo istruzione. Per evitare che la rotazione di un elemento influenzi tutti gli elementi successivi, è vitale utilizzare lo stack di stato.

context.save() & context.restore(): `save()` memorizza lo stato corrente del contesto (matrice di trasformazione, colori, spessori, font) impilandolo in memoria. `restore()` estrae l'ultimo stato salvato, resettando il contesto esattamente a come era prima del blocco di modifiche.

Se non si salva lo stato prima di una `translate` o `rotate`, le modifiche continueranno ad accumularsi, spingendo potenzialmente gli elementi successivi fuori dall'area visibile del canvas.

```
1 function drawRotatedElement(context) {
2   context.save(); // Salvo lo stato "pulito"
3
4   context.translate(100, 100); // Sposto l'origine dove voglio disegnare
5   context.rotate(1.57); // Ruoto l'asse
6
7   // Disegno partendo dall'origine locale
8   context.beginPath();
9   context.moveTo(0, 0);
10  context.lineTo(30, 80);
11  context.stroke();
12
13  context.restore(); // Annulla la traslazione e la rotazione per le funzioni future
14 }
```

14.3 Stili e Rappresentazione Visiva

14.3.1 Personalizzazione di Tratti e Riempimenti

Oltre a tracciare percorsi geometrici, il Canvas permette di definire in modo dettagliato lo stile visivo delle forme attraverso proprietà del contesto che ricalcano la sintassi CSS.

Proprietà di stile fondamentali:

- `lineWidth`: Imposta lo spessore della linea in pixel.
- `lineJoin`: Definisce la forma degli angoli dove due segmenti si incontrano (es. `round` per arrotondarli).
- `strokeStyle`: Determina il colore o lo stile del contorno della forma.
- `fillStyle`: Determina il colore o lo stile del riempimento interno dell'area.

```
1 // Esempio di applicazione stili
2 context.lineWidth = 4;
3 context.lineJoin = 'round';
4 context.strokeStyle = '#77BB00'; // Colore del bordo
5 context.fillStyle = '#339900'; // Colore di fondo
6
7 context.fill(); // Applica il riempimento al path corrente
8 context.stroke(); // Traccia il contorno rendendolo visibile
```

14.3.2 Disegno di Curve (Quadratic Curves)

Per tracciare percorsi non lineari, l'API offre funzioni matematiche avanzate per generare curve fluide.

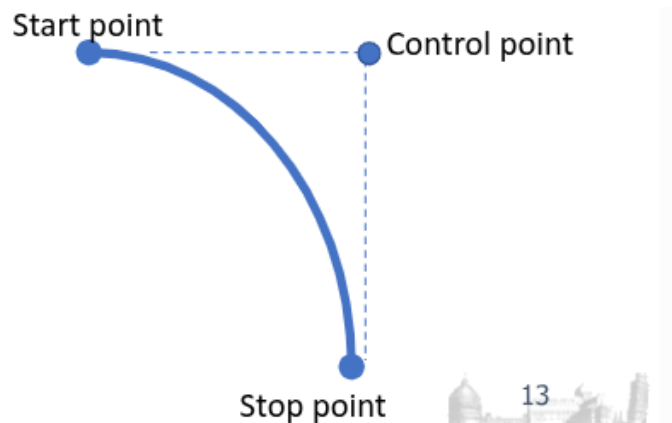


Figura 14.2: Drawing Curves

Curva Quadratica (`quadraticCurveTo`): Una curva determinata da un punto di origine corrente (start point), un punto di destinazione (stop point) e un singolo “punto di controllo” (control point). Quest’ultimo non viene toccato dalla linea, ma agisce come una forza magnetica che “tira” il segmento verso di sé per generare la curvatura.

La sintassi richiesta è `context.quadraticCurveTo(xC, yC, xS, yS);`, dove `(xC, yC)` sono le coordinate del punto di controllo e `(xS, yS)` rappresentano le coordinate del punto di arrivo.

14.4 Immagini, Pattern e Gradienti

14.4.1 Inserimento di Immagini

L'API del Canvas permette di iniettare, scalare e manipolare immagini rasterizzate all'interno dell'area di disegno. Le immagini possono subire le stesse trasformazioni spaziali (rotazione, scala) dei path.

Asincronia del caricamento: Un errore da esame molto comune è tentare di disegnare l'immagine prima che il browser ne abbia completato il download. Prima di invocare il render sul canvas, è imperativo attendere il caricamento completo utilizzando l'evento `onload`.

```
1 const wood = new Image();
2 wood.src = "wood.png";
3
4 // L'operazione di disegno avviene SOLO a caricamento ultimato
5 wood.onload = function () {
6   // drawImage(immagine, coordinataX, coordinataY, larghezza, altezza)
7   context.drawImage(wood, -8, -50, 16, 50);
8 };
```

14.4.2 Pattern di Sfondo

Un'immagine può essere impiegata non solo come elemento a sé stante, ma anche come “trama” ripetibile per riempire una forma o un tracciato.

La funzione `context.createPattern(immagine, regola)` accetta una stringa CSS per definire la ripetizione: `repeat` (default in entrambe le direzioni), `repeat-x`, `repeat-y`, o `no-repeat`. L'oggetto pattern restituito può essere assegnato direttamente a `fillStyle` o `strokeStyle`.

14.4.3 Transizioni: I Gradienti

I gradienti consentono sfumature di colore graduali gestite algebricamente. La loro implementazione richiede un processo in tre step:

1. **Creazione dell'oggetto:** Si definisce l'estensione del gradiente nello spazio.
2. **Assegnazione dei Color Stop:** Si fissano i punti (in percentuale da 0 a 1) in cui il colore cambia lungo la transizione, supportando anche il canale alpha per le trasparenze (rgba).
3. **Applicazione:** Si imposta l'oggetto gradiente come stile di riempimento o contorno.

```
1 // 1. Creo un gradiente lineare orizzontale
2 const trunkGradient = context.createLinearGradient(-8, -50, 16, -50);
3
4 // 2. Aggiungo le tappe di colore
5 trunkGradient.addColorStop(0, '#663300'); // Inizio: marrone scuro
6 trunkGradient.addColorStop(0.2, '#996600'); // Punto 20%: marrone chiaro
7 trunkGradient.addColorStop(1, '#000000'); // Fine: nero
8
9 // 3. Applico come stile di riempimento e colore il rettangolo
10 context.fillStyle = trunkGradient;
11 context.fillRect(-8, -50, 16, 50);
```

14.5 Trasformazioni Matematiche Avanzate

14.5.1 Scaling e Comportamento dell'Origine

Oltre alla traslazione e alla rotazione, il canvas permette di ridimensionare gli elementi.

Attenzione al sistema di riferimento: Operazioni come `scale(x, y)` e `rotate(angle)` agiscono sempre sull'origine corrente. Se scali una forma che *non* è disegnata all'origine (0, 0), la funzione moltiplicherà per il fattore di scala anche le sue coordinate di posizionamento, allontanandola dal centro o spingendola potenzialmente fuori dall'area visibile.

```
1 context.save();
2 // Sposto l'origine dove voglio disegnare
3 context.translate(200, 180);
4 // Scalo l'elemento del 20% in più su entrambi gli assi
5 context.scale(1.2, 1.2);
6 drawTree(context); // Funzione che disegna attorno a 0,0
7 context.restore();
```

14.5.2 La Matrice di Trasformazione (transform)

Per un controllo granulare e combinato sulle trasformazioni spaziali (inclusa l'inclinazione, o *skewing*, non disponibile come funzione singola), si utilizza la manipolazione diretta della matrice di trasformazione affine.

transform(a, b, c, d, e, f): Moltiplica la matrice di trasformazione corrente per la matrice definita dai parametri. La formula matematica alla base calcola le nuove coordinate (x_{new}, y_{new}) in questo modo:

$$\begin{bmatrix} x_{new} \\ y_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Significato dei parametri:

- **a:** Ridimensionamento orizzontale (scale X).
- **b:** Inclinazione orizzontale (skew X).
- **c:** Inclinazione verticale (skew Y).
- **d:** Ridimensionamento verticale (scale Y).
- **e:** Traslazione orizzontale (move X).
- **f:** Traslazione verticale (move Y).

14.6 Testo e Ombreggiature

14.6.1 Renderizzare il Testo

Il Canvas può disegnare stringhe di testo rasterizzate. Esistono due funzioni primarie: `fillText` per il testo pieno e `strokeText` per il solo contorno.

Sintassi: `fillText(text, x, y, [maxwidth])`. Il parametro opzionale `maxwidth` costringe il browser a restringere automaticamente il font se la stringa supera la larghezza massima indicata.

```
1 context.save();
2 context.font = "40px Arial"; // Stile CSS standard
3 context.fillStyle = '#003388';
4 context.textAlign = 'left'; // Allineamento rispetto alla coordinata x data (start, end, left, right, center)
5
6 context.fillText('Progettazione Web', 100, 40);
7 context.restore();
```

14.6.2 Ombre (Shadows)

Le ombreggiature possono essere applicate a tracciati, immagini e testo impostando quattro proprietà specifiche del contesto prima di invocare il comando di disegno (`fill` o `stroke`).

```
1 context.shadowColor = 'rgba(0, 0, 0, 0.3)'; // Colore con trasparenza
2 context.shadowOffsetX = -5; // Spostamento orizzontale (negativo = sinistra)
3 context.shadowOffsetY = -5; // Spostamento verticale (negativo = alto)
4 context.shadowBlur = 3; // Grado di sfocatura (Gaussiana)
5
6 context.fillText('Testo con Ombra', 100, 40);
```

14.7 Animazioni ed Eventi Interattivi

Il Canvas nativamente crea immagini statiche “usa e getta”. Per creare un’animazione o un gioco interattivo, è necessario instaurare un ciclo di rendering continuo (Game Loop) sincronizzato con il refresh del monitor.

`window.requestAnimationFrame(callback)`: È il metodo moderno per gestire le animazioni web. Comunica al browser che si desidera eseguire un'animazione e richiede che il browser chiami la funzione specificata (callback) per aggiornare un'animazione prima del prossimo repaint dello schermo. Sostituisce i vecchi ed inefficienti `setInterval`.

Ciclo di vita dell'animazione:

1. Pulizia del Canvas (cancellare il frame precedente con `clearRect` o coprendolo con un `fillRect` di sfondo).
2. Aggiornamento logico delle variabili (posizioni, angoli, calcoli).
3. Salvataggio dello stato (`save`).
4. Disegno dei nuovi frame con i nuovi parametri.
5. Ripristino dello stato (`restore`).
6. Richiamo ricorsivo di `requestAnimationFrame`.

```
1 // Esempio logico di Game Loop e ascolto eventi
2 let angle = 0;
3 let direction = 0;
4 const canvas = document.getElementById('canvas');
5 const ctx = canvas.getContext('2d');
6
7 // 1. Gestione Eventi (Input dell'utente)
8 document.addEventListener('keydown', (event) => {
9   if (event.key === 'a') direction = -1;
10  else if (event.key === 'd') direction = 1;
11 });
12
13 document.addEventListener('keyup', () => { direction = 0; });
14
15 // 2. Loop di Render
16 function draw() {
17   // Pulisco/ridisegno lo sfondo
18   ctx.fillStyle = "black";
19   ctx.fillRect(0, 0, canvas.width, canvas.height);
20
21   // Aggiorno la logica (la direzione modifica l'angolo ad ogni frame)
22   angle += direction * 0.05;
23
24   // Disegno l'elemento
25   ctx.save();
26   ctx.translate(canvas.width / 2, canvas.height / 2);
27   ctx.rotate(angle);
28
29   ctx.fillStyle = "white";
30   ctx.fillRect(-25, -25, 50, 50); // Esempio: un quadrato ruotante
31
32   ctx.restore();
33
34   // 3. Richiedo il prossimo frame
35   window.requestAnimationFrame(draw);
36 }
37
38 // Avvio l'animazione
39 draw();
```


15 Gestione stato(18)

15.1 Gestione dello Stato nelle Applicazioni Web

15.1.1 Il Problema dello Stato

Stato (State): La capacità di un'applicazione web di mantenere traccia delle informazioni, delle selezioni o dell'identità di un utente attraverso diverse interazioni o richieste HTTP.

A differenza delle classiche applicazioni desktop, che risiedono stabilmente in memoria all'interno di un singolo processo unificato, le applicazioni web operano tramite una serie di richieste HTTP disconnesse tra loro. Il protocollo HTTP è intrinsecamente *stateless* (senza stato): non distingue nativamente due richieste provenienti dalla stessa fonte rispetto a due richieste provenienti da fonti diverse.

Senza un intervento a livello di programmazione, per il web server è impossibile sapere se l'utente che sta richiedendo la pagina del checkout (`checkout.php`) è lo stesso che ha appena aggiunto un articolo al carrello sulla pagina dei prodotti (`product.php`).

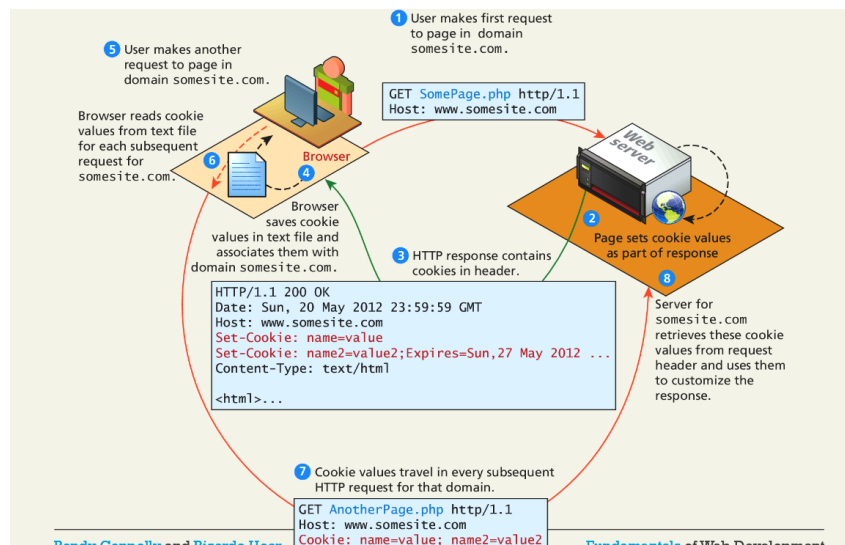


Figura 15.1: Schema di funzionamento base dei Cookie tra Browser e Web Server (ciclo request/response).

15.1.2 Metodi di Trasferimento delle Informazioni

Per risolvere il problema dello stato, sono disponibili meccanismi in grado di trasferire informazioni tra client e server durante le richieste HTTP. Oltre ai sistemi più complessi (come le sessioni, affrontate in seguito), le due modalità più immediate sono le Query String e i Cookie.

15.1.2.1 Query String (Metodo GET e POST)

I dati vengono passati in modo esplicito nella richiesta HTTP che il browser invia al server.

- **GET:** Le variabili vengono accodate direttamente all'URL (es. `process.php?artist=Picasso&year=1906`). Questo approccio è visibile all'utente, ha limiti di lunghezza e non è adatto a dati sensibili.
- **POST:** Le informazioni viaggiano nel corpo della richiesta HTTP (es. header `Content-Type: application/x-www-form-urlencoded`), garantendo maggiore capienza e pulizia dell'URL, molto utilizzato nell'invio dei form.

15.1.3 I Cookie (Approccio Client-Side)

Cookie: Piccoli file di testo contenenti coppie `nome=valore` inviati da un sito web e memorizzati nel browser dell'utente. Costituiscono un approccio lato client per la persistenza dello stato.

Anche se memorizzati sul browser, i dati dei cookie viaggiano costantemente all'interno degli **header HTTP**:

1. Il server invia un header `Set-Cookie` nella sua risposta HTTP.
2. Il browser salva il valore localmente in associazione al dominio.
3. Per ogni successiva richiesta verso quel dominio, il browser inietta automaticamente un header `Cookie` contenente i dati salvati, permettendo al server di riconoscere lo stato.

Attenzione all'esame e alla sicurezza: I siti non devono fare affidamento esclusivo sui cookie per funzioni ipercritiche, in quanto l'utente può cancellarli, disabilitarli o manometterli. Non inserire mai dati sensibili in chiaro.

15.1.3.1 Anatomia e Opzioni di un Cookie

La sintassi completa impostata dal server tramite header prevede varie regole: `Set-Cookie: nome=valore[; expires=data][; domain=dominio][; path=percorso][; secure]`

- **Expire (Scadenza):** Determina la durata di vita del cookie.
 - *Session Cookie:* Senza una scadenza dichiarata, il cookie viene eliminato alla chiusura della sessione del browser.
 - *Persistent Cookie:* Viene fornita una data temporale (timestamp); il cookie sopravvive sul disco del client finché non scade.
- **Domain (Dominio):** Indica a quali domini inviare il cookie. Di default corrisponde all'host che lo ha generato. È utile per condividere lo stato tra sottodomini (es. impostando `.yahoo.com`, il cookie sarà inviato a `finance.yahoo.com`, `my.yahoo.com`, ecc.).
- **Path (Percorso):** Limita l'invio del cookie solo alle risorse che si trovano sotto un certo path dell'URL (es. `/blog`).
- **Secure:** Se settato, indica al browser di trasmettere il cookie al server solo ed esclusivamente in presenza di una connessione cifrata sicura (HTTPS).

15.1.3.2 Gestione dei Cookie in PHP

Il PHP fornisce la funzione integrata `setcookie()` e l'array superglobale `$_COOKIE`.

Restrizione di Protocollo Cruciale: Poiché i cookie sono istruzioni passate tramite gli header HTTP, la chiamata a `setcookie()` deve obbligatoriamente avvenire **prima** di aver prodotto qualsiasi output (inclusi tag HTML come `<html>` o anche semplici spazi vuoti). In caso contrario, lo script PHP genererà un errore fatale del tipo *Headers already sent*.

```
1 <?php
2 // 1. Scrittura di un Cookie
3 // Calcolo della scadenza: momento attuale + 1 giorno (60s * 60m * 24h)
4 $expiryTime = time() + (60 * 60 * 24);
5
6 // Imposta nome, valore, scadenza, path, dominio e flag secure
7 setcookie("Username", "Ricardo", $expiryTime, "/~mario/", "unipi.it", true);
8
9 // 2. Lettura di un Cookie
10 // Prima di utilizzarlo, è sempre prassi verificare che esista
11 if(isset($_COOKIE['Username'])) {
12     echo "L'utente recuperato dal cookie è: " . $_COOKIE['Username'];
13 } else {
14     // Gestione del caso in cui non ci sia cookie valido
15     echo "Nessun cookie trovato.";
16 }
17 ?>
```

15.2 Serializzazione degli Oggetti

15.2.1 Concetto e Funzionamento

Serializzazione: Il processo di conversione di un oggetto complesso in una sequenza di byte (stringa binaria) per consentirne la memorizzazione persistente o la trasmissione. La **deserializzazione** è il processo inverso che restituisce l'oggetto originale.

In PHP, questo meccanismo è fondamentale per mantenere lo stato tra diverse richieste HTTP. Poiché ogni richiesta è un'esecuzione separata del programma, gli oggetti andrebbero persi se non venissero salvati in una forma "congelata".

- `serialize($obj)`: Restituisce una stringa contenente il flusso di byte che rappresenta il valore dell'oggetto, incluse tutte le sue variabili.
- `unserialize($str)`: Utilizza la stringa per ricreare i valori originali delle variabili e istanziare nuovamente l'oggetto.

Per un corretto funzionamento di `unserialize()`, la definizione della classe dell'oggetto deve essere già stata caricata (tramite `include` o `require`) prima della chiamata alla funzione.

15.2.2 L'interfaccia Serializable

In scenari complessi, potrebbe essere necessario personalizzare il modo in cui un oggetto viene serializzato (ad esempio, escludendo connessioni al database o risorse temporanee).

Esempio di Implementazione:

```
1 class Artista implements Serializable {
2     private $nome;
3     private $opere;
4
5     public function serialize() {
6         // Definiamo esattamente cosa salvare in un array
7         return serialize([
8             'n' => $this->nome,
9             'o' => $this->opere
10        ]);
11    }
12
13    public function unserialize($data) {
14        // Recuperiamo i dati e riassegniamo le proprietà
15        $dati = unserialize($data);
16        $this->nome = $dati['n'];
17        $this->opere = $dati['o'];
18    }
19 }
```

15.3 Session State (Stato di Sessione)

15.3.1 Funzionamento Interno

Session State: Un meccanismo di stato basato sul server che consente alle applicazioni web di memorizzare e recuperare oggetti di qualsiasi tipo per ogni sessione utente unica.

A differenza dei cookie, i dati della sessione risiedono sul **server** (in memoria o su disco). Il collegamento tra l'utente e i suoi dati avviene tramite un **Session ID**:

1. Al primo avvio, PHP genera un ID unico di 32 caratteri.
2. Questo ID viene inviato al browser tramite un cookie di sessione (solitamente chiamato `PHPSESSID`).
3. In ogni richiesta successiva, il browser invia l'ID; il server lo usa per ricaricare il "dizionario" di dati associato a quell'utente specifico.

In PHP, la variabile superglobale per gestire questi dati è `$_SESSION`. È obbligatorio chiamare `session_start()` all'inizio di ogni script per abilitare il caricamento dei dati.

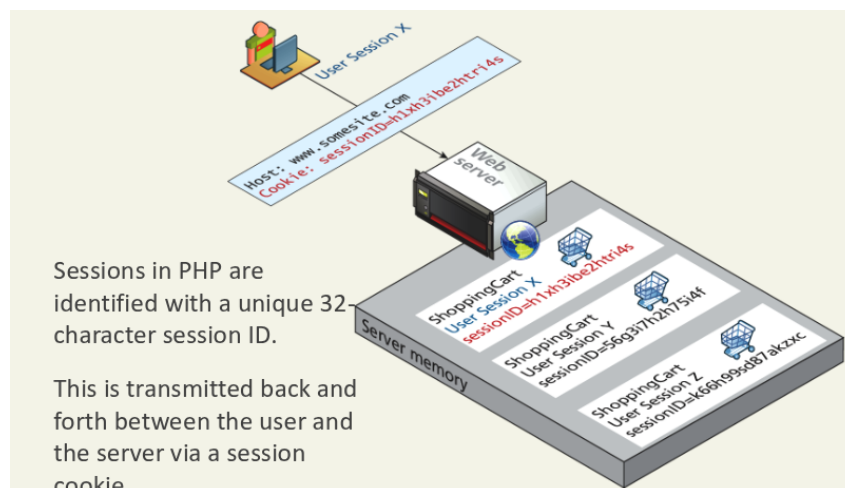


Figura 15.2: Schema del collegamento tra SessionID nel cookie e ShoppingCart nella memoria del server.

15.3.2 Sessioni in Ambienti Distribuiti (Web Farm)

Nelle applicazioni ad alto volume che utilizzano più server (Web Farm) dietro un bilanciatore di carico (**Load Balancer**), sorge un problema: se la prima richiesta va al Server A e la seconda al Server B, il Server B non troverà i dati della sessione salvati localmente su A.

Soluzioni comuni:

1. **Sticky Sessions (Sessioni Appiccicose):** Configurare il bilanciatore per inviare tutte le richieste di una sessione allo stesso server.
2. **Shared Storage (Archivio Condiviso):** Utilizzare una posizione centralizzata accessibile a tutti i server, come un database o un server **Memcached**.

Nel file `php.ini`, è possibile configurare il gestore di salvataggio: `session.save_handler = memcache`.

15.4 Web Storage HTML5

Introdotti con HTML5, `localStorage` e `sessionStorage` sono API JavaScript che integrano i cookie per la persistenza lato client, ma con differenze sostanziali.

- **localStorage:** I dati persistono anche dopo la chiusura del browser (senza scadenza prefissata).
- **sessionStorage:** I dati vengono persi al termine della sessione (chiusura del tab/finestra).

Differenze con i Cookie:

- **Capacità:** I cookie sono limitati a circa 4KB; il Web Storage permette diversi MB (solitamente 5MB+).
- **Traffico:** I cookie vengono inviati al server con *ogni* richiesta HTTP. I dati del Web Storage restano sul client e sono accessibili solo via JavaScript, riducendo il carico di rete.

15.4.0.1 Esempio Pratico (JavaScript)

```
1 // Scrittura
2 localStorage.setItem("username", "Ricardo");
3 sessionStorage.setItem("session_start", new Date());
4
5 // Lettura
6 const user = localStorage.getItem("username");
7 if (user) {
8     console.log("Benvenuto " + user);
9 }
```

Un uso pratico del Web Storage è il caching di contenuti statici o dati XML/JSON scaricati asincronamente (AJAX), per evitare di ripetere richieste pesanti ai servizi web durante la stessa sessione.

16 Accessibilità(19)

16.1 Introduzione all'Accessibilità Web

L'accessibilità web è il risultato dell'interazione tra diversi componenti fondamentali: i contenuti (testo, codice, markup), gli user agent (browser, lettori multimediali), le tecnologie assistive (screen reader, tastiere alternative), gli utenti (con le loro strategie adattive), gli sviluppatori e gli strumenti di authoring o valutazione.

WAI (Web Accessibility Initiative): Iniziativa del W3C che fornisce linee guida riconosciute a livello internazionale come standard per l'accessibilità del web.

Le linee guida principali si dividono in tre macro-categorie:

- **WCAG** (Web Content Accessibility Guidelines): Focalizzate sull'accessibilità dei contenuti web.
- **UAAG** (User Agent Accessibility Guidelines): Indirizzate a browser e agenti utente.
- **ATAG** (Authoring Tool Accessibility Guidelines): Dedicate agli strumenti per la creazione dei contenuti.

16.1.1 I Principi delle WCAG 2.2 (POUR)

Le WCAG 2.2 si basano su 13 linee guida organizzate in 4 principi fondamentali, noti con l'acronimo POUR. Per ogni linea guida esistono dei criteri di successo (Success Criteria - SC) verificabili, suddivisi in tre livelli di conformità: **A** (minimo), **AA** (intermedio) e **AAA** (massimo).

Attenzione Legislativa: In Unione Europea, chiunque offra servizi al pubblico (non solo enti pubblici, ma anche privati che gestiscono e-commerce, banking, telecomunicazioni) deve garantire che i propri siti web raggiungano almeno il livello di conformità **AA**.

16.1.1.1 Percepibile (Perceivable)

Le informazioni e i componenti dell'interfaccia devono essere presentati in modo che gli utenti possano percepirli con i propri sensi, senza risultare "invisibili".

- **Alternative Testuali (SC 1.1):** Fornire testo alternativo per contenuti non testuali (es. immagini) in modo che possa essere convertito in Braille o letto da sintesi vocale.
- **Media Temporizzati (SC 1.2):** Fornire alternative come sottotitoli o audiodescrizioni per flussi audio e video.
- **Adattabile (SC 1.3):** Contenuti presentabili in modi diversi (es. layout semplificato o lettura lineare) senza perdere struttura o informazioni.
- **Distinguibile (SC 1.4):** Separare chiaramente il primo piano dallo sfondo, curando il contrasto e non affidando la comunicazione delle informazioni unicamente all'uso del colore.

16.1.1.2 Utilizzabile (Operable)

I componenti dell'interfaccia e la navigazione non devono richiedere azioni fisiche che l'utente non è in grado di compiere.

- **Accessibile da Tastiera (SC 2.1):** Tutte le funzionalità devono essere usabili solo con la tastiera, evitando la “trappola della tastiera” (il focus non deve mai rimanere bloccato in un singolo elemento).
- **Tempo Adeguato (SC 2.2):** Fornire tempo sufficiente per leggere e interagire. Se un'animazione o uno scorrimento dura più di 5 secondi, l'utente deve poterlo mettere in pausa o nascondere.
- **Epilessia e Reazioni Fisiche (SC 2.3):** Evitare animazioni improvvise o lampeggiamenti eccessivi (non più di tre lampeggiamenti al secondo).
- **Navigabile (SC 2.4):** Fornire strumenti chiari per orientarsi e trovare contenuti facilmente.

16.1.1.3 Comprensibile (Understandable)

Le informazioni e le operazioni dell'interfaccia non devono risultare oltre la capacità di comprensione dell'utente.

- **Leggibile (SC 3.1):** Testo chiaro e linguisticamente definito (es. tramite attributi della lingua).
- **Prevedibile (SC 3.2):** L'aspetto e il funzionamento delle pagine devono comportarsi in modo coerente e come ci si aspetta.
- **Assistenza (SC 3.3):** Aiutare l'utente a evitare o correggere errori durante l'inserimento di dati, fornendo feedback chiari.

16.1.1.4 Robusto (Robust)

I contenuti devono essere tecnicamente solidi.

- **Compatibilità (SC 4.1):** Mantenere l'aderenza agli standard web (HTML/CSS validi) per garantire che il codice venga interpretato correttamente da una vasta gamma di user agent, incluse le tecnologie assistive attuali e future.

I criteri di successo specifici (A, AA, AAA) indicano la criticità della barriera. Ad esempio, fornire un'alternativa testuale base è livello A (assolutamente necessario), mentre fornire l'audiodescrizione per un video preregistrato è livello AA.

16.2 WAI-ARIA (Accessible Rich Internet Applications)

WAI-ARIA: È una specifica del W3C che definisce un insieme di attributi HTML aggiuntivi. Viene utilizzata per fornire semantica extra agli elementi e migliorare l'accessibilità laddove l'HTML nativo risulta carente.

I concetti principali su cui si basa ARIA si dividono in tre categorie: Ruoli, Proprietà e Stati.

16.2.1 Ruoli (Roles)

I ruoli definiscono cosa un elemento è o la funzione che svolge all'interno dell'interfaccia. Erano strettamente necessari prima dell'avvento di HTML5, mentre oggi vanno usati **solo se strettamente necessari** (ovvero quando non esiste un tag semantico adeguato).

- **Ruoli Widget:** Definiscono componenti interattivi personalizzati.
 - `role="button"`: Rende un `div` o uno `span` identificabile come pulsante.

- Altri esempi: `checkbox`, `slider`, `tablist`, `tab`.
- **Ruoli Strutturali:** Strutturano visivamente il documento.
 - `role="article"`: Sezione autonoma (emula `<article>`).
 - `role="heading"`: Titolo (spesso accompagnato da `aria-level` per definirne il grado, emulando `h1-h6`).
 - `role="list"` / `role="listitem"`: Emulano `ul` / `li`.
- **Ruoli Landmark:** Definiscono i punti di riferimento della pagina, duplicando in gran parte il valore dei tag HTML5.
 - `banner` (`<header>`), `navigation` (`<nav>`), `main` (`<main>`), `contentinfo` (`<footer>`), `search` (`<search>`).
- **Ruoli Live Region:** Gestiscono annunci dinamici.
 - `alert`: Messaggio urgente, letto immediatamente.
 - `status`: Informazione di stato, letta in modo discreto.
 - `progressbar`: Comunica lo stato di avanzamento di un'operazione.

16.2.2 Proprietà e Stati

Le **proprietà** descrivono le caratteristiche immutabili o le relazioni tra gli elementi.

- `aria-label`: Fornisce un'etichetta testuale invisibile (utile quando non c'è testo visibile a schermo).
- `aria-labelledby`: Collega un elemento a un ID testuale complesso per fargli da etichetta.
- `aria-describedby`: Fornisce informazioni aggiuntive collegando l'elemento all'ID di un testo esplicativo (es. requisiti di una password).
- `aria-required`: Indica che un input è obbligatorio.
- `aria-current`: Indica la posizione corrente (es. la pagina attiva in una paginazione).

Gli **stati** descrivono la condizione temporanea di un elemento e cambiano dinamicamente tramite JavaScript.

- `aria-checked`: Indica se una casella (o un toggle) è spuntata.
- `aria-expanded`: Indica se un menu a tendina o un pannello è aperto o chiuso.
- `aria-selected`: Indica se un elemento in una lista custom è selezionato.
- `aria-hidden`: Nasconde l'elemento alle tecnologie assistive (utile per icone puramente decorative).
- `aria-disabled`: Indica che il componente è disabilitato e non interagibile.

Esempio di accessibilità su un componente personalizzato: Se si crea un pulsante custom con un `div`, non basta aggiungere un evento click. Per renderlo accessibile, bisogna aggiungere ruolo, gestione da tastiera ed etichetta testuale.

```

1 <div class="pulsante-custom"
2   role="button"
3   tabindex="0"
4   aria-label="Salva i dati nel database"
5   onclick="salvaDati()"
6   onkeydown="gestisciTasto()">
7   Salva
8 </div>
```

Suggerimenti e Anti-pattern ARIA:

1. Non usare un ruolo ARIA se esiste l'elemento HTML nativo (es. sbagliato: `<a role="button">`, corretto: `<button>`).
2. Evita la ridondanza (es. sbagliato: `<ul role="list">`).

16.3 Gestione del Focus e Navigazione

16.3.1 L'attributo tabindex

Il `tabindex` controlla se e come un elemento riceve il focus durante la navigazione sequenziale tramite il tasto `Tab`.

- **tabindex positivo** (es. 1, 2): Imposta un ordine forzato. Gli elementi vengono navigati in ordine numerico crescente prima di ogni altro elemento.
- **Elementi nativi interattivi**: (`<a>`, `<button>`, `<input>`) ricevano il focus nel naturale ordine in cui appaiono nel DOM.
- **tabindex="0"**: Rende un elemento non interattivo (come un `div`) capace di ricevere il focus, inserendolo nel normale flusso del DOM insieme agli elementi nativi.
- **tabindex="-1"**: Rimuove l'elemento dalla navigazione sequenziale tramite `Tab`, ma permette che riceva il focus via JavaScript (es. `element.focus()`).

Valori Positivi Sconsigliati: È fortemente sconsigliato utilizzare valori di `tabindex` positivi, in quanto la sequenza logica di navigazione con la tastiera deve sempre rispecchiare l'ordine visivo e il flusso naturale del DOM.

16.4 ARIA Live Regions

Nelle applicazioni web moderne (SPA, caricamenti asincroni), l'aggiornamento dei contenuti tramite DOM (come notifiche, caricamenti o messaggi di errore) non viene rilevato automaticamente dagli screen reader.

Live Regions: Sono porzioni di pagina configurate per segnalare alle tecnologie assistive i cambiamenti dinamici, permettendo loro di annunciare l'aggiornamento all'utente senza richiedere il ricaricamento della pagina.

Si definiscono principalmente tramite l'attributo `aria-live`, che accetta tre livelli di priorità:

1. **off**: (Default) Gli aggiornamenti non vengono annunciati.
2. **polite**: L'annuncio viene messo in coda e letto dallo screen reader non appena l'utente smette di interagire con la pagina.
3. **assertive**: Interrompe immediatamente la lettura corrente dello screen reader per annunciare un messaggio critico (da usare con parsimonia, es. errori gravi o timeout).

È pratica comune preparare un contenitore vuoto nel DOM con attributo `aria-live` già assegnato. Quando JavaScript inietterà il testo all'interno di quel contenitore, lo screen reader ne rileverà il cambiamento e lo annuncerà in base al livello di cortesia impostato.

16.5 Contesto, Lingua e Strumenti di Analisi

16.5.1 Lingua del Documento e Ordine Logico

Segnalare correttamente la lingua è cruciale affinché lo screen reader utilizzi il motore di sintesi vocale adeguato. L'attributo globale `lang` (es. `lang="en"`) va inserito nel tag root `<html>`. Se ci sono porzioni di testo in lingue diverse all'interno della stessa pagina, l'attributo `lang` può essere applicato inline su elementi specifici come un `<span>`.

Inoltre, il codice sorgente deve sempre avere un ordine logico. Le tecnologie assistive leggono linearmente la struttura del DOM, per cui è fondamentale non stravolgere la logica di lettura abusando dei posizionamenti CSS (è possibile testare questo aspetto disabilitando completamente gli stili CSS della pagina).

16.5.2 Strumenti per la Verifica dell'Accessibilità (Auditing)

Per garantire conformità e robustezza, si utilizzano diversi strumenti analitici:

- **Verifica del Contrasto:** Tool che analizzano la ratio di contrasto tra testo e background per soddisfare i criteri WCAG (es. WebAIM Contrast Checker). È importante, in caso di errori form validation, non fare affidamento unicamente sul colore rosso per segnalare uno stato di errore, ma accoppiarvi icone o testo.
- **WAVE:** Estensione web per l'ispezione visuale dei problemi di accessibilità.
- **Accessibility Tree (Albero dell'Accessibilità):** Gli ispettori integrati nei browser (DevTools in Chrome o Firefox) mettono a disposizione l'albero accessibilità. Questo schema derivato dal DOM mostra esattamente quali "nomi", "ruoli", "stati" e "proprietà" vengono esposti e tradotti dal browser per essere inviati allo screen reader.

17 JavaScript - Funzioni Avanzate(20)

17.1 Funzioni Avanzate degli Array

JavaScript offre diverse funzioni integrate per iterare, cercare e manipolare gli array seguendo un approccio dichiarativo e funzionale. Queste funzioni operano richiamando una funzione di callback per ogni elemento dell'array.

17.1.1 Iterazione con `forEach`

`forEach()`: Esegue una funzione di callback specifica una volta per ogni elemento presente all'interno dell'array.

È lo strumento ideale quando si desidera semplicemente scorrere un array per eseguire operazioni di side-effect (come la stampa su console o l'aggiornamento dell'interfaccia utente), senza la necessità di restituire un nuovo array o alterare quello originale.

```
1 const paintings = [  
2   {title: "Girl with a Pearl Earring", artist: "Vermeer"},  
3   {title: "Sunflowers", artist: "Van Gogh"}  
4 ];  
5  
6 paintings.forEach(p => {  
7   console.log(p.title + " by " + p.artist);  
8 });
```

17.1.2 Ricerca: `find` e `filter`

`find()`: Restituisce il *primo* elemento nell'array che soddisfa la condizione implementata nella funzione di callback. Se nessun elemento corrisponde, restituisce `undefined`.

`filter()`: Crea e restituisce un *nuovo array* contenente tutti gli elementi che superano il test della funzione di callback.

Entrambi i metodi richiedono che la funzione passata come argomento restituisca un valore booleano (`true` o `false`). In scenari più complessi, all'interno della callback è possibile sfruttare costrutti avanzati come le espressioni regolari per effettuare ricerche sofisticate sui dati.

```
1 // Cerca solo il primo match  
2 const courbet = paintings.find(p => p.artist === 'Courbet');  
3  
4 // Cerca tutti i match, restituendo un nuovo array  
5 const vangoghs = paintings.filter(p => p.artist === 'Van Gogh');
```

17.1.3 Trasformazione con map

map(): Scorre l'array originale e crea un nuovo array della medesima dimensione. Ogni elemento del nuovo array è il risultato della funzione di trasformazione applicata all'elemento corrispondente.

Focus Progettazione: `map()` è estremamente utile per la manipolazione dinamica del DOM. Un tipico pattern d'esame prevede l'utilizzo di `map()` per trasformare un array di oggetti JSON crudi in un array di elementi HTML (es. tag `<li>`) o in nodi DOM reali pronti per essere iniettati nella pagina.

```
1 // Generazione di un array di nodi DOM a partire dai dati
2 const options = paintings.map(p => {
3   let item = document.createElement("li");
4   item.textContent = `${p.title} (${p.artist})`;
5   return item;
6 });
```

17.1.4 Aggregazione con reduce

reduce(): Esegue una funzione “riduttore” su ciascun elemento dell'array, elaborandoli uno alla volta e accumulandoli per restituire in output un singolo valore finale.

La funzione di callback di `reduce()` richiede due parametri fondamentali: l'accumulatore (che conserva il risultato parziale) e l'elemento corrente. È buona norma (e previene bug inaspettati) fornire sempre un **valore iniziale** per l'accumulatore come secondo argomento del metodo `reduce()`.

```
1 const myarr = [10, 20, 30, 40];
2 const valoreIniziale = 0;
3
4 const somma = myarr.reduce((accumulatore, corrente) => {
5   return accumulatore + corrente;
6 }, valoreIniziale);
7
8 // Output: 100
```

17.1.5 Ordinamento Personalizzato con sort

sort(): Ordina gli elementi di un array in place. Il comportamento predefinito converte gli elementi in stringhe e li ordina in senso crescente.

Per ordinare array di oggetti (es. in base a una proprietà numerica o alfabetica), è essenziale fornire a `sort()` una funzione di comparazione personalizzata. Questa funzione confronta due elementi (**a** e **b**) e deve restituire: * Un valore `< 0` se **a** deve precedere **b**. * Un valore `> 0` se **b** deve precedere **a**. * `0` se sono equivalenti.

```
1 const artworks = [
2   {title: "AAA", year: 1999},
3   {title: "BBB", year: 1980},
4   {title: "CCC", year: 1720}
5 ];
6
7 // Ordinamento crescente per anno utilizzando una sintassi a freccia concisa
8 const sortedAsc = artworks.sort((a, b) => a.year - b.year);
```

17.2 Moduli JavaScript (ES6)

Nelle applicazioni Web moderne, che includono librerie esterne e centinaia di file, sorge il problema dei **Name Conflicts** (conflitti di nomenclatura): funzioni o variabili con lo stesso nome si sovrascrivono silenziosamente, causando bug critici. JavaScript (ES6) risolve questo problema attraverso l'implementazione dei moduli.

Modulo ES6: Un file contenente codice JavaScript in cui tutte le variabili, funzioni e classi definite sono, per impostazione predefinita, private e isolate all'interno del modulo stesso (scoping del modulo locale).

Per istruire il browser a trattare uno script come un modulo (abilitando l'isolamento e le istruzioni import/export), è **obbligatorio** specificare l'attributo `type="module"` nel tag `<script>` del documento HTML.

```
1 <script src="main.js" type="module"></script>
```

17.2.1 Export: Condividere il Codice

Per rendere accessibili le funzioni o le variabili all'esterno del proprio modulo, si utilizza la parola chiave `export`. Esistono diverse sintassi operative:

- **Esportazione Diretta:** Inserendo la keyword `export` immediatamente prima della dichiarazione.
- **Esportazione Raggruppata:** Elencando tutti gli identificatori da esportare alla fine del file.
- **Rinominazione (Alias):** Modificando il nome con cui l'identificatore viene esportato tramite la keyword `as`.
- **Default Export:** Unica esportazione predefinita per file, molto utilizzata nei moduli che espongono un'unica classe o funzionalità centrale.

```
1 // --- File: mathUtils.js ---
2
3 // 1. Esportazione diretta
4 export const VOTO_MAX = 30;
5
6 // Funzione privata (non esportata)
7 function logInterno() { console.log("Log di sistema"); }
8
9 function quadrato(x) { return x * x; }
10 function moltiplica(a, b) { return a * b; }
11
12 // 2. Esportazione raggruppata e 3. Rinominazione
13 export { quadrato, moltiplica as eseguiMoltiplicazione };
14
15 // 4. Default Export
16 export default function(a, b) { return a + b; }
```

17.2.2 Import: Integrare il Codice

Per incorporare e utilizzare il codice esportato da altri moduli, si sfrutta la parola chiave `import`.

Attenzione in fase d'esame: Non mescolare mai l'importazione di un "default export" con l'importazione di elementi nominati all'interno della stessa destrutturazione. Il default import *non* richiede le parentesi graffe `{}`.

```
1 // --- File: main.js ---
2
3 // Importazione Default (nessuna graffa, il nome può essere arbitrario)
4 import somma from './mathUtils.js';
5
6 // Importazione Nominale (nomi esatti racchiusi tra graffe, possibilità di usare 'as')
7 import { VOTO_MAX, eseguiMoltiplicazione as multi } from './mathUtils.js';
8
9 // Namespace Import (importa tutto raggruppato in un singolo oggetto)
```

```
10 import * as MathLib from './mathUtils.js';
11
12 console.log(MathLib.quadrato(5)); // Output: 25
13 console.log(MathLib.VOTO_MAX);   // Output: 30
```

17.3 Web API (Integrate ed Esterne)

Le API (Application Programming Interfaces) permettono di estendere notevolmente le funzionalità del client. Si dividono in **API Integrate** (nativamente supportate dal browser) ed **API Esterne** (fornite da terze parti e da includere esplicitamente).

17.3.1 Web Speech API (Integrata)

Web Speech API: Interfaccia nativa del browser che fornisce funzionalità hardware di sintesi vocale (Text-to-Speech) e riconoscimento vocale (Speech-to-Text).

```
1 // Esempio base di Text-to-Speech
2 const frase = new SpeechSynthesisUtterance('La progettazione Web è meravigliosa');
3 const synth = window.speechSynthesis;
4
5 // Avvia la riproduzione tramite il motore vocale del sistema operativo
6 synth.speak(frase);
```

17.3.2 Geolocation API (Integrata)

Fornisce un'interfaccia sicura per richiedere le coordinate geografiche (latitudine e longitudine) del dispositivo dell'utente.

Le API di geolocalizzazione funzionano in modo asincrono. Necessitano l'approvazione esplicita dell'utente tramite un prompt di sicurezza del browser e, nei browser moderni, richiedono tassativamente un contesto sicuro (connessione HTTPS).

```
1 if (navigator.geolocation) {
2   navigator.geolocation.getCurrentPosition(
3     (position) => {
4       const lat = position.coords.latitude;
5       const lon = position.coords.longitude;
6       console.log(`Posizione rilevata - Lat: ${lat}, Lon: ${lon}`);
7     },
8     (error) => {
9       console.error("Geolocalizzazione fallita o negata", error);
10    }
11  );
12 }
```

17.3.3 API Esterne: Google Maps e Plotly.js

L'utilizzo di API esterne obbliga all'inclusione di file script provenienti da server di terze parti (CDN).

Le API esterne rappresentano un'esternalità fuori dal proprio controllo aziendale. Versioni datate o snippet di codice presi da libri obsoleti potrebbero cessare di funzionare a causa di aggiornamenti lato server dei provider (es. nuove policy o meccanismi di autenticazione API key, come nel caso storico di Google Maps).

Google Maps: Per istanziare una mappa è necessaria l'inclusione asincrona (`async defer`) dello script ufficiale di Google contenente una chiave API valida, abbinata a un elemento contenitore `<div id="map"></div>` nel DOM, le cui dimensioni spaziali devono essere rigorosamente forzate tramite CSS.

Plotly.js: Libreria open-source d'eccellenza per la Data Visualization. Consente di iniettare dinamicamente grafici SVG interattivi partendo da un array di dati.

```
1 // Esempio concettuale di Plotly.js (Richiede inclusione della CDN plotly.js)
2 window.addEventListener("load", () => {
3   const data = [
4     {
5       x: [4, 5, 6, 7, 8, 9, 10, 11],
6       y: [23, 25, 13, 15, 10, 13, 17, 20]
7     }
8   ];
9
10  const layout = { title: 'Simple Line Chart' };
11  const options = { responsive: true };
12
13  // "chartDiv" corrisponde all'ID del contenitore HTML vuoto
14  Plotly.newPlot("chartDiv", data, layout, options);
15 });
```


18 Sicurezza Web(21)

18.1 Sicurezza dell'Informazione

18.1.1 La Triade CIA

La Triade CIA: Rappresenta i tre principi fondamentali della sicurezza delle informazioni.

- **Confidentiality (Riservatezza):** Mantenere la privacy dei dati archiviati e trasmessi. Previene la divulgazione delle informazioni a terzi non autorizzati.
- **Integrity (Integrità):** Assicurare che i dati siano accurati, corretti e non subiscano alterazioni non autorizzate.
- **Availability (Disponibilità):** Garantire che le informazioni e i sistemi siano sempre accessibili alle persone autorizzate nel momento del bisogno.

Una violazione della riservatezza può consistere in un furto di dati da un database (es. carte di credito). La perdita di integrità si manifesta quando un utente malintenzionato manipola informazioni nel sistema, come alterare un ordine o un indirizzo. La perdita di disponibilità si verifica comunemente tramite attacchi Denial of Service (DoS) che bloccano l'accesso agli utenti legittimi.

18.1.2 Concetti Chiave: Vulnerabilità, Minaccia, Rischio e Attacco

- **Vulnerabilità (Vulnerability):** Un difetto o una debolezza nella progettazione, implementazione o gestione di un sistema (es. software non aggiornato, mancanza di validazione degli input, configurazioni hardware o umane errate).
- **Minaccia (Threat):** Il potenziale di violazione della sicurezza. È un possibile pericolo, un evento o un'azione che potrebbe sfruttare una vulnerabilità per causare danni (es. hacker, dipendenti malintenzionati, malware, disastri ambientali).
- **Rischio (Risk):** L'aspettativa di perdita, espressa come la probabilità che una minaccia specifica sfrutti una data vulnerabilità, unita alla gravità del risultato dannoso.
- **Attacco (Attack):** Un assalto intenzionale e deliberato alla sicurezza del sistema da parte di una minaccia intelligente (es. SQL Injection, DDoS, Phishing).

Esempio della catena causale: Un server database non aggiornato (**Vulnerabilità**) viene individuato da un criminale informatico durante una scansione (**Minaccia**). Questo scenario genera un'alta probabilità di violazione critica dei dati (**Rischio**), che culmina nel momento in cui l'hacker esegue codice remoto sfruttando il difetto per rubare informazioni (**Attacco**).

18.2 Secure By Design

L'approccio *Secure By Design* si basa sul presupposto che esistano utenti malintenzionati. Impone di mantenere un atteggiamento di continua diffidenza verso gli input utente (e persino i valori di sistema interni) durante tutte le fasi del ciclo di vita del software.

18.2.1 Fasi del Ciclo di Sviluppo Sicuro

L'integrazione della sicurezza non è un'aggiunta finale, ma deve avvenire in ogni stadio:

- **Requisiti:** Definizione delle esigenze di privacy e stesura della security policy (basata sulla Triade CIA).
- **Progettazione (Design):** Valutazione delle minacce (Threat assessment), analisi dei rischi e pianificazione delle ridondanze.
- **Implementazione:** Pratiche di programmazione difensiva, revisioni del codice (code review) e pair programming.
- **Testing:** Scrittura di test case, unit test focalizzati sulla sicurezza e test di vulnerabilità.
- **Distribuzione (Deployment):** Penetration testing ("attacca te stesso") e rimozione assoluta di eventuali valori di configurazione di default.

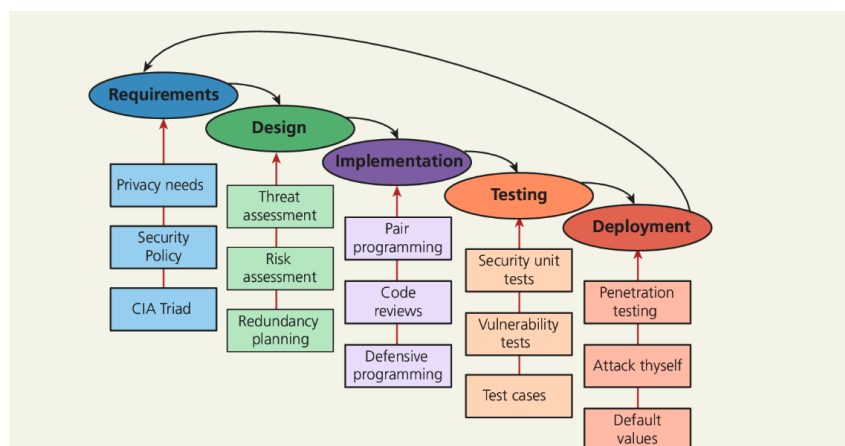


Figura 18.1: Diagramma delle fasi Secure By Design

18.3 Autenticazione e Autorizzazione

18.3.1 Fattori di Autenticazione

Autenticazione: Il processo volto a validare che un utente sia effettivamente chi dichiara di essere. Si basa sulla verifica di uno o più fattori:

- **Cosa sai (Knowledge):** Password, PIN, risposte a domande di sicurezza.
- **Cosa hai (Ownership):** Dispositivi fisici come smartphone, smart card, token crittografici.
- **Cosa sei (Inherence):** Caratteristiche biometriche come impronte digitali, scansione della retina o geometria del volto.

MFA (Multi-Factor Authentication): L'autenticazione a singolo fattore (SFA) è vulnerabile e debole. I sistemi moderni richiedono l'autenticazione a più fattori, in cui devono essere validati almeno due fattori distinti (es. combinazione di password e token inviato sullo smartphone) per garantire l'accesso.

18.3.2 Autorizzazione e Principio del Privilegio Minimo

Attenzione a non confondere i termini: l'autenticazione **garantisce l'accesso** al sistema, l'autorizzazione definisce **cosa l'utente può fare** una volta entrato.

Principio del privilegio minimo (Principle of Least Privilege): Regola fondamentale di sicurezza che impone di fornire agli utenti (e alle singole porzioni di software) esclusivamente i diritti e i privilegi strettamente necessari per completare le proprie attività, riducendo così la superficie di attacco.

18.3.3 Autenticazione di Terze Parti e OAuth

Molti servizi permettono di delegare la verifica dell'identità a provider esterni (come Google o Facebook). **OAuth** è un protocollo standard per l'autorizzazione sicura basata su token.

I quattro ruoli in OAuth:

- **Resource Owner:** L'utente finale titolare dei dati.
- **Resource Server:** Il server in cui risiedono effettivamente i dati protetti.
- **Client:** L'applicazione web che sta richiedendo l'accesso ai dati per conto del Resource Owner.
- **Authorization Server:** Il sistema che autentica l'utente e rilascia i token di accesso (spesso integrato con il Resource Server).

Il flusso di lavoro OAuth:

1. Il Client reindirizza l'utente all'Authorization Server, comunicando il proprio identificativo (`client_id`) e una URL di callback.
2. L'utente si autentica sul server di terze parti (se non lo è già) e autorizza l'app.
3. L'Authorization Server reindirizza l'utente di nuovo al Client fornendo un **Authorization Code**.
4. Il Client, lavorando in background (server-to-server), scambia questo codice e il proprio segreto (`secret`) per ottenere un **Access Token**.
5. Il Client usa l'Access Token per richiedere la risorsa protetta al Resource Server.

18.4 Crittografia

La crittografia risolve il problema della trasmissione sicura dei dati tra un mittente (Alice) e un destinatario (Bob), prevenendo le intercettazioni da parte di terzi (Eve).

18.4.1 Cifrari Storici e Crittografia Simmetrica

I cifrari simmetrici utilizzano la **stessa chiave** sia per crittografare che per decrittografare il messaggio.

- **Cifrario di Cesare:** Cifrario a sostituzione che sposta le lettere dell'alfabeto di un numero fisso.
- **Problema della distribuzione:** I cifrari semplici sono vulnerabili all'analisi delle frequenze (es. in inglese la "E" compare con il 12% di frequenza). Per essere sicuro, un cifrario deve appiattire questa distribuzione.
- **Cifrario di Vigenère:** Utilizza una parola chiave per alterare la sostituzione, nel tentativo di nascondere la distribuzione delle lettere.

18.4.2 Cifrari a Blocchi Moderni

Elaborano il messaggio suddividendolo in blocchi di bit e applicando sostituzioni e permutazioni iterative (round).

- **DES (Data Encryption Standard):** Algoritmo storico, elabora blocchi da 64 bit con chiavi da 56 bit ed esegue 16 round di operazioni (incluso lo XOR con sotto-chiavi e le S-boxes). Ormai deprecato.
- **AES (Advanced Encryption Standard):** Sostituto del DES. Utilizza blocchi da 128 bit e chiavi da 128, 192 o 256 bit. È esponenzialmente più sicuro contro attacchi brute force.

18.4.3 Il Problema dello Scambio delle Chiavi e Diffie-Hellman

Il Problema della Chiave Simmetrica: Come possono Alice e Bob concordare una chiave segreta comunicando su un canale insicuro?

Scambio di chiavi Diffie-Hellman: Sfrutta l'aritmetica modulare per creare un segreto condiviso allo scoperto.

1. Alice e Bob concordano un numero primo p e un generatore g .
2. Alice sceglie un intero casuale a e invia a Bob $g^a \bmod p$.
3. Bob sceglie un intero casuale b e invia ad Alice $g^b \bmod p$.
4. Entrambi possono calcolare il segreto condiviso usando la regola delle potenze:

$$(g^b)^a \bmod p = (g^a)^b \bmod p = g^{ab} \bmod p$$

18.4.4 Crittografia a Chiave Pubblica (Asimmetrica) e RSA

Risolve in modo definitivo il problema dello scambio chiavi utilizzando una coppia di chiavi:

- **Chiave Pubblica (K^+):** Distribuita a tutti, usata per *crittografare*.
- **Chiave Privata (K^-):** Segreta, usata per *decrittografare*.

L'algoritmo RSA: Si basa sulla difficoltà computazionale di fattorizzare numeri primi molto grandi.

1. Si scelgono due grandi numeri primi p e q .
2. Si calcola $n = pq$ e $z = (p-1)(q-1)$.
3. Si sceglie e coprimo con z .
4. Si sceglie d tale che $ed \bmod z = 1$.

- Chiave pubblica: (n, e)
- Chiave privata: (n, d)

Matematica di RSA: Cifratura:

$$c = m^e \bmod n$$

Decifratura:

$$m = c^d \bmod n$$

Per comunicazioni reali, RSA è computazionalmente oneroso. Si utilizza quindi RSA solo per stabilire una connessione sicura iniziale, tramite la quale scambiarsi una **Session Key** (chiave simmetrica, es. AES) che verrà poi usata per la trasmissione dei dati, essendo molto più veloce.

18.5 Firme Digitali e HTTPS

18.5.1 Firme Digitali e Funzioni di Hash

Le firme digitali garantiscono tre aspetti: **Autenticità** (chi l'ha inviato?), **Integrità** (è stato modificato?) e **Non ripudio**. Si basano sulle funzioni di Hash (come MD5 o SHA-1) che generano un "digest" univoco a partire da un messaggio.

Processo di firma:

1. Alice calcola l'hash del messaggio.
2. Alice **crittografa l'hash con la propria chiave privata** (creando la firma).
3. Alice invia messaggio e firma a Bob.
4. Bob calcola l'hash del messaggio ricevuto.
5. Bob **decrittografa la firma usando la chiave pubblica** di Alice.
6. Se l'hash decrittografato corrisponde a quello calcolato da Bob, il messaggio è autentico e integro.

18.5.2 HTTPS e Certificate Authorities

HTTPS è il protocollo HTTP incapsulato su Transport Layer Security (TLS). Per evitare attacchi Man-in-the-Middle, Bob deve essere sicuro che la chiave pubblica ricevuta appartenga effettivamente al server (es. Google) e non a un truffatore.

Certificate Authority (CA): Una terza parte fidata e indipendente che "lega" (bind) una chiave pubblica a un'entità specifica tramite un **Certificato** (es. standard X.509). La CA firma il certificato contenente la chiave pubblica del server usando la propria chiave privata.

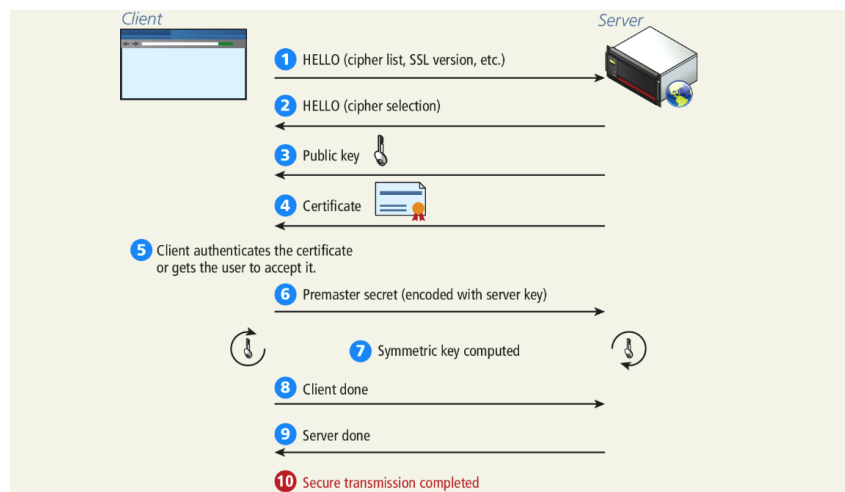


Figura 18.2: Secure Handshake HTTPS

Certificati Self-Signed: Offrono la stessa cifratura di quelli emessi da una CA, ma il browser mostrerà un avviso di sicurezza (“Connessione non affidabile”) poiché l’identità del server non è validata da un ente terzo.

18.6 Security Best Practices e Attacchi Comuni

18.6.1 1. Archiviazione Sicura dei Dati (Password)

MAI salvare le password in chiaro nel database. Una compromissione del DB esporrebbe tutte le credenziali.

- **Hashing:** Salvare solo l’hash della password (es. con MD5 o SHA-256). Quando l’utente fa login, si fa l’hash dell’input e lo si confronta con quello salvato.
- **Salting:** L’hashing semplice è vulnerabile agli attacchi “Rainbow Table” (ricerche inverse tramite Google o database di hash precalcolati). La soluzione è il **Salt**: aggiungere una stringa casuale (unica per ogni utente) alla password prima di farne l’hash, memorizzando il salt nel DB accanto all’hash.

18.6.2 2. SQL Injection (SQLi)

Avviene quando input non sanitizzati vengono concatenati direttamente nelle query SQL, permettendo a un utente di alterarne la struttura logica.

```
1 // Esempio VULNERABILE
2 $user = $_POST['username'];
3 // Se $user = ''; TRUNCATE TABLE Users; #" l'attaccante svuota il DB
4 $sql = "SELECT * FROM Users WHERE uname='$user'";
```

18.6.3 3. Cross-Site Scripting (XSS)

L’attaccante inietta script JavaScript malevoli in una pagina web, che verranno eseguiti dal browser della vittima (es. per rubare i cookie di sessione).

- **Reflected XSS (Non persistente):** Lo script malevolo è passato tramite URL (parametri GET) e il server lo “riflette” direttamente nella pagina di risposta. Richiede l’interazione della vittima (es. click su link malevolo in un’email).
- **Stored XSS (Persistente):** Molto più grave. Lo script malevolo viene salvato nel database del server (es. nei commenti di un blog) e viene eseguito automaticamente da tutti gli utenti che visitano la pagina infetta.

Difese contro XSS:

1. **Filtrare l’input utente:** Rimuovere i tag pericolosi (usando librerie come HTMLPurifier o sanitizer, `strip_tags` di PHP è spesso insufficiente).
2. **Escaping dell’output:** Convertire i caratteri speciali in entità HTML (es. `<` diventa `<`) prima di stampare a schermo variabili provenienti dal DB, rendendo lo script inoffensivo e letto solo come testo.

18.6.4 4. Insecure Direct Object Reference

Avviene quando i riferimenti interni agli oggetti (es. nomi dei file, ID utente) vengono esposti direttamente nell’URL. Un utente malevolo potrebbe dedurre la sequenza e accedere a dati non suoi (es. da `/images/10.jpg` prova ad accedere a `/images/11.jpg`). **Difesa:** Usare ID offuscati/hash generati casualmente (es. `9a76eb...jpg`) oppure instradare la richiesta di download attraverso uno script PHP che controlla l’autorizzazione di sessione prima di restituire il file.

18.6.5 5. Denial of Service (DoS e DDoS)

- **DoS:** Attacco volto a sovraccaricare il server di richieste per impedirne il funzionamento regolare, bloccando gli utenti legittimi.
- **DDoS (Distributed DoS):** Le richieste arrivano da macchine multiple (spesso botnet di computer infettati a insaputa dei proprietari). Difficile da bloccare. **Difesa:** Uso di cache aggressiva per le pagine dinamiche e dimensionamento/ridondanza adeguata della banda (oltre all’uso di servizi specializzati come Cloudflare).

